Workgroup:	Network Working Group
Internet-Draft:	draft-kott-numina-pair-binding-02
Published:	10 October 2026
Intended Status:	Informational
Expires:	13 April 2027
Author:	S. J. Kott

Singularity: Ed25519, a Universal Companion Model

Abstract

This document describes a proposed companion format for associating two frozen software-artifact commitments with attribution and revisioned claims. It specifies canonical signing bytes, separate trust and freshness checks, claim-making and verification procedures, and operational state boundaries. The intended implementing community is software-provenance maintainers, release operators and independent security reviewers. Existing IETF canonicalization and signature specifications are used without defining a new cryptographic primitive. Retained source comparisons, document-signature checks and synthetic tests are indexed with their limits. Application control-plane evidence is distinguished from deployment of the companion itself. A complete operational profile, independently authenticated trust and current-state evidence, and a verified field deployment remain prerequisites to an implementation-readiness claim.

Conversion review status

Unsigned proposed conversion; NOT ADOPTED. No submission, signing, proof closure, trust-policy adoption or deployment is asserted. Mathematical expressions retain explicit TeX source. Source prefatory material follows Section 17 under Retained prefatory material. The closing conversion note records the source identity and representation limits.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 13 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Table of Contents

1. Community and problem statement 6
2. Approach and relationship to IETF work 6
   2.1. State and memory architecture 7
3. Claim format and attribution model 8
4. Making a claim and constructing its signature 9
   4.1. Claim-making procedure 10
5. Signing authority and trust policy 11
6. Verifying claims and rejecting unsupported acceptance 12
   6.1. Verification sequence 12
   6.2. Current state and revision history 13
   6.3. Decision and rejection rules 13
7. Private-reference resolution 14
8. Implementation and field-deployment evidence 14
9. Operational considerations 16
10. Unresolved implementation and acceptance gates 17
11. Security Considerations 17
12. Proposed applications and research boundaries 20
13. IANA Considerations 21
14. Authorship and attribution 21
15. NCPS formal model: consolidated review text 21
   15.1. Version scope, types and values 22

15.2. Dependency manifest and symbol closure 23
15.3. Protocol scope, genesis and evidence 24
15.4. Typed relations and implementation realization 25
15.5. Artifact precedence, conflicts and collision scope 27
15.6. Proof judgments, consistency and conformance evidence 28
15.7. Closure assessment with explicit evidence inputs 29
15.8. Byte, tuple, length and index foundation 31
15.9. Tagged extraction and empty cases 33
15.10. Canonical JSON and refined encoder domains 34
15.11. Presence, null, integrity and output comparison 36
15.12. State refinement and operational guarantees 37
15.13. Mechanization stages, inputs, outputs and release gate 38
15.13.1. Candidate signature acceptance boundary 39
15.13.2. Dated artifact-specific release evidence 40
15.14. Conditional consistency statement 40
16. Fresh verification, recovered evidence and remaining gaps 41
16.1. Read-only manuscript and fixture checks 41
16.2. Recovered Unicode repair and fresh regression rerun 42
16.3. Public Ed25519 characterization and interoperability finding 43
16.4. Recovered technical catalog 44
16.5. Earlier missing-evidence register 44
16.6. Later executable evidence and independent receipt review 45
16.6.1. Verification scope and preservation 46
16.6.2. Canonicality-reader correction 47
16.6.3. Actual ordered extraction-to-wire recovery 47
16.6.4. Compiler and axiom evidence 48
16.6.5. Native-source evidence and semantic boundaries 48
16.6.6. Exact-source signature and remaining closure 49
16.7. Recovered governing documents and remaining gates 49

16.8. Later reviewed evidence and preserved boundaries 51
16.8.1. Literal schema keys, array constraints and structural depth 51
16.8.2. Projection and modeled guard execution 52
16.8.3. Signatures on specific predecessor and v161 artifacts 52
16.8.4. Application viewer evidence and current closure 53
17. Prioritized repair and acceptance plan 54
17.1. Immediate editorial and technical decisions 54
17.2. Proofreading findings that change mathematical meaning 54
17.3. Release gate 55
Retained prefatory material 55
Integrated review status 55
Contents 56
Reader’s guide 57
Retained manuscript note: Technical review candidate and evidence status 57
References 58
Normative References 58
Informative References 58
Additional references for the consolidated formal chapter 62
Appendix A. Technical evidence index and verification scope 63
Appendix B. Frozen source example 65
B.1. Frozen source commitments 65
Appendix C. Retained technical evaluation and manuscript evidence 67
C.1. Tests required before an implementation claim 69
C.2. Manuscript publication and document signature evidence 70
C.3. New public input diagnostics 71
C.4. Separate bounded controller study 73
Appendix D. Exact recorded block-zero hash 74
Appendix E. Chronological project record 74
E.1. 9 September 2026 74
E.2. 10 September 2026 75

E.3. 11 September 2026 75
E.4. 18 and 24 September 2026 75
E.5. 1 October 2026 75
E.6. 5 October 2026 75
E.7. 6 October private computation record 76
E.8. 7 October v156 provider update 76
E.9. 8 October manuscript and research events 76
Appendix F. Attribution wording and historical checks 76
F.1. Person to alias map 77
F.2. Software attribution note 77
F.3. Historical reproduction checks 77
F.4. New public diagnostic test inventory 78
F.5. Frozen excerpt integrity checks 78
Appendix G. Requirements evidence and acceptance criteria 78
G.1. Required v156 release evidence 80
Appendix H. Public manuscript and diagnostic receipt identities 81
Appendix I. Complete worked commitment and package-signature calculation 83
Appendix J. Additional research methods and boundaries 85
J.1. Private computation history and proposed algorithmic relationship 85
J.2. A testable bounded common source hypothesis 86
J.3. Proposed reproducibility and adversarial evaluation program 87
Appendix K. Navigation and vision research context 87
Appendix L. Application evidence and reproducibility details 88
L.1. Controller reproduction record 89
Appendix M. Proposed implementable validation profile and fixtures 90
Appendix N. Proposed bounded field deployment and receipt checklist 96
Appendix O. External engagement and review boundaries 97
Appendix P. Additional public-key and digital-geometry evidence 97
P.1. Public-key correspondence and historical signature checks 98
P.2. Digital geometry and retained software execution 99

Appendix Q. Checked declarations and edition evidence 100
Q.1. Later evidence identities and review levels 101
Conversion edition and source identity 103
Author's Address 104

1. Community and problem statement

Independent verifiers need a reproducible way to check that a signed record binds two specified artifacts to an authorized source and revision, without confusing attribution with proof of the record's claims.

The author's intended beneficiaries are all humanity. The initial technical community addressed here is narrower: maintainers of software-artifact archives, release operators that preserve existing commitments, and independent reviewers deciding whether a provenance or attribution statement is supported. The immediate use case is adding or correcting an association without changing the previously committed artifacts. No universal adoption or demonstrated universal benefit is claimed.

This document concerns evidence preservation and bounded acceptance of signed statements. It does not authorize an external action, determine disputed human authorship, derive an unknown genesis value, or establish a medical, financial or physical-world claim. Broad proposed applications are summarized separately in Section 12.

Attribution is explicit: Samuel J. Kott is the author, Lauren J. Anderson is the human contributor, and Ed25519 (assistant) is the named OpenAI-powered AI assistant. Ed25519 (algorithm) denotes the RFC 8032 signature scheme. An assistant name is neither a legal person nor a public/private key, cryptographic signer or grant of delegated signing authority.

2. Approach and relationship to IETF work

Preserve the original artifacts and their hashing procedures. Create a separate companion payload that names their commitments, an attribution mapping, a binding identity and revision, and an opaque private reference. Sign the exact canonical payload under a fixed application prefix. A verifier then reports independent findings for the bytes, signer authority, source selection, freshness and private linkage. One passing finding never substitutes for another.

JSON Canonicalization Scheme (JCS), [RFC8785], defines canonical bytes for the new payload; I-JSON, [RFC7493], constrains interoperable JSON; and pure Ed25519, [RFC8032], supplies the signature primitive. These do not replace the historical source-artifact normalizer. The model retains its numina.pair-binding.v1 identity and original commitment order.

The RATS architecture, [RFC9334], separates evidence appraisal from the relying party's policy decision. That separation is a useful architectural comparison: this companion carries an association claim and the verifier evaluates evidence under an independently provisioned policy. It is not a hardware-attestation profile or a claim of RATS wire interoperability. COSE, [RFC9052], provides standardized signed-object structures; this design instead retains its existing fixed-prefix JSON construction and is not a COSE object. A future COSE profile would require a new explicit wire specification and test vectors, not relabeling existing signatures.

in-toto [IN-TOTO] and DSSE [DSSE] are further comparison points for development evidence and signed envelopes. No novelty or comparative security advantage is established. The additional navigation and vision literature is retained as non-normative research context in Appendix K; it is not the protocol's IETF rationale.

2.1. State and memory architecture

The proposed companion requires distinct stores or equivalently isolated records: immutable original artifacts and pinned hash profiles; immutable companion revisions containing canonical payload and envelope bytes; independently administered key and scope policy; an authenticated accepted-revision view; an access-controlled private-reference map; and verifier decision records identifying evidence, policy version and observation time. The architecture is a requirement for a future companion deployment, not evidence that all six components are already operational.

The accepted-revision view identifies binding_id, revision and canonical payload digest. A verifier checks it against the retained predecessor chain and rejects conflicting same-revision digests. Caches retain the policy/version/observation context and report age; loss of current authority leaves freshness unverified. Restoration must preserve revision monotonicity and prior decisions. A hash-linked log alone does not stop a privileged administrator from replacing the whole log and recomputing its hashes.

The retained Numina application has a different, inspected control-state implementation: a durable singleton row, hash-linked events, revision-bound actions and caller-scoped projections. The source names its table numina_control_manifest; an older architecture document says numina_control_state. This discrepancy is recorded rather than hidden. Application journal revisions, manuscript revisions and companion binding revisions are separate counters. Its canonicalization and authority policy must not be assumed equivalent to JCS or companion trust. [APP-CONTROL] [APP-SOURCE]

Human or agent context is a derived narrative projection, not signing authority or canonical accepted state. A proposed persistent-context record carries its source/version, observation time, access class and correction history. Updates invalidate affected cached summaries; unavailable provenance remains explicit. Retention, authorized deletion and recovery policy must state which evidence remains verifiable after removal. This describes a design boundary, not immutable model memory or automatic history availability to every future assistant.

3. Claim format and attribution model

The private envelope contains exactly payload and signature_b64u; a null signature marks an unsigned candidate. Status is computed externally. The fixed schema string is numina.pair-binding.v1. Table 1 summarizes the required components. binding_id remains the record identity field. Ed25519 identifies the signature algorithm, never a replacement for binding_id or proof of signing identity.

Table 1. Private payload components and their roles.

Component	Required meaning
Record identity	schema; stable opaque binding_id; positive safe-integer revision; actual issued_at in UTC, to seconds.
Predecessor	previous_payload_sha256 is null at revision 1 and otherwise the lowercase SHA-256 of the prior canonical payload bytes.
Scope	scope.project_id, scope.release_id, scope.canonical_source_id: exact project, release, and immutable canonical source. A planned release is not an observed publication.
Private association	genesis_ref is an approved opaque reference. It contains neither the genesis value nor a revealing derivative or secret locator.
Sources	Exactly two ordered records: source_1 then source_2. Each carries source_id, alias, artifact_id, hash_profile_id, and sha256.
Attribution	attribution.people, attribution.note, attribution.personToAlias. people contains exactly two name/label records in personal order: Sam Kott / 1A, then Lauren Anderson / 1B. Source order stays separate.
Signer	algorithm is Ed25519; public_key_b64u encodes the raw 32-byte key; key_id is sha256: plus the SHA-256 of those raw bytes.

Table 1: Retained source label: Table 1

The payload object has exactly schema, binding_id, revision, issued_at, previous_payload_sha256, scope, genesis_ref, sources, attribution and signer. scope has exactly project_id, release_id and canonical_source_id. Each source has exactly source_id, alias, artifact_id, hash_profile_id and sha256. attribution has exactly people, note and personToAlias; each people entry has name and label. signer has exactly algorithm, public_key_b64u and key_id. Unknown or missing members are rejected. Status and verification results are outside the signed payload.

schema equals numina.pair-binding.v1. revision is an integer from 1 through 9007199254740991. issued_at is an actual UTC issuance time to seconds. previous_payload_sha256 is null only for an established initial revision, otherwise a lowercase 64-hex predecessor payload digest. Source

sha256 values are lowercase 64-hex strings. The key and signature decode to exactly 32 and 64 bytes respectively. Identifier values and accepted artifact/profile bindings require an independently approved deployment profile; a string passing JSON parsing is insufficient. The available evidence contains no complete approved profile or schema-valid operational instance. [INCOMPLETE-COMPANION]

The raw mathematical digest and its v1 JSON representation are distinct. Section 15.8 defines Digest32 and the hexLower/unhex correspondence; a source sha256 or non-null previous_payload_sha256 is a JSON string containing exactly the lowercase hex encoding. key_id retains its literal sha256: prefix followed by the raw-public-key digest's lowercase hex encoding. These existing fields are not replaced by byte arrays, and artifact bytes are not replaced by their digest strings.

The fixed example maps source_1 / UNIVERSAL_PAIR_1 to Lauren J. Anderson, label 1B, and source_2 / UNIVERSAL_PAIR_2 to Samuel J. Kott, label 1A. Personal display order remains Sam Kott / 1A followed by Lauren Anderson / 1B. The exact commitments and approved person-to-alias note are retained in Appendix B.1 and Appendix F. The AI-assistance display name Ed25519 does not add a third source commitment or replace the signature-algorithm field.

For implementation review, this revision proposes the fixed-pair validation profile in Appendix M. Its numeric limits, identifier grammar, timestamp restriction and rejection rules are new design choices, not recovered historical configuration. The supplied JSON Schema and fixtures make these choices inspectable; accepting this document does not provision keys, registry entries or live policy. The immutable source mapping is unchanged.

The note field's length and character rules establish only schema validity. A real binding must additionally carry the exact approved attribution note or an independently approved versioned replacement under the applicable policy. An arbitrary syntactically valid note, including the test-only note in Appendix M, does not establish approved attribution. Scope, canonical v156/source equality, independent signer trust, freshness and required private mapping remain separate pending gates.

4. Making a claim and constructing its signature

The new companion payload is parsed with duplicate-member detection and validated against a closed, versioned schema before use. Unsupported schemas, unknown fields, missing required fields, malformed encodings, and an unexpected number of sources are rejected. Revisions are positive integers within the interoperable safe-integer range. Field-level constraints must be explicit; accepting an arbitrary string as an artifact or profile identifier would leave the hash's meaning unresolved.

JCS, specified in RFC 8785, supplies a deterministic UTF-8 representation of a JSON value [RFC8785]. Object properties are ordered recursively by UTF-16 code units; array order is retained. Whitespace between tokens is omitted. Strings are preserved without Unicode normalization, and invalid Unicode input is rejected. Number serialization follows the specified

ECMAScript behavior. I-JSON supplies related interoperability constraints, including unique member names [RFC7493]. This design additionally limits revisions to safe integers and treats hashes and identifiers as strings.

Let payload be the validated payload and C = UTF8(JCS(payload)). C has no byte-order mark or appended newline. The signature field is outside payload. The following specifies signing; it does not report an executed signing operation. The separator is one zero byte (0x00).

```
<CODE BEGINS>M = ASCII("NUMINA-PAIR-BINDING/V1") || 0x00 ||
UTF8(JCS(payload))
signature = Ed25519.Sign(private_key, M)
<CODE ENDS>
```

Here, || means byte concatenation. This produces a signature over the payload, which contains the source hashes and genesis reference. It does not calculate the genesis hash. No verified genesis-derivation equation is available in the evidence we inspected.

The algorithm is pure Ed25519 as specified in RFC 8032 [RFC8032]. No extra application prehash is inserted. In particular, signing SHA-256(M) would define a different construction. The prefix is part of the message passed to pure Ed25519; it is not an Ed25519ctx parameter. The 64-byte signature and 32-byte public key are represented as unpadded base64url. Decoding must reject malformed input and enforce exact lengths.

The pure-Ed25519 message construction above remains unchanged by the candidate acceptance policy in Section 15.13. That policy concerns which signatures a verifier accepts, not a new prehash, application prefix, signature format or signing authority. Policy adoption and operational use remain separate gates.

The SHA-256 of C identifies the payload for revision linkage. An optional envelope-file digest identifies the exact serialized envelope bytes and can differ when formatting changes. These digest roles must be labeled explicitly. Neither one replaces either frozen source commitment. The construction specifies bytes to sign but supplies no operational key, signature, or signed public payload.

4.1. Claim-making procedure

1. Establish the claim's project, release, immutable source, intended audience and approved attribution. Obtain the independent signing policy, accepted binding state and permitted disclosure scope. If any required authority or identity is unavailable, stop before issuance and retain an explicitly incomplete candidate.

2. Inventory the original artifacts and original hash-profile code by path, version, byte count and raw-file digest. Reproduce the two commitments under those profiles. Record failures without changing the expected hashes or substituting a new normalizer.

3. Choose the approved binding_id and next revision under the deployment's serialized update policy. For a successor, copy the accepted predecessor's canonical payload digest. Do not borrow a control-journal or manuscript revision. Resolve the approved opaque reference only through the authorized mapping process.

4. Construct and schema-check the complete payload. Populate the actual issued_at at issuance. Produce C and M exactly as specified above. Send M to the authorized signer; keep the private key out of the payload, report and public package.

5. Verify the returned signature against the independently approved public key and M before marking issuance complete. Persist the canonical payload, envelope bytes, digests, predecessor and issuance receipt without altering the original artifacts.

6. Request an atomic update of the authenticated current-state view conditioned on the expected predecessor. Retain conflict and failure receipts. Publish only approved companion bytes after a verifier repeats the required checks; record any external deposit separately. No completed issuance or publication is asserted by describing this procedure.

5. Signing authority and trust policy

A signature can establish consistency between a message and a public key. Acceptance as an authorized Numina statement additionally requires an independent trust entry that binds that key to the exact project, release, and signing role. The verifier recomputes the key fingerprint from the raw key bytes, matches the approved key material and scope, and applies validity and revocation rules defined by the trust policy.

A public key carried inside a payload is an input to verification, not an authority to trust itself. A malicious party can create a different key and sign a false attribution statement. Trust bootstrapping must therefore occur through a separately authenticated channel. The trust policy also needs a documented decision about evaluating revocation and key validity at issuance, verification time, or both; this paper does not assert that such a policy already exists.

An authorized signature attests the signer's association statement. Personal identity, authorship, mathematical derivation, and equality with current source artifacts each need their own evidence. The absence of any one of those findings must remain visible even when the signature check succeeds.

Ed25519 (assistant) is the assistant's user-assigned display name. That label identifies no legal person, public/private key, cryptographic signer or delegated signing authority. In the record schema, Ed25519 (algorithm) names the RFC 8032 signature algorithm; authorization and key identity are checked separately.

The deployment profile must identify who administers trust, how initial keys and policy updates are authenticated, permitted project/release/source/role scopes, key validity intervals, revocation evaluation time, and recovery authority. A candidate containing its own public key cannot

provision that trust. The retained incomplete candidate lacks an approved operational signer and these policy receipts; historical document or package keys are not promoted to companion authority.

A newly inspected public-only package establishes the raw bytes of one historical Ed25519 public key and their correspondence to its SSH fingerprint. This closes a public-key encoding gap for that exact key. It does not establish current private-key custody or authorize a companion signer, scope, profile or payload. The historical SSHSIG signature uses a different signing format. See Appendix P. [PUBLIC-KEY-READBACK]

6. Verifying claims and rejecting unsupported acceptance

A verifier produces a structured result rather than one undifferentiated success value. At minimum, it reports schema validity, attribution-map validity, payload signature validity, signer authorization, source commitment reproduction, current-source equality, revision freshness, and private genesis linkage. Each result records the target checked, the evidence used, the observation time where relevant, and a reason for any failure or unavailable evidence.

6.1. Verification sequence

1. Parse the envelope once under size and depth limits; detect duplicate keys before a parser could discard them. Check the closed envelope and payload schemas. A null signature yields an unsigned pending state. A malformed or failing signature yields an invalid state rather than pending.

2. Construct C and M exactly as in Section 4. If signature_b64u is null, retain the unsigned pending signature result and skip signature decoding and verification. Otherwise, decode the key and signature, enforce lengths and encodings, recompute the key fingerprint, and verify with the required Ed25519 variant. The deployment profile must fix and test the exact RFC 8032 signature-verification acceptance policy, including decoding, canonical scalar/point handling and cofactor-related behavior. Section 15.13 records a NOT ADOPTED review proposal; this candidate does not select an operational policy. Policy-specific positive and negative vectors and independently justified verifier agreement are required for interoperability acceptance. Reuse the same validated payload for subsequent decisions. Unsigned or invalid records cannot be accepted as signed; permitted independent diagnostic checks may still report separate results.

3. Compare project, release, signing role, and immutable source identifier with independently expected values. Evaluate the approved trust entry and key policy. A cryptographically valid statement outside its authorized scope is rejected for that scope.

4. Resolve each artifact and its pinned hash profile, recompute the original commitments, and compare both outputs. Separately check whether the source being evaluated is the canonical source named in scope. Historical hash reproduction is reported only for the snapshot actually inspected.

5. Obtain independently trusted current-state evidence for the binding identifier and revision. Check predecessor linkage, revocation, and same-revision conflicts. Resolve genesis_ref only through an authorized private channel when that check is available. Return the individual results without promoting an unavailable check to success.

6.2. Current state and revision history

A signed timestamp is the signer's statement about issuance. It does not show that no later revision exists. A predecessor digest establishes a link to an earlier payload; it does not, by itself, establish which branch is current. Acceptance therefore depends on a current-state authority or an independently authenticated policy source that identifies the accepted revision and payload digest.

A stale revision, a revoked statement, or divergent payloads at the same binding identifier and revision must not be accepted as the unique current record. If current-state evidence is unavailable, the correct freshness result is unverified. A future implementation must specify update ordering, rollback handling, evidence caching, and how an offline verifier communicates the age of its last trusted state.

6.3. Decision and rejection rules

Table 2. Independent verifier outcomes

Condition	Required handling
Malformed envelope, duplicate/ unknown/missing fields, wrong source order, invalid Unicode or unsupported schema	Invalid input; do not accept or execute. Diagnostic results must not imply a valid payload.
signature_b64u is null	Unsigned pending; do not decode or verify a nonexistent signature, and never report signed acceptance.
Malformed key/signature, fingerprint mismatch or cryptographic failure	Invalid signature or signer binding. Do not reclassify a failure as merely missing evidence.
Valid signature, but unauthorized key/ role/scope or revoked policy	Reject for the requested scope. Signature success remains a separate fact.
Artifact/profile unavailable or reproduction unequal	Report unavailable or mismatch for that exact check; no current-source equality is inferred.
Stale, conflicting or untrusted accepted-revision evidence	Reject claimed current acceptance; where evidence is absent, freshness is unverified.

Condition	Required handling
Private resolver unavailable or reader not authorized	Private linkage unverified; other independent findings may still be reported.
All checks required by the relying policy succeed for the pinned target	Accept only that stated scope under that policy/ version. The result does not prove the underlying attribution/scientific claim or authorize unrelated action.

Table 2: Retained source label: Table 2

A decision record contains the target identifiers and digests, each check's result and reason, evidence locators and observation times, verifier/version, trust-policy version, and cache age where used. A missing required check cannot be silently omitted. Downstream users receive the structured result; a single verified label is insufficient. This reporting contract is proposed and requires interoperability tests before a deployment claim.

7. Private-reference resolution

The proposed private-reference model uses an opaque identifier independent of the genesis value. The public schema does not prescribe an actual reference, registry endpoint, or private record identifier. This manuscript separately records the existing block-zero hash in Appendix D at the author's request; publishing the value does not establish a trusted resolution service.

Successful authorized resolution would establish that the reference points to the private record identified by the authenticated mapping. It would not establish an inverse derivation of genesis from the pair or a unique common origin. Reported source-input dependence is a separate forward-computation fact. A reader without resolution evidence receives an unverified private-linkage status even when other checks succeed.

The resolver has its own authentication, access-control and mapping-version boundary. It returns only the permitted result with a verifiable receipt; a caller-supplied URL or record name is not fetched merely because it appears in a signed payload. Existence, association and mathematical derivation are separate claims. The exact previously recorded genesis and collection procedure appear in Appendix D. No authenticated operational mapping receipt has been recovered.

8. Implementation and field-deployment evidence

Evidence is grouped by what was actually observed. The complete retained manuscript checks, source comparisons, diagnostic operations, negative controller findings and acceptance records are indexed in Appendix A and the following appendices. Reported historical runs are not presented as reruns of this candidate.

Table 3. Implementation and deployment status

Object	Evidence and current limit
Companion wire model and procedures	Specified in this document. No complete operational instance, authenticated trust/current-state/private-resolution set, or field-used companion verifier was recovered.
Retained application control plane	Architecture document and static server-source excerpt corroborate revision-bound authorization, journal consistency and atomic internal record commits. Source inspection is not a fresh runtime test or proof of current deployed-source equality.
Historical application observations	The September 15 security record reports 19 production read/initialization checks (18 responding, one requiring attention), plus version 76 reviewer sign-in and a persisted account-owned request. These are reported application exercises, not companion acceptance or independent penetration testing.
Provider deployment metadata	v156 metadata reports succeeded for source df099d3ae4952a9966cf3b9e4ef299626dc7d338. Exact archive bytes were not verified; a host publication receipt does not establish a deployed companion.
Later v161 observation and artifact signatures	A separately identified v161 source/version/deployment observation and its binding manifest now have independently checked owner-requested signatures. The retained observation, archive representation limits and signature scope are described in Section 16.8. This is not an operational companion payload or a new runtime check.
Current public projection	An October 9 read reports currentJournalRootVerified=false and null revision/eventCount/journalContinuity. It exposes a projection, not an authenticated current private journal.
Synthetic controller	128 held-out toy episodes: loss 186,670 versus hold 187,656 and fixed contraction 183,350; performance and direct-input safety gates failed. No real-world outcome dataset or field predictive result exists in that experiment.

Table 3: Retained source label: Table 3

A field-deployment claim requires the exact deployed source/build identities, operator and environment, actual accepted companion bytes, independent trust and current-state receipts, verifier outputs, protected mapping evidence where required, observed usage over a declared interval, failure/revocation/recovery exercises, and a reproducible access path for an authorized independent reviewer. Website availability, a provider succeeded flag, signed manuscripts and unit tests do not collectively supply those missing observations.

Additional retained application evidence includes an October 1 v139 public-interface check, an October 4 authenticated journal display at revision 142, and a v157 source/deployment receipt. The v157 report names source 9e65e4606a7e51a54f2ea2260a2f0654ecf049fb and 1,188 tests: 1,187 passed, 0 failed, 1 Swift/Foundation test skipped on Linux. These are historical or reported application observations. The exact source identities, times, receipt hashes, unsuccessful checks and access limitations are retained in the technical catalog. [TECHNICAL-CATALOG]

Additional retained local software evidence comprises a digital geometry package and a test log recording 974 passes and zero failures. Fresh readback checks all 25 STL/OBJ artifact digests and measures the assembled OBJ bounds; the application report identifies nine geometry and nine action-framework tests. This is local digital-artifact evidence, not physical fabrication or a deployed companion. Its scope and remaining gates are recorded in Appendix P. [LOCAL-GEOMETRY-TESTS]

9. Operational considerations

Keep source artifacts, companion revisions and operational logs under distinct identities. Apply least-privilege issuance, trust-administration and resolver roles. A client or model proposes a command; server-side policy determines authorization from current state. Never let a narrative, uploaded record or client-supplied status grant itself authority. The retained application follows this separation in its inspected control paths, but a companion service needs its own verified integration. [APP-SOURCE]

Specify request, nesting, string and artifact-size limits; time budgets; storage quotas; and behavior under exhausted resources before deployment. The application guide reports 16 KiB requests, five-second read limits and journal-wide validation, with operation-specific bounds; those are application-profile claims, not automatically the companion's negotiated limits. Full-journal replay cost grows with retained history. No measured companion throughput, availability, recovery-time objective or restoration drill is established. [APP-CONTROL]

For uncertain writes, reconcile by the persisted operation identity and expected revision before retrying. Preserve conflicting branches and failed issuance attempts. Credential rotation does not erase history; recovery must keep binding identity, accepted revisions and authenticated policy continuity. A rollback to an older application may remove controls and cannot be called equivalent enforcement. Do not clear nonces, journals or trust records to make a failed check pass.

Record source/build/runtime versions, time synchronization assumptions, policy/key changes, observation windows, and operational owners. Keep secret keys and sensitive resolver locations outside public reports. Public-safe evidence must identify what an independent reviewer can reproduce, what requires authorized access, and what was not available. An incident runbook is an untested procedure until an actual drill or event provides receipts. [APP-SECURITY]

Key lifecycle policy records custodian, purpose, permitted scope, policy version and validity/revocation intervals. Rotation requires independent authorization of the new key and an explicit transition rule for old signatures; a compromised old key cannot authorize its own recovery by

signature alone. Historical records retain their original keys and verdict context. Framing or semantic changes require an explicit new schema/profile version and migration tests; never reinterpret already signed bytes.

Monitor rejected signatures, unknown keys, source mismatches, same-revision conflicts, unavailable or old current-state evidence, resolver denials, quota exhaustion and journal growth. Define responsible operators and escalation thresholds before field operation. No such monitoring deployment or response service is demonstrated here. Record provider request/outcome identities separately; idempotent internal commits do not establish an external provider effect.

10. Unresolved implementation and acceptance gates

The retained candidate is explicitly incomplete and unsigned. It lacks approved binding identity/history, actual issuance time/predecessor, an authenticated opaque private mapping, final artifact/profile registry identifiers, and approved signer material. Its source lookup inspected 143 event rows (revisions 0–142); five event values and the manifest were truncated. That bounded search recovered no operational binding fields but cannot prove they do not exist. [INCOMPLETE-COMPANION]

Before claiming interoperable implementation, publish the complete closed schema and deployment profile, canonical/signature vectors, independently implemented verifier results, trust/current-state/revocation rules, limits and recovery behavior. Run tampering, parsing, source-order, key-substitution, stale/forked-revision, authorization, unavailable-dependency and preservation tests. Before claiming deployment, supply the field receipts listed in Section 8. The detailed acceptance checklist is retained in Appendix G.1. No document edit closes an unavailable-evidence gate.

11. Security Considerations

This analysis follows the RFC 3552/BCP 72 approach of identifying assets, adversaries, attacks, countermeasures and residual risk. [RFC3552] Protected assets include original evidence bytes, association integrity, signing authority, accepted revision state, private mappings, audit history and service availability. An attacker can read public artifacts, submit malformed records, replay or reorder messages, substitute evidence and keys, or compromise a client. On-path modification and passive observation are considered. Signing hosts, trust administrators, private resolvers and database administrators are privileged boundaries; their compromise is not solved by a bare signature.

The following list identifies controls required by the companion design, with application-level evidence identified separately. A stated mitigation is not an executed test. The retained record does not establish that every mitigation is implemented, independently assessed, or field-tested. Cryptography authenticates specified bytes under a key; it does not establish the truth of an authorized signer's statement.

Threats introduced or exposed by the design

T1 Parser and canonicalization ambiguity

Mechanism: Duplicate members, invalid Unicode, number handling or source-order changes can make implementations sign or evaluate different values.

Handling: Reject before decision; parse once, enforce closed schema and I-JSON constraints, preserve array order, use JCS for the payload only. Pin the original artifact normalizers.

Residual risk and evidence limit: Independent cross-implementation vectors and approved operational profile and full conformance remain outstanding.

T2 Artifact or profile substitution

Mechanism: An attacker substitutes a path, hash profile, registry or current source while retaining a familiar attribution label.

Handling: Compare independently expected scope, raw-file/profile pins and registry version; reproduce each original commitment; report current-source equality separately.

Residual risk and evidence limit: A malicious approved artifact can still satisfy its hash. Historical equality does not attest a current runtime.

T3 Key substitution or unauthorized signing

Mechanism: A self-supplied key signs false attribution or a valid signer acts beyond scope.

Handling: Provision trust independently; recompute fingerprint; evaluate project/release/source/role, validity and revocation. Do not promote historical test keys.

Residual risk and evidence limit: Compromised signer or dishonest authorized claims can remain correctly signed; external review and recovery are required.

T4 Replay, rollback or equivocation

Mechanism: A valid old record or competing same-revision payload is represented as uniquely current.

Handling: Authenticate accepted binding state; check predecessor and policy; reject stale/conflicting accepted states and disclose cache age.

Residual risk and evidence limit: A dishonest or unavailable current-state authority can conceal updates. No transparency or independent checkpoint service is demonstrated.

T5 Durable-state or journal compromise

Mechanism: Deleting/replacing state or rolling back backups changes what history appears current.

Handling: Retain append-only evidence, verify sequence/links/state binding, use transactional revision checks and independently protected recovery evidence.

Residual risk and evidence limit: An administrator can rewrite a self-contained hash chain. Independent backup restoration and off-system anchoring are unverified.

T6 Private-reference disclosure or misuse

Mechanism: Stable references, resolver responses or debug records reveal protected information or enable unauthorized correlation.

Handling: Use opaque references, authenticated caller-scoped resolution, minimized public results and protected audit records. Do not fetch untrusted locators from claims.

Residual risk and evidence limit: Opaque stable identifiers still permit correlation; access control and encryption need actual deployment verification.

T7 Transport substitution and interception

Mechanism: A network attacker observes private data or replaces policy/resolver/service responses.

Handling: Use authenticated confidential transport for protected online exchanges and verify object signatures/policy identities independently.

Residual risk and evidence limit: Object signatures do not encrypt data; transport authentication does not prove record truth. Production endpoint settings remain separately testable.

T8 Forged client authority, CSRF or hostile content

Mechanism: A browser request, proxy header, document or model output tries to broaden permissions or execute a claim.

Handling: Reauthorize server-side against current state; validate identity ingress and Origin for cookie-authenticated writes; treat external text as data.

Residual risk and evidence limit: Source-visible application controls do not prove dispatcher-bypass resistance in the deployed topology or cover every future companion route.

T9 Resource exhaustion

Mechanism: Oversized/deep JSON, expensive artifact hashing, repeated signatures or full-journal reads exhaust compute/storage.

Handling: Apply explicit profile limits and early rejection, rate/time budgets and bounded evidence retrieval; retain failure reasons.

Residual risk and evidence limit: Approved operational limits, load-test results and recovery targets for a companion deployment are not established; the appendix supplies proposed input limits for review.

T10 Claim/status confusion

Mechanism: A passed digest, synthetic test, website publication or operator attestation is shown as universal verification.

Handling: Return independent typed findings with source, time, version and limitation; require all policy-required evidence for the stated decision.

Residual risk and evidence limit: Human interfaces can still overstate results; review presentation and reliance decisions, not only cryptographic code.

T11 Build or evidence mutation

Mechanism: Build steps rewrite source, receipts or policy after a prior audit/signature.

Handling: Inventory evidence before/after, pin exact source/build artifacts and preserve historical bytes; regenerate only maintained projections.

Residual risk and evidence limit: A current engineering audit is still active. No SLSA level, immutable builder or independently secured provenance is established by this draft.

T12 Unsafe recovery or external effects

Mechanism: Retries duplicate uncertain operations, or a signed statement is treated as permission for unrelated actions.

Handling: Persist operation identity and outcome, compare revisions, reconcile before retry, and keep execution authorization separate.

Residual risk and evidence limit: The inspected control plane records internal effects; independent external execution or settlement requires its own authority and receiving-system evidence.

Operational response requires revoking affected authority, preserving original evidence, identifying the impacted scope, reconciling uncertain writes, and verifying recovery before resumption. Availability loss is preferable to silently accepting an unauthorized or unverifiable record. The application security document records a runbook and required restoration evidence; it does not establish a completed incident drill. [APP-SECURITY]

12. Proposed applications and research boundaries

The author proposes applications in chemistry, physics, healthcare, humanitarian work and civil-rights records. The concrete capability described here is preserving evidence and traceable claim revisions for authorized workflows. Historical records can receive append-only corrections without erasing the earlier record; the protocol does not alter past physical events. Present actions require separate authorization. Future-event or scientific predictions require predefined

targets, independent observations, held-out evaluation, baselines and uncertainty reporting. No medical efficacy, physical forecasting, or universal next-action capability is demonstrated by this record format.

The intended humanitarian value is improved accountability for evidence and authorized decisions. Ed25519 is the existing signature algorithm defined in RFC 8032, not a newly invented algorithm. Neither contribution rights, an IETF submission, an institutional inquiry nor a signed record gives this system control over unrelated networks or people. Economic value, scientific recognition and regulatory status are separate questions that require their own evidence; no guaranteed outcome is asserted.

13. IANA Considerations

This document has no IANA actions. The application-specific schema identifier numina.pair-binding.v1 and message prefix NUMINA-PAIR-BINDING/V1 are described as design-study values; this document requests no registry, code point, or global allocation for them.

14. Authorship and attribution

Samuel J. Kott is the author; his original author contact information is retained in the preserved source manuscript. Lauren J. Anderson remains the human contributor associated with source_1 / UNIVERSAL_PAIR_1 / 1B. Samuel J. Kott remains associated with source_2 / UNIVERSAL_PAIR_2 / 1A. These are the author's descriptive universal-source and discovery attributions; they do not independently establish discovery priority or coauthor consent. The exact approved map is preserved in Appendix F.

Ed25519 (assistant) is the user-assigned name of the OpenAI-powered AI assistant credited for drafting, formalization assistance, technical organization, specification consultation, technical review and source-scoped documented checks. It is an AI-assistance role rather than a legal person, human coauthor, public/private key, cryptographic signer, delegated signing authority or IETF official. In protocol fields, Ed25519 (algorithm) separately names the RFC 8032 signature algorithm. The author is responsible for reviewing claims, citations, evidence and disclosure before any external submission.

15. NCPS formal model: consolidated review text

Status and authority. This chapter consolidates the NCPS sections developed in the drafting conversation into one proposed formal core. It is part of this paper, not a separate research paper. It replaces the conflicting conversational formulations for purposes of this review edition. The underlying claims, source commitments and historical observations remain separately identified. The complete proposed chapter has not been accepted as a faithful formalization by a proof-assistant kernel. Specific source-bound byte, codec, extraction, assessment and bounded-state results have checked proofs; Section 16.6 distinguishes their exact scope from the remaining obligations. A definition, a desired invariant and a completed proof are different evidence categories.

The consolidated definitions are proposed replacement text only. The exact source and clause map identify this review artifact; they do not establish author approval or a normative freeze. Mathematical obligations listed here remain obligations unless an exact, separately identified checked artifact discharges them. No such discharge is inferred from the act of integration.

The governing authority for an approved release is its exact versioned normative specification together with its explicitly pinned normative dependency manifest. A schema, test fixture, implementation or mechanization SHALL NOT silently override it. A discrepancy between these artifacts is a release blocker to report and resolve through a versioned review artifact, not permission to choose whichever result is operationally convenient. The proposed source in this candidate is not yet that approved release. This precedence rule does not retroactively change any frozen source artifact, normalization profile, signature or published manuscript.

Uppercase requirement words in this proposed chapter have the requirement-level interpretation specified by BCP 14, RFC 2119 and RFC 8174. They describe obligations of a conforming implementation, not evidence that an existing implementation meets them. [RFC2119; RFC8174]

15.1. Version scope, types and values

Let $\mathrm{Version}$ be the declared set of specification-version identifiers. Fix $v\in\mathrm{Version}$. All definitions in this chapter use that same version and the same pinned profile unless a cross-version mapping is explicitly supplied.

Declare the types $\mathrm{Specification}(v)$, $\mathrm{Manifest}(v)$, $\mathrm{ImportEnvironment}(v)$, $\mathrm{Formalization}(v)$ and $\mathrm{Development}(v)$. Their inhabitants are, respectively,

```
\[
\begin{aligned}
\Sigma_v &: \mathrm{Specification}(v),\\
\Lambda_v &: \mathrm{Manifest}(v),\\
\Delta_v &: \mathrm{ImportEnvironment}(v),\\
M_v &: \mathrm{Formalization}(v),\\
\mathfrak F_v &: \mathrm{Development}(v).
\end{aligned}
\]
```

Thus M_v is a machine-readable formalization artifact and $\mathfrak F_v$ is a mechanized development; neither symbol denotes a type. The semantic-system record type is

```
<CODE BEGINS>SemanticSystem(v) = record {
  specification : Specification(v),
  manifest      : Manifest(v),
  imports       : ImportEnvironment(v),
  formalization : Formalization(v)
}
<CODE ENDS>
```

with value `\(\mathfrak S_v=(\Sigma_v,\Lambda_v,\Delta_v,M_v)\)`. The mechanized development, proof certificates and execution evidence are explicit additional inputs to an assessment, not hidden components inferred from that tuple.

The notation `\(s_v\)` is a declared alias for the same value `\(\mathfrak S_v : \mathrm{SemanticSystem}(v)\)`; neither denotes the record type. The canonical record field is `imports`. The correction draft's `importEnvironment` is a prose alias for that field only, not another field or a wire-compatible rename. An actual field/schema rename requires a separately versioned schema and an explicit adapter.

Identical version labels do not by themselves establish identical content. The release manifest SHALL pin the exact specification, schema, serialization, proof-rule and policy artifacts. A cross-version import uses the dependency's own qualified identity and pinned version; it is not an implicit switch to another interpretation of `\(v\)`.

Changing normative source bytes or a pinned normative dependency creates a new immutable release identity, even if a display label is reused. The approved specification and manifest identify imported declarations; a formalization artifact is checked against that authority and cannot silently become a competing source. Unpopulated manifest entries and unresolved declaration identities remain release blockers.

Bound variables have their stated local scopes. Global symbols have qualified identifiers. Notation such as `\(\mathcal R\)`, `\(\mathcal C\)` or `\(\mathcal J\)` SHALL NOT be reused for unrelated global types. A symbol occurrence resolves to one declaration; a composite expression does not itself require a separate symbol declaration.

15.2. Dependency manifest and symbol closure

A dependency declaration contains a dependency identifier, exact version, immutable content identifier, exported-symbol identifiers, their kinds and signatures, and the referenced definitions, axioms, rules and constraints needed to interpret those exports. Merely naming a module such as `truth-domain-v1` does not supply its contents.

The manifest `\(\Lambda_v\)` is a finite record of direct dependencies and their required transitive closure. The imported environment `\(\Delta_v\)` is the resolved map from qualified symbol identifiers to complete declarations, not a union of untyped name strings.

Each global declaration has exactly one owner: an imported dependency or this version's local specification. A definition may contain several mutually exclusive clauses or equations within one definition block. A later use is a reference, not a redeclaration. Local bindings inside quantifiers, functions and proofs remain distinct from global declarations.

Resolution SHALL reject missing, ambiguous, conflicting or shadowed global declarations. Imported-first lookup SHALL NOT hide a forbidden local/import collision. A manifest establishes which declaration is used; it does not establish the truth or consistency of imported axioms.

For the import graph, an edge points **from a dependent artifact to its prerequisite**. A required dependency chain SHALL be finite and acyclic after exact versions have been resolved. This rule governs imports and authority dependencies. It does not prohibit recursion admitted by the selected calculus's termination or positivity rules, and it does not require the prose cross-reference graph to be acyclic.

The release record SHALL identify the trusted logical foundation and checker versions. Names for these dependencies are not evidence of available, type-checked or consistent modules.

Bibliographic links containing master, latest or another moving branch are discovery references, not immutable normative dependency pins. Namespace disjointness must be checked before lookup; no imported-first convention may shadow a conflicting local declaration. The manifest must identify the actual imported content and exact exported declaration signatures.

15.3. Protocol scope, genesis and evidence

Let `\(X_v^c\)` and `\(Y_v^c\)` be the two version-specific canonical-source domains, `\(\mathrm{Genesis}(v)\)` the protocol-genesis artifact type, and `\(\mathrm{DerivationFailure}(v)\)` the typed derivation-failure domain. The proposed derivation has type

```
\[
\mathcal D_v:X_v^c\times Y_v^c\longrightarrow
\mathrm{Success}(\mathrm{Genesis}(v))+
\mathrm{Failure}(\mathrm{DerivationFailure}(v)).
\]
```

The concrete schema, ordered input bytes, normalization, framing, algorithm and expected output SHALL be fixed before an implementation claim. The paper's existing recorded chain/block-zero value, application bootstrap event, two frozen source commitments and proposed protocol-genesis artifact are different objects. This definition does not prove that the recorded chain value can be derived from the two commitments.

For valid complete inputs, the selected derivation algorithm SHALL return its specified result independently of local time, mutable external state, randomness or implementation choice. Explicit evidence snapshots or parameters may be inputs; their values cannot be read implicitly during a purportedly pure derivation.

A version may define a finite registry of failure **types** while permitting parameterized failure values. A finite type registry does not imply a finite set of all failure instances. The version SHALL prescribe canonical parameter encoding and deterministic selection when several conditions apply. Failure is tagged separately from success. Resource exhaustion, unavailable services or implementation defects SHALL NOT be fabricated as a deterministic protocol failure prescribed for otherwise identical valid inputs.

The governed object kinds are policy, identity, internal evidence, decision, receipt and protocol state. They belong to the protocol namespace; arbitrary external facts do not derive from that namespace. External evidence is retained or referenced with provenance. Internal observations, canonicalizations and assessments identify their source evidence and execution context.

Verifying an issuer's signature can establish a specified digital-provenance result under a trust policy. It does not establish the underlying real-world statement's truth. Canonicalization and policy interpretation SHALL NOT overwrite the original external evidence. Corrected or superseding internal objects receive new immutable records and explicit links to their predecessors.

15.4. Typed relations and implementation realization

Let `\(\mathrm{RelId}_v\)` be a finite registry of distinct relation identifiers. For each identifier `\(r\)`, let `\(\mathrm{Inst}_v(r)\)` be its declared argument-tuple type and let `\(\mathrm{Extension}_v(r)\)` be a subset of that type. The registry identifies declarations, not bare extensions: two empty relations with different signatures remain distinct entries.

Let `\(\mathbb B=\{\mathrm{false},\mathrm{true}\}\)`, with distinct constructors. A binary relation has an instance type `\(A\times B\)`; a ternary relation has an instance type `\(A\times B\times C\)`. These component types are not the Boolean result type of membership evaluation.

For each registered relation define its characteristic function on the stated instance domain:

```
\[
\chi_{v,r}:\mathrm{Inst}_v(r)\to\mathbb B,
\qquad
\chi_{v,r}(q)=\mathrm{true}\iff q\in\mathrm{Extension}_v(r).
\]
```

An executable profile SHALL supply a terminating membership procedure and a correspondence obligation. Naming a mathematical characteristic function does not supply an algorithm for an arbitrary relation.

Define the dependent function space and its distinguished semantic family by

```
\[
\mathrm{EvalFamily}(v):=
\prod_{r:\mathrm{RelId}_v}(\mathrm{Inst}_v(r)\to\mathbb B),
\qquad \chi_v:\mathrm{EvalFamily}(v),
\]
```

where `\(\chi_v(r)=\chi_{v,r}\)`. There is no additional arrow from `\(\mathrm{RelId}_v\)` to the entire dependent product: the product already is the type of a relation-indexed family. This distinction follows the ordinary typing of dependent functions. [LEAN-TYPES]

Let `\(\mathrm{Implementation}(v)\)` contain candidate computational implementations. Let `\(\mathrm{TotalEvaluator}(v)\)` be the refined domain furnishing a total, correctly typed evaluator on every `\(r:\mathrm{RelId}_v\)` and `\(q:\mathrm{Inst}_v(r)\)`. A profile that selects a smaller relation registry must declare that restricted registry and its types explicitly; it cannot silently leave arguments of the declared full dependent product undefined. Define

```
\[
\operatorname{Realize}_v:
\mathrm{TotalEvaluator}(v)\to\mathrm{EvalFamily}(v).
\]
```

For `\(K:\mathrm{TotalEvaluator}(v)\)`, `\(\operatorname{Realize}_v(K)(r)\)` is its total evaluator for `\(r\)`. Declare `\(\mathrm{Sem}(v):=\mathrm{EvalFamily}(v)\)` and `\(\operatorname{realize}_v:=\operatorname{Realize}_v\)` as definitional notation aliases, with `\(\chi_v:\mathrm{Sem}(v)\)`. Neither alias adds an extra arrow or a new semantic domain.

An arbitrary `\(K:\mathrm{Implementation}(v)\)` has no total Realize map without the required totality and typing witness. A separately declared partial evaluation interface may return a Boolean result, Unavailable(reason), or Error(errorReceipt) on its explicit domain; it must specify its domain and termination/resource behavior. Unavailability, nontermination and host failure are not silently coerced to false or to membership in TotalEvaluator(v).

Define relation agreement, for `\(K:\mathrm{TotalEvaluator}(v)\)`, by

```
\[
\mathrm{RelationAgreement}_v(K):\!\iff
\forall r:\mathrm{RelId}_v\;\forall q:\mathrm{Inst}_v(r),\quad
\operatorname{Realize}_v(K)(r)(q)=\chi_v(r)(q).
\]
```

For the same `\(K\)` and an exact profile `\(P\)`, define `\(\mathrm{ProfileConformance}_v(K,P)\)` as RelationAgreement together with every mandatory, named obligation of `\(P\)`: typing; input validation and parsing/serialization; error and result behavior; complete evidence context; policy; and applicable state/refinement obligations. Thus ProfileConformance implies

RelationAgreement; the reverse implication is not asserted. Two fully conforming implementations agree on normative results for identical complete inputs under the same profile. Their programs, algorithms, timing and resource consumption need not coincide.

For a fixed K with RelationAgreement, the maps from the singleton type $\mathbf{1}$ to $\mathrm{Sem}(v)$ selecting χ_v and $\operatorname{Realize}_v(K)$ are equal extensionally under the chosen logic. Equivalently, these semantic families are equal. This parallel-arrow statement concerns relation agreement only; it does not prove full profile conformance or identity of implementation mechanisms.

Required evidence remains separately scoped: termination/totality and typed evaluation, relation agreement, and the profile-specific obligations. Selected byte/extraction/presence proofs or bounded finite-DAG checks cannot by themselves establish these universal evaluation and conformance claims.

15.5. Artifact precedence, conflicts and collision scope

Let $\mathrm{Artifact}(v)$ and $\mathrm{Construct}(v)$ be the artifact and construct domains. Preserve the relation arities:

$$
\operatorname{Precedes}_v \subseteq \mathrm{Artifact}(v) \times \mathrm{Artifact}(v),
$$

$$
\operatorname{Conflict}_v \subseteq \mathrm{Artifact}(v) \times \mathrm{Artifact}(v) \times \mathrm{Construct}(v).
$$

Precedence is irreflexive and transitive, hence asymmetric. Conflict is symmetric in its artifact arguments. Reification of a conflict into a record does not change this relation's arity. An atomic conflict record identifies one canonically ordered artifact pair and one construct. Multi-artifact or multi-construct reports SHALL carry a finite set of atomic witnesses rather than pretending that one pair covers all cases. A dependency cycle is a graph-level finding with a complete cycle witness.

Artifact identity and normative content are separate functions. Two publication artifacts may contain equal normative content under an allowed mirror policy without having equal artifact identifiers. A digest is an integrity commitment, not a mathematical guarantee of collision-free identity.

Resolution is relative to an identified artifact-set snapshot and authenticated history. A record is resolved only after its replacement, withdrawal or correction is evidenced and the resulting snapshot is revalidated. Selecting the higher-precedence text identifies the governing interpretation but does not repair an inconsistent lower-precedence artifact. A formerly resolved conflict can recur in a later snapshot; resolved is not a timeless terminal state.

For bounded identifier resolution, the affected preimage set SHALL be finite, explicitly scoped and complete for the declared collision-state snapshot. The profile fixes a nonnegative bound `\(K_{\max}\)`; the earlier proposed default is 255. Resolution at that final permitted level is success when all required groups are singleton; remaining ambiguity causes rejection. No larger level is silently attempted.

The extended-identifier construction SHALL encode the level, version, base identifier and complete predecessor unambiguously. Any claimed monotonic refinement of collision groups requires proof that distinct predecessor identifiers cannot merge at the next level. Final identities are assigned only under the complete snapshot's assignment rule. New arrivals or changed collision state require explicit reevaluation or migration; local uniqueness cannot establish permanent global uniqueness.

15.6. Proof judgments, consistency and conformance evidence

A proof judgment identifies the claim, target artifact, specification/profile version, complete proof inputs, evidence snapshot, applicability context and approved rule environment. The same complete tuple SHALL yield the same normative judgment. The artifact and version alone do not determine the outcome when the supplied evidence differs.

Keep the proof-status enum `\(\{\mathrm{Proven},\mathrm{NotProven},\mathrm{NotApplicable}\}\)` separate from `\(\mathbb B\)`. Missing evidence, rejected certificates and explicit counterexamples SHALL have different reason codes. NotProven is not a proof that the claim is false. NotApplicable requires a specification-defined applicability determination; it is not a substitute for unavailable evidence.

Every claimed theorem SHALL identify its assumptions and a proof artifact or a clear pending status. Kernel acceptance is evidence about the encoded theorem under its dependencies. Correspondence between that theorem and the intended manuscript claim remains a separate obligation. In Lean, proof-hole and axiom dependencies must be inspected; a successful elaboration alone is insufficient. [LEAN-PROOFS; LEAN-AXIOMS]

Internal consistency means joint satisfiability of the normative requirements in the admitted interpretation framework. Individual satisfiability or pairwise compatibility is insufficient: `\(x=y\)`, `\(y=z\)` and `\(x\ne z\)` are pairwise satisfiable over a domain with at least two elements, but not jointly satisfiable. Consistency alone does not imply a unique model, a deterministic evaluator or a decision procedure.

Unique name resolution, type correctness, uniqueness of type up to the specified type equality, and uniqueness of semantic evaluation are separate obligations. Do not infer a unique typing derivation from unique semantics. Additional type-theoretic rules can provide several valid derivations of the same judgment.

Finite tests establish results for the tested inputs and implementations. A claim over every possible input requires a proof with explicit assumptions or an exhaustive finite domain. Cross-tool agreement is valuable corroboration, not a substitute for a faithful encoding or an audited trusted base.

15.7. Closure assessment with explicit evidence inputs

Define `\(\mathrm{ClosureInput}(v)\)` as a record containing the semantic-system value, mechanized development, supplied proof/evidence artifacts, and exact check-profile identity. This makes `\(\mathfrak F_v\)` explicit instead of reading it from outside a declared one-input interface.

The closure-check index order is dependency, symbol, type, semantic, proof, conformance and mechanization. Each check is a named procedure with declared inputs, finite artifact coverage, prerequisites, termination conditions and output reasons. The meanings are as follows.

Check	Required assessment
Dependency	Required import closure is present, pinned, unambiguous and acyclic; exported definitions and constraints resolve.
Symbol	Every in-scope occurrence resolves to one declaration or local binder, with no forbidden collision or undeclared import.
Type	The selected checker accepts all required declarations, applications and theorem statements under the pinned type rules.
Semantic	Every required construct has an identified interpretation and a checked correspondence obligation where claimed; unproved correspondence is reported, not assumed.
Proof	Every required theorem has an accepted certificate under the declared kernel and allowed axiom policy.
Conformance	The required profile, oracle, fixtures and implementation results are present and the stated finite or proved obligations pass.
Mechanization	Required stage artifacts, refinement evidence and traceability are present, version-aligned and checked.

Table 4: No numbered table label in the integrated source

Use a tagged check outcome, not an invented third Boolean:

```
<CODE BEGINS>CheckOutcome =
    Pass
  | Fail(Reason)
  | Blocked(PrerequisiteIds)
  | Incomplete(MissingEvidenceIds)
<CODE ENDS>
```

Here `\(\mathrm{Reason}\)` is a versioned reason-code record; prerequisite and missing-evidence identifiers are finite lists of qualified identifiers. CheckOutcome is the result layer. The evaluator lifecycle is a distinct tagged layer:

```
<CODE BEGINS>EvaluatorLifecycle =
    Unrun
  | Running
  | Finished(CheckOutcome)
  | Crashed(ErrorReceipt)
<CODE ENDS>
```

The assessment retains one identified lifecycle entry per declared check, including its outcome if Finished and its associated evidence/reasons. The correction draft's `notChecked` label is deprecated because it conflates lifecycle and evidence status. A producer must supply its actual meaning before conversion; there is no implicit mapping from notChecked to Incomplete, Blocked, false or any other constructor. A host crash is Crashed, not a mathematical Fail. No enum mapping in this paragraph produces a completed legacy Boolean result; that separate projection is blocked as specified in Section 15.11.

The evaluator validates its request and immutable inputs, evaluates the check graph in deterministic topological order, records blocked checks without fabricating direct failures, and executes all independent checks that remain available. Each check is evaluated once per run; memoized prerequisites are not recursively rerun. Findings are returned in a defined identifier order. Runtime crashes or external unavailability are not silently relabeled as protocol falsehoods.

For a structurally valid assessment, aggregation examines the exact mandatory-check set pinned by the check profile. The following ordered decision table is exhaustive for that set. All rows retain every individual lifecycle, result, reason and evidence reference; precedence of a failure does not erase pending work.

Condition on mandatory checks	Aggregate assessment	Retained information
At least one Finished(Fail(reason))	Fail	Every completed failure reason and all other finished or pending check records.

Condition on mandatory checks	Aggregate assessment	Retained information
No Finished(Fail), and at least one Unrun, Running, Crashed, Finished(Blocked) or Finished(Incomplete)	IncompleteAssessment	Every distinct pending, crash, prerequisite and missing-evidence reason, together with all completed results.
Every mandatory check is Finished(Pass)	CompletePass	All check records and any profile-declared optional diagnostics.

Table 5: No numbered table label in the integrated source

Optional failures remain diagnostics only under an explicitly declared profile; a failing mandatory check may not be redesignated optional to obtain CompletePass. The seven named checks remain mandatory unless a separately versioned profile expressly and legitimately changes the claim's scope. Invalid assessment structure is rejected by the assessment validator outside this valid-result type. Observation time and host metadata belong to the execution envelope, not the deterministic result core. CompletePass describes this assessment under its evidence and profile; it does not certify universal soundness, operational deployment or publication approval.

The seven-check conjunction is an aggregation rule. It does not prove that arbitrary semantic consistency or implementation conformance is decidable. Executability applies to the concrete finite checks or supplied certificates and their declared limits.

Closure preservation under extension requires discharge of new obligations as well as preservation of old assumptions. Closure under composition requires a checked joint compatibility/merge obligation. Neither property follows merely from preserving old meanings or resolving cross-references. These remain conditional proof obligations until mechanized.

A result from a declared four-node finite evaluator with a fairness assumption concerns that bounded model and its encoded properties only. It does not verify all seven NCPS checks, universal closure soundness, monotonicity/compositionality, or implementation refinement. Each such broader claim requires its own exact model or proof, correspondence argument and receipt.

15.8. Byte, tuple, length and index foundation

Let `\(\mathbb N=\{0,1,2,\ldots\}\)`, `\(\mathbb{Byte}=\{n\in\mathbb N\mid n<256\}\)` and `\(\mathrm{Fin}(n)=\{i\in\mathbb N\mid i<n\}\)`. Define

```
\[
\mathbb{Byte}^{\,n}:=\mathrm{Fin}(n)\to\mathbb{Byte},
\qquad
\mathcal B:=\coprod_{n\in\mathbb N}\mathbb{Byte}^{\,n}.
\]
```

A tuple `\(x:\mathbb{Byte}^{\,n}\)` is a function on the finite index set. A byte sequence is the tagged pair `\(\iota_n(x)=(n,x)\in\mathcal B\)`. The injection images are disjoint. Strictly, a sequence belongs to the image of `\(\iota_n\)`, not to the untagged tuple carrier itself.

The unique empty tuple is `\(():\mathbb{Byte}^{\,0}\)`. The empty byte sequence is

```
\[
\varepsilon:=\iota_0(()).
\]
```

Define length and indexing by

```
\[
|\iota_n(x)|:=n,
\qquad
\iota_n(x)[i]:=x(i)\quad(i<n).
\]
```

Indexing is undefined outside that bound. The empty sequence has no valid index; it has neither a first nor a final byte. For a nonempty sequence, the first index is 0 and the last is `\(|b|-1\)`.

For `\(0\le i\le j\le|b|\)`, let `\(\operatorname{slice}(b,i,j)\)` be the length-`\(j-i\)` sequence whose component at `\(k<j-i\)` is `\(b[i+k]\)`. Prefixes are `\(\operatorname{slice}(b,0,k)\)`, suffixes are `\(\operatorname{slice}(b,k,|b|)\)`, and contiguous subsequences are `\(\operatorname{slice}(b,i,j)\)`. Empty boundaries yield `\(\varepsilon\)`; the full interval yields `\(b\)`. Different intervals may yield equal byte values, so the value alone does not identify an occurrence.

Concatenation `\(b\cdot c\)` has length `\(|b|+|c|\)`, with bytes from `\(b\)` first and then bytes from `\(c\)`. Its identity is `\(\varepsilon\)`. Sequence equality means equal length and equal bytes at every valid index. Tuple notation for a sequence abbreviates its length-tagged injection; it does not erase the length tag.

The byte domain contains arbitrary finite bytes. It is not restricted to text. Structured parser results, proof terms, records and extraction sets are not themselves byte sequences; their explicit serialized encodings are. The mathematical index convention does not restrict internal storage layout or optimization when observable byte behavior is preserved. Integer endianness is a separate, field-specific rule.

For compatibility with the correction notation, declare `Nat` as `\(\mathbb N\)`, `Byte` as `\(\mathbb{Byte}\)`, `ByteTuple(n)` as `\(\mathbb{Byte}^{\,n}\)` and `Bytes` as `\(\mathcal B\)`. These are aliases of the definitions above, including their empty value, tagged injection and index bounds.

Define `Digest32 := ByteTuple(32)`. A pinned SHA-256 computation maps arbitrary finite Bytes to Digest32. Define `Hex64Lower` as the strings of exactly 64 lowercase ASCII hexadecimal characters. The encoding functions `hexLower : Digest32 → Hex64Lower` and `unhex : Hex64Lower → Digest32` have the required inverse laws `unhex(hexLower(d)) = d` and `hexLower(unhex(h)) = h` on those respective domains. These are explicit codec obligations; example round trips alone are not a universal bijection proof.

A v1 JSON digest field is the JSON string containing `hexLower(d)`. Its UTF-8 encoding is a wire representation of a digest, not the 32-byte digest value or the original artifact bytes. The original source commitments, raw-key fingerprint digest, predecessor payload digest, SHA-256 of canonical payload C, optional exact-envelope digest and recorded genesis value keep their separate roles and unchanged encodings. Arbitrary artifacts continue to be hashed under their individually pinned historical profiles; this clarification changes no fixture, C/M bytes or original commitment.

15.9. Tagged extraction and empty cases

Let `\(\mathcal P_{\mathrm{fin}}(\mathcal B)\)` be the finite subsets of `\(\mathcal B\)`, and let `\(\mathbf1=\{\mathrm{unit}\}\)`. The extraction result type is

```
\[
\mathrm{ExtractionResult}(v):=
\mathbf1+\mathcal P_{\mathrm{fin}}(\mathcal B).
\]
```

The left constructor `\(\operatorname{inl}(\mathrm{unit})\)` is rejection; `\(\operatorname{inr}(E)\)` is parse success with extracted set `\(E\)`. The injections have disjoint images and are injective. Parse success here does not imply schema validity, canonicality, authorization or acceptance of a protocol action.

The profile supplies a total parse-status predicate and an auxiliary extractor defined only on the subtype of successfully parsed bytes. The total tagged interface rejects when the predicate is false and otherwise returns the auxiliary extraction in the right constructor. Auxiliary does not mean non-normative: its behavior is specified, although it need not be a public API.

Define erasure by

```
\[
\operatorname{erase}(\operatorname{inl}(\mathrm{unit}))=\varnothing,
\qquad
\operatorname{erase}(\operatorname{inr}(E))=E.
\]
```

Then `\(\operatorname{TextFieldSlices}_v=\operatorname{erase}\circ\operatorname{TextFieldExtraction}_v\)`. An empty erased result cannot distinguish rejection from successful empty extraction. A nonempty erased result establishes parse success under the stated parser only.

Three values must remain distinct:

```
\[
\operatorname{inl}(\mathrm{unit}),\qquad
\operatorname{inr}(\varnothing),\qquad
\operatorname{inr}(\{\varepsilon\}).
\]
```

The last means one distinct extracted value whose byte length is zero. Its extraction set is nonempty. Sets preserve distinct values but not discovery order, multiplicity, offsets or field names. Any occurrence-preserving interface SHALL use separately typed occurrence records or sequences.

For JSON, the profile SHALL state whether extraction means raw token spans or UTF-8 re-encodings of decoded string values, and whether member names are included. These are different operations. This proposed interface uses decoded string values designated by the schema; positions require a separate interface. A finite set cannot reconstruct the original record or substitute for validation of the complete JSON document.

15.10. Canonical JSON and refined encoder domains

For this NOT ADOPTED candidate profile, RFC 8785 supplies JCS. At every external JSON intake boundary, the entire incoming document SHALL be strict UTF-8 without a leading UTF-8 BOM and SHALL contain exactly one complete JSON value. Duplicate member names are rejected after escape decoding, before a parser could discard one. Validate decoded names and string values as well as incoming bytes: escaped lone surrogates are not detected by a raw UTF-8 check. Enforce the declared I-JSON restrictions, including noncharacter rejection. Preserve Unicode strings without normalization. U+FEFF within a JSON string is not a leading transport BOM and remains valid subject to that field's other restrictions. Arbitrary non-UTF-8 binary Bytes remain valid mathematical byte sequences outside this JSON transport boundary. [RFC3629; RFC8259; RFC7493; RFC8785]

JCS sorts object names by UTF-16 code units and preserves array order. Earlier schema-member-order requirements do not govern JCS output. The schema controls field presence, types and array semantics, not JCS property order. A set-to-array conversion needs its own explicit deterministic ordering before JCS. [RFC8785]

Let `\(\mathrm{JsonValue}(v)\)` be the normalized, JCS-admissible JSON-value domain and `\(\mathrm{JcsBytes}(v)\subseteq\mathcal B\)` its canonical byte image. Numbers are interpreted in the declared JSON numeric model, not as arbitrary host-language values. The fixed companion profile uses safe integers for revision and strings for identifiers and digests; distinctions lost through numeric rounding must not be claimed preserved. [RFC8785; RFC7493]

For schema `\(S\)`, let `\(\mathrm{ValidRecord}_v(S)\)` be the schema-valid, JCS-representable record type. Require an injective projection `\(j_{v,S}\)` to `\(\mathrm{JsonValue}(v)\)` and a specified inverse on its image. `RecordValid[v,S]` is an explicit definitional alias for `\(\mathrm{ValidRecord}_v(S)\)`, not a second schema or an unrestricted host-object type. Define

```
\[
\operatorname{Serialize}_{v,S}:=
\operatorname{JCS}\circ j_{v,S},
\qquad
\mathrm{CanonBytes}_v(S):=
\operatorname{image}(\operatorname{Serialize}_{v,S}).
\]
```

The serializer is total on valid records. Its record parser is total on `\(\mathrm{CanonBytes}_v(S)\)` and is its inverse there. These are refinement obligations; JCS alone does not prove that an arbitrary NCPS projection is injective.

`CanonicalBytes[v,S]` is an explicit definitional alias for `\(\mathrm{CanonBytes}_v(S)\)`, the exact serializer image above. Primitive Parse operates on that refined image; its stated totality and inverse laws require the named injectivity and round-trip proofs. Current finite checks do not establish the complete registry's type correctness or all such proofs.

External intake is a different function, with arbitrary finite bytes as input and a tagged rejection or validated record as output. It enforces the full JSON intake rules above, schema/representability checks and candidate limits before admitting a record. Strict canonical decoding additionally reserializes the validated envelope and compares every input byte. Pretty formatting or an appended newline is therefore rejected by the strict canonical decoder; a separate intake API may accept those for canonicalization within its limits. Neither API expands the domain of the primitive refined-image parser. The empty byte sequence is valid UTF-8 but is not a JSON value.

The single governing candidate input bound is 16,384 incoming UTF-8 bytes, inclusive; the postcanonical serialized envelope must also be at most 16,384 bytes. The candidate maximum depth is 8, inclusive. Depth is defined by a walk of the decoded JSON value: the root value has depth 1; each object member name and value is visited at its containing object's depth plus 1; each array element is visited at its array's depth plus 1; scalar leaves count. The maximum over all these visited nodes is the document depth. This is this candidate's explicit proposed choice; the plan's 1 MiB/depth-64-container alternative also remains unapproved and is not a demonstrated operational configuration. Its container-counting alternative is not silently equated to this name/value/leaf walk. The candidate limits remain NOT ADOPTED pending

owner selection; later finite boundary receipts in Section 16.6 do not supply that decision. The earlier engineering reconciliation had not verified the original runner's depth convention. Static inspection on 9 October 2026 now confirms this walk convention in both recovered retained-original/create_profile_vectors.py and candidate/create_profile_vectors.py: root d=1, rejection for d>8, and names, values and array elements traversed at d+1. This inspection is not a whole-generator rerun of the historical 21-case suite or proof of the proposed postcanonical-size rule; new boundary checks retain their separate receipts.

The semantic adapters have types

```
\[
\operatorname{ToRecord}_{v,S}:\mathrm{Object}(v)
\rightharpoonup\mathrm{ValidRecord}_v(S),
\]
```

```
\[
\operatorname{ToObject}_{v,S}:\mathrm{ValidRecord}_v(S)
\rightharpoonup\mathrm{Object}(v).
\]
```

Both require an explicit representability relation and unique result. Their inverse properties are additional obligations; independent uniqueness in each direction does not imply inverseness. Name the complete encoder `\(\operatorname{EncodeObject}_{v,S}:=\operatorname{Serialize}_{v,S}\circ\operatorname{ToRecord}_{v,S}\)`, partial on the object universe and total when restricted to its actual domain. Name the complete decoder `\(\operatorname{DecodeObject}_{v,S}:=\operatorname{ToObject}_{v,S}\circ\operatorname{Parse}_{v,S}\)`. The lowercase `encodeObject` and `decodeObject` are notation aliases for these composites; neither is a competing name for the primitive record serializer/parser. Round-trip identities are asserted only on specified round-trip domains.

15.11. Presence, null, integrity and output comparison

A field has a declared presence attribute, required or optional, and a declared value type, nullable or non-nullable. Model its structural state as the disjoint alternatives `\(\mathrm{Absent}\)`, `\(\mathrm{Null}\)` and `\(\mathrm{Value}(t)\)`. Schema guards determine which alternatives are admitted. Null is never the same state as absence, false, a missing proof or an execution error. The ordinary-value branch expressly excludes the null constructor.

A Boolean is exactly true or false. Nullable Booleans and multi-state proof/verification outcomes use separate tagged types; they do not add values to the Boolean type. The eight Boolean fields of a completed legacy ClosureResult remain required and non-nullable. Assessment findings and incomplete executions use the distinct result envelope described in Section 15.7.

Legacy schema missing. Seven named checks do not identify eight legacy fields. `CompletedLegacyClosureResult` SHALL NOT be constructed until the exact eight field names, schema/version, mandatory check-to-field projection, field meanings and all required completed evidence are identified. No eighth field, enum mapping or Pass-to-Boolean projection is invented here. In particular, Blocked, Incomplete, Unrun, Running and Crashed cannot be converted to a completed legacy result by silently inserting false. The exact legacy schema, total projection and preservation proofs remain PENDING. A tagged assessment may still be reported with these limitations; it is not a completed legacy Boolean summary.

Integrity-input construction is a transformation of structured data **before** JCS serialization. It SHALL identify the exact self-digest field or field paths replaced, the exact placeholder, and all excluded members. Retain verified child digests unless the profile explicitly states otherwise. A placeholder may not satisfy the original digest-field schema, so define a separate integrity-preimage type rather than declaring the transformed object valid under the unmodified schema.

Hash the resulting canonical bytes with the pinned algorithm. Do not replace substrings inside already canonicalized bytes or recursively erase every nested digest by accident. Excluded metadata is not protected by that digest. An integrity hash does not authenticate its publisher or prove source truth.

For interoperability, compare the deterministic **result core**: pinned inputs, profile, check identifiers, canonical outcomes and reasons. Verifier build identity, observation time, host information and operational logs may differ in separately labeled envelopes. Requiring complete reports with different timestamps or verifier versions to have identical bytes would contradict their schemas.

15.12. State refinement and operational guarantees

The Lean and TLA+ developments are implementations of the same proposed mathematical obligations; neither tool automatically validates the other. Define concrete and abstract state types, initial predicates, next-state relations and reachable-state predicates. Let `\(\rho_v\)` be total on all reachable concrete states and map them to abstract states.

Initialization requires every concrete initial state to map to an abstract initial state. Step refinement requires

```
\[
\mathrm{Reach}_T(s)\land\mathrm{Next}_T(s,t)
\Longrightarrow
\bigl(\rho_v(s)=\rho_v(t)\bigr)
\lor\mathrm{Next}_L(\rho_v(s),\rho_v(t)).
\]
```

The equality alternative is an abstract stuttering step. A finite-step simulation may be used instead only when that distinct refinement rule is stated. A partial helper is permissible only if its domain contains every reachable source state relevant to the refinement claim. Restricting obligations to an arbitrary smaller domain must not hide reachable bad states. [TLA-REFINEMENT]

Safety preservation follows only for proved abstract invariants under a validated refinement relation. Liveness additionally requires explicit fairness/progress assumptions and proof that each fair concrete trace maps to a trace satisfying the required abstract temporal property. Safety simulation alone does not establish liveness. One-way refinement is not equivalence; equivalence requires the separately specified reverse or observational correspondence. [TLA-LIVENESS]

The witness may equivalently be typed `\(\rho_v:\mathrm{ReachableTLA}(v) \to\mathrm{LeanState}(v)\)`, with reachable-state coverage proved against the concrete model. Declare the observation function and trace correspondence, including which abstract stuttering is permitted. Liveness transfer additionally requires the relevant nondivergence/progress argument; a fair concrete execution must not disappear into unaccounted infinite abstract stuttering. No Lean–TLA+ refinement proof, full reachable-state correspondence or independent liveness proof is claimed checked by this integration.

Operational nondeterminism, concurrency and implementation scheduling may remain internal. Claimed deterministic judgments require the same complete evidence/state snapshot and policy version. Section 16.6 records checked safety correspondence for one fixed four-node model. General state correspondence, machine-checked TLA translation and temporal-liveness transfer remain unproved.

15.13. Mechanization stages, inputs, outputs and release gate

Stage	Inputs	Outputs and acceptance evidence
Core mathematics	Governing text, pinned imports and symbol/type registry	Formal development, theorem statements, proof certificates, axiom/termination audit and checked correspondence to intended claims.
Operational model	Core development and declared state/authority requirements	State machine, refinement witness, safety proofs and explicit liveness/fairness obligations with their status.
Corpus generation	Frozen core, operational model, profiles and generator	Versioned vectors, expected normative results, generator identity and coverage description.

Stage	Inputs	Outputs and acceptance evidence
Corpus verification	Generated vectors and independent or formally justified oracle	Executed results, negative cases, traceability and a statement of finite coverage; no automatic universal guarantee.
Independent validation	Pinned primary development and separately implemented model/checker	Correspondence arguments, differential findings, exact tool versions and unresolved discrepancies.

Table 6: No numbered table label in the integrated source

Each stage consumes identified artifacts rather than implicit latest state. A stage may discover a defect in an earlier stage; the response is a versioned correction with preserved earlier evidence, not a prohibition on reporting the defect. Generation and testing are data dependencies, not automatically mathematical refinements.

15.13.1. Candidate signature acceptance boundary

The signature proposal supplied for this reconciliation is a version-specific acceptance candidate, NOT ADOPTED. It identifies an exact Node 26.10.0/OpenSSL 3.5.8 verifier behavior with raw 32-byte keys, raw 64-byte signatures, a canonical-scalar check, pinned adapter requirements and rejection of the 165 named disagreement cases in its frozen corpus. Its separate `ncps-signature-policy-proposal-v1/policy.json`, adapter and corpus must be pinned by exact bytes and independently reviewed before an operational profile can adopt them. This descriptive reference does not import an unavailable policy artifact by filename or select an operational signing key.

Agreement with native libsodium 1.0.18 on a reported 919-vector corpus, when supported by matching receipts, is finite-corpus evidence only. It does not prove internal point-decoding, subgroup or cofactor behavior, a universal RFC 8032 acceptance predicate, or conformance on inputs outside that corpus. A stronger mathematical policy needs separately specified rules, correspondence/proof obligations and independent validation. The pure-Ed25519 C/M preimage and every existing public fixture remain unchanged.

Operational acceptance requires an approved policy identity and scope; exact decoding/canonical scalar/point/cofactor decisions; policy-specific positive and negative expectations; matching independently justified implementations; and preserved explanations for every disagreement. Signature validity remains independent of authorization, key custody, scope, freshness, private linkage, source equality and real-world claims. A manuscript signature, SSHSIG or public test key supplies none of those missing operational gates.

15.13.2. Dated artifact-specific release evidence

Each claimed result SHALL name its exact input and output artifacts, immutable versions/digests, profile and schema, observation/run time and timezone, toolchain, command or checker, expected and observed result, and coverage limits. Re-read historical receipts, fresh execution, source equality and receiving-system acceptance are different evidence classes. A result for an earlier XML, Markdown, fixture, implementation or environment does not establish the same result for this candidate.

The 46 review findings retain their individual acceptance gates. The eight integrated action subjects and seven reconciliation proposals are organizational mappings, not 46 closure receipts. Missing exact declarations, profile identities, schema/projection definitions, universal proof obligations, signature/depth-policy approval, operational trust/current state, private mapping, genesis derivation and bounded field receipts remain open until matching evidence or an explicitly approved scope change resolves them. Existing failed controller results, source/repository differences and external publication/signing holds remain part of the record.

A normative-release claim requires the integrated source, complete profile and imports, valid schemas, all proofs required for the claimed theorem set, reproducible test results, unresolved-conflict handling and the applicable external acceptance evidence. The scope of security or compliance SHALL name the relevant profile and assessed environment. JCS adoption, document signatures and tool selection alone do not establish deployed security, regulatory approval or universal scientific validity.

15.14. Conditional consistency statement

The tuple `\(\mathfrak S_v\)` identifies the proposed interpretation context. Its existence as a tuple does not prove that its requirements are jointly satisfiable or that the formalization is faithful.

When unique symbol resolution, type correctness, the required semantic correspondence, determinism, sound proof checking and applicable refinement obligations have been established for the pinned artifacts, conforming implementations SHALL agree on the stated normative results for identical admissible complete inputs. Conflict resolution additionally requires valid resolution evidence for the evaluated artifact-set snapshot.

This is the bounded release claim the paper seeks to support. Remaining proof, interoperability, deployment and scientific-evidence requirements are recorded explicitly rather than being supplied by a final invariant sentence.

In this candidate, those conditions are not asserted discharged. RelationAgreement, full ProfileConformance, aggregate CompletePass, a completed legacy Boolean summary and an external acceptance decision have distinct types and evidentiary requirements. No final sentence or shared use of the word conformance collapses those gates.

16. Fresh verification, recovered evidence and remaining gaps

Retained run boundary: the observations in this section are the dated 9 October 2026 predecessor-review records identified here. This integration does not rerun them or change their target. In particular, a statement below that exact Node 26 execution was unrun is a statement about that historical review, not a claim that no later separate run can exist. Any later receipt must be reported under its own identity and cannot retroactively change these results.

This section adds evidence obtained during the whole-paper review. It does not rewrite the earlier outcome history. Commands, exact inputs, output records and digests are supplied in the evidence package. The review source was `draft-kott-numina-pair-binding-02-working-successor.xml`, 164,153 bytes, with SHA-256:

```
<CODE BEGINS>e57e41c19baf998861f71a3cefe9a7d1a97cf9f452d04b6acba4027794a78b51
<CODE ENDS>
```

The predecessor is unchanged. The generated review edition has its own identity in the delivery manifest. It is not covered by an earlier manuscript signature.

16.1. Read-only manuscript and fixture checks

At 17:32 UTC on 9 October 2026, all 11 named checks in `fresh-checks.json` passed. They established strict UTF-8 and no leading BOM for the manuscript; XML well-formedness; 126 unique anchors and 89 resolving cross-references; the fixture and arithmetic results below; and the printed public signature check described below. These checks are not RFCXML RelaxNG validation, publication acceptance or full protocol conformance.

Artifact/check	Observed result	Scope
Canonical fixture payload	1,184 bytes; printed digest reproduced	Supplied ASCII-key / safe-integer fixture only
Framed signature input	1,207 bytes; printed digest reproduced	23-byte prefix including separator plus payload
Python / Node comparison	Exact payload and framed-message bytes matched	Python 3.13.5 and Node v22.16.0; not the separately requested Node 26 target
Pair diagnostics	Sum, XOR, forward hash and reverse hash reproduced	Public input arithmetic; not an unknown-genesis derivation

Artifact/check	Observed result	Scope
Printed public signature	Valid on printed 107-byte SSHSIG-framed message; changed message rejected	Referenced package not rehashed in this particular check; signer authority not inferred
Raw public-key digest	Printed digest reproduced	Exact public bytes only

Table 7: No numbered table label in the integrated source

Canonical payload SHA-256:

```
<CODE BEGINS>3174a10bd2dd52e932a17522cb7b49d67281e025e9c89d7d78f0725b596eb958
<CODE ENDS>
```

Framed-message SHA-256:

```
<CODE BEGINS>054d474c95f33afa5cf51f69581b7c0df16b68161db1091bd1c36d5453eaf235
<CODE ENDS>
```

16.2. Recovered Unicode repair and fresh regression rerun

The 36,497-byte `Numina-checker-noncharacter-fix-2026-10-09.zip` was recovered and measured. Its SHA-256 is:

```
<CODE BEGINS>2886a87c28f80efbcf9d469e387297874cd7b39a914da814175d2c6a125e2bda
<CODE ENDS>
```

The preserved harness, unchanged schema and source fixtures were rerun at 17:39 UTC using Python 3.13.5. The original checker passed the 17 valid controls and failed the 32 expected noncharacter-rejection assertions. The repaired candidate passed all 49 assertions. The historical 21-case generator was not rerun. The harness loads only the declared validation functions and writes separate new receipts.

Checker	Passed	Failed	Interpretation
Preserved original	17	32	Defect reproduced; expected nonzero test exit
Repaired candidate	49	0	Targeted regression passed

Table 8: No numbered table label in the integrated source

The result concerns decoded member-name and string-value noncharacter checks and valid controls. It is not a complete JCS corpus, production rollout or signed-companion acceptance. `noncharacter-original.json` and `noncharacter-repaired.json` retain every input hash and expected/actual result.

16.3. Public Ed25519 characterization and interoperability finding

The 3,050,634-byte public harness review v3 was recovered with SHA-256:

```
<CODE BEGINS>f90beec51c1d6c587710df6e0ca66a2d4820828048b588fcc5880c9d85a00fdd
<CODE ENDS>
```

Its retained records report 27 passing regression tests, ten successful CLI evidence probes and two earlier complete characterization runs. This review newly validated the retained receipt consistency and the equality of the two retained runs, then executed a new isolated characterization using the unchanged public corpus and harness. The 27-test harness suite and ten CLI probes were not reexecuted here and remain retained evidence.

The fresh run began at 17:39:33 UTC on 9 October 2026. It used Python 3.13.5, cryptography 46.0.4 linked to OpenSSL 3.5.5, and native libsodium 1.0.18. It made 1,838 public-key verification calls: 914 adversarial CCTV cases and five RFC 8032 positive controls on each provider. All 919 per-case receipts were produced and passed the receipt-consistency checker. No signing or key-generation API was used.

Provider	Adversarial cases accepted	Adversarial cases rejected	RFC controls accepted
cryptography / OpenSSL	208	706	5 of 5
Native libsodium	43	871	5 of 5

Table 9: No numbered table label in the integrated source

There were **165 differing provider decisions** across the 914 adversarial cases. Unexpected API exceptions and failed positive controls were zero. These are descriptive backend acceptance results; they are not a finding that one provider violated a selected NCPS policy, because no complete operational acceptance policy was supplied or adopted. They demonstrate why positive controls alone do not establish interoperability. RFC 8032 verification behavior, including the cofactor choice and strict decoding policy, must be fixed in the profile. [RFC8032]

The unchanged derived corpus contains 919 cases and has SHA-256:

```
<CODE BEGINS>3a3a04ba2dc28750a95ad67e931411cc7b68d90c39edc42106f16d46b3469434
<CODE ENDS>
```

At the 17:39 UTC review stage described here, the exact Node 26.10.0/npm 11.19.1 target execution had not been performed. The local Node v22.16.0 fixture comparison remains separately labeled. The later exact-target receipts are reported in Section 16.6; they do not change this earlier run or its environment.

16.4. Recovered technical catalog

The 120,995-byte technical-evidence review package was recovered and its whole-file digest matched the manuscript's stated identity:

```
<CODE BEGINS>472e0f052b7760cd034e409f2a1d624ed6c57f22a52156d6aef12fde6fd855b7
<CODE ENDS>
```

It includes the evidence matrix, selected static source extracts, historical controller protocol/ results, source observations and the field-deployment checklist. Recovery and digest measurement do not constitute rerunning all embedded experiments or reobserving their original external systems. Its evidence remains labeled by source, time and scope.

16.5. Earlier missing-evidence register

The following is the 17:39 UTC review-stage snapshot. Section 16.6 records later completed scoped work and Section 16.7 states the remaining edition-level gaps. A cell below requesting proof or corpus execution must not be read as denying the later evidence.

Requirement	Evidence now supplied	Remaining requirement
Single readable paper	Integrated text, DOCX, PDF and source manifest	Author review of proposed formal consolidation and unresolved semantic decisions
UTF-8 / Unicode checks	Fresh targeted regression with preserved failing original	Full profile-wide parser and JCS adversarial corpus
Signature implementation comparison	Fresh 919-case, two-provider characterization and all per-case receipts	Adopt exact NCPS acceptance rules and resolve every profile-relevant disagreement
Proofs of NCPS invariants	Explicit corrected statements and assumptions	Actual Lean/other-kernel proof artifacts, axiom audit and correspondence review
Operational refinement	Typed obligations including stuttering and liveness scope	Executed TLA+ or equivalent model and mechanically checked refinement evidence

Requirement	Evidence now supplied	Remaining requirement
Operational companion	Historical source/evidence records and public fixture	Approved payload, scoped signer authority, trust/revocation policy, current state and required private mapping
Current field deployment	Retained source and provider observations	Exact current source/build binding, field-case receipts and independent relying-party confirmation
Recorded genesis derivation	Exact recorded value and public diagnostic results	Concrete proposed derivation, input definition and independently reproducible target check
Regulatory or certification claim	Technical control/evidence distinctions	Named regime, scoped legal/ compliance assessment and actual receiving-authority evidence where applicable

Table 10: No numbered table label in the integrated source

The new evidence closes recovery and bounded-reproduction gaps. It does not fill absent authority, proof, deployment or scientific-efficacy evidence by assertion.

16.6. Later executable evidence and independent receipt review

Evidence cut for this proposed edition: 9 October 2026, 21:27 UTC. Compilation, model-checking and corpus runs cited here occurred in the separately retained engineering packages. This manuscript integration freshly checks bytes, receipt consistency and the indicated public signature; it does not claim to rerun those suites. Exact package, source and receipt hashes are supplied in verification-summary.json and the evidence index. Counts below overlap through inherited sources and are never summed into a protocol-completion score.

The corrected v2 evidence supports specific mathematical and finite-test results, while preserving the finding matrix's disposition of 43 partial findings, three pending findings (R08, R37 and R46), and no fully closed findings. Candidate gates R13, R14, R17, R19, R20, R25, R27, R39 and R41 are complete in their declared scopes; the literal mathematical gates R17, R19, R20 and R25 are checked. These achievements do not require an operational approval to become mathematical results. Their incorporation into the governing manuscript and the remaining conditions of each original finding are separate obligations. Later attachments do not retroactively reclassify this stable snapshot.

A fresh read-only review authenticated all seven available corrected archives against the handoff's sizes and SHA256 values and compared 661 extracted member files with their archive bytes. The eighth archive, the byte-representation bridge, was unavailable; a retained copy of its exact source matches the declared source hash, but does not substitute for verification of the missing archive. No bundled script or historical proof, serializer, validator, model-checking or

signature suite was rerun for this review. The available original issue-register JSON matches its reported SHA256. Its ordered identifiers, priorities, labels and all 46 acceptance gates agree exactly with the matrix. Six finding/action fields in five rows differ only in PDF-derived typography; both forms are preserved explicitly rather than described as literally identical.

The corrected serialization corpus contains 421 positions and 418 distinct raw inputs, comprising 37 positive and 384 negative cases. Complete envelopes contribute 13 positives and 98 negatives; generic JSON contributes 24 positives and 286 negatives. All 37 distinct required-member omissions across ten object instances reject in both validators. The 264 noncharacter positions are included in these totals. Nine named test groups pass under Node 26.10.0 and npm 11.19.1; retained Python 3.12.14 results agree. Fresh receipt readback confirms every input hash, size and expected verdict, matching valid canonical bytes, and preservation of all 394 historical cases in relative order. Candidate profile limits and operational adoption remain separate.

Lean 4.24.0 accepted the final exact-source unit with 95 named theorems and 239 audited declarations, using trust level zero and kernel type checking. Only propext and Quot.sound occur. The fixed seven-slot assessment preserves reasons, makes failure dominate missing evidence, and prevents a pending required slot from yielding Pass. The state-simulation certificate covers the four-node diamond, edge mask 51, all 256 immutable atomic-oracle assignments and seven prefix ordinals: 1,792 positions, not a distinct-state count. Reachable-state induction supports the selected safety results. The initial Classical.choice audit failure and its constructive correction remain distinguishable. The all-64-graph Lean attempt was terminated without acceptance. TLA correspondence is a reviewed manual transcription; machine-checked translation and temporal-liveness transfer are unproved.

The separate retained TLC model covers all 64 validated forward-edge graphs on four nodes. Its positive runs cover 5,184 noncrash and 16,384 crash input cases, checking 12 invariants and four temporal properties under the model's assumptions. The bad-aggregate and unfair-scheduling controls produce the expected counterexamples. The v2 portability correction verifies two relocations and nine rejection controls; it does not rerun TLC or expand its scope.

The corrected JCS receipts compare canonicalize 5.1.0, rfc8785 0.1.4 and Java canonicalization 1.1 in separate processes: 10,034 public positions, including 10,032 exact outputs and two required nonfinite rejections, plus two synthetic companion comparisons. There are no mismatches; both byte-corruption controls pass. Exact clean source revisions are enforced, with 18 early rejection checks. These implementations share reference ancestry; finite agreement does not establish clean-room independence or universal conformance. Separately, 22 Lean digest-codec theorems and 12 Node tests support the raw32/lowercase64 projection. Full JSON/JCS inverse laws, whole-source equality, legacy projection definitions, trust/adoption gates and whole-finding closure remain outside these results.

16.6.1. Verification scope and preservation

An independent read-only review freshly recomputed the retained package hashes, compared every archive member with the extracted bytes, reconciled the package manifests and compiler audit logs, and verified the detached signature of the exact PairWireRecovery source. All 644 files across the four archives matched: 355 in the canonicality-reader correction, 147 in pair-wire

recovery, 128 in native-source evidence and 14 in the public signature package. The correction, pair-wire and native manifests respectively bind 354, 146 and 106 files; these manifest counts must not be confused with complete archive-member coverage. In particular, the native manifest omits its reader execution receipts, whereas the separately measured full archive covers those receipts. All 60 files of the original canonicality packet were also compared directly with the separately retained original ZIP and remain byte-identical; its SHA256 is `c01aeb5fb7bc81e55f4d0d5c1a07767d803274eb344868a92c57928aee160a4f`. No bundled verifier, compiler, native build or corpus suite was executed during this review.

16.6.2. Canonicality-reader correction

The reader correction addresses the historical integrity-check bypass under optimized Python, without changing the retained Lean sources or upgrading the scope of their proofs. Fresh static inspection finds zero Python Assert nodes in the successor reader and its regression driver, compared with 22 in the original reader. The retained regression receipt and hash-matched logs record two successful successor readbacks, 12 rejections covering six corruption/missing-file/unsafe-path scenarios in normal and optimized modes, and two historical controls. The historical normal run rejected a recorded one-byte source corruption; its optimized counterpart incorrectly accepted it. Those 16 recorded reader runs started zero Lean processes. The disposable corrupted source itself is absent from the retained bundle, so its one-byte mutation characterization is attributed to the historical receipt rather than remeasured here. The regression tests were not rerun in this review.

16.6.3. Actual ordered extraction-to-wire recovery

`PairWireRecovery.lean` contains exactly 15 new theorem declarations and two definitions, `recoverWire` and `recoveredWire`. Its exact source is 5,568 bytes, SHA256 `51ea8ef76197ba53ca5a9e3bd1de6f82fff8ff52ad37238c9933e29f2ac39eae`, including the final newline. All seven inherited source dependencies freshly match their earlier retained copies. The new source imports the unchanged `ParserRefinement` and `PairExtraction` modules, composes their actual ordered `occurrences` and `recoverOccurrences` operations with `encodePair`, and directly invokes the concrete `NCPSByteFraming.Canonicality.pair_canonicality` theorem. Thus the inverse conclusion is obtained from the actual codec and adapter rather than supplied as an abstract inverse-law premise. Every successful ordered occurrence list recovers its original accepted wire, and equal successful ordered lists imply equal wires. The statements also cover parser success and failure, encoded pairs, duplicate occurrences and the pair of empty payloads.

This recovery property applies to the ordered occurrence list. The set-valued extractor has an explicit swap collision and is not injective: finite-set projection loses order and multiplicity. The underlying `FiniteSet` is an extensional predicate with a finite-list witness in Prop, not an operational enumeration. The adapter excludes success with an empty set, which supports its restricted empty-erasure/failure equivalence. The generic Core erasure collision between rejection and success with an empty set remains intact.

16.6.4. Compiler and axiom evidence

The retained final receipt records Lean 4.24.0, trust level zero and kernel type checking enabled, with eight source-module compilations plus one version check, all exiting zero with zero reported errors or warnings. Fresh review reconciled all 119 recorded axiom audits with the retained compiler stdout: 15 new theorem audits, 98 inherited theorem audits, two new definition audits and four inherited definition audits. The 17 new declarations use only `propext` and `Quot.sound`; more precisely, `recoverWire` and `swapped_wires_distinct` have no axiom dependencies, `recoveredWire` depends on `propext`, and the other 14 new theorems depend on `propext` and `Quot.sound`. Nine inherited audited declarations retain disclosed `Classical.choice` dependencies. The earlier broad two-axiom policy rejection is preserved: all ten recorded compiler/audit/version commands succeeded, but the policy check rejected inherited Choice dependencies. It is not relabeled as final acceptance.

The figure 113 is a count of individually audited theorem entries, not a count of completed NCPS obligations or of all source declarations. The eight modules contain 162 theorem declarations in total; the 49 declarations in `ReferenceNCPSCore.lean` have compilation evidence in this run but no individual audit entries in this receipt. The retained portability evidence additionally records one relocated success and six optimized precompiler rejection controls, with no Lean processes started by the negative controls. None of these compiler or control runs was repeated in this review. Executable and top-level Std/Init pins are retained, but full transitive runtime attestation remains explicitly false.

16.6.5. Native-source evidence and semantic boundaries

Fresh review validates 81 selected native-source files totaling 6,869,519 bytes. The actual retained headers declare Node 26.10.0 with release flag 1 and V8 14.6.202.34 with candidate flag 0. The acquisition receipt identifies a 62,055,788-byte Node source archive containing 42,182 members, SHA256 `7b3a546d33cb7e15a43bdd7a57e0be5d5fd5ffc553e6e4c120033e66f0ba20c5`, and records matching selected member bytes and prior authenticated release checksums. The complete archive is excluded from the reviewed portable bundle: its hash/member comparison and the earlier release-signature authentication were read as retained evidence, not repeated here. Native reader logs retain normal/optimized success, five negative controls and an optional 81-member archive comparison. Default reader success covers 106 manifest-bound files, not all 128 archive members.

The selected code exposes relevant simdutf UTF8 validation, fast_float decimal parsing and Dragonbox number serialization, alongside JSON, string, sorting, Buffer and calendar paths. Source availability and location inspection do not prove native algorithm correctness, a checked call graph, source-to-installed-binary correspondence or correspondence with executable Lean semantics. Open bridges include the actual restricted JSON grammar, decoded-key duplicate policy and token-map/schema closure; UTF8/UTF16 and escaping; bounded decimal/Number/stringify behavior; JCS key ordering and traversal; and calendar, base64url, fingerprint, depth and byte-limit closure. The proposed pristine-CLI domain uses unique decoded-key/null-prototype objects, dense ordered arrays, scalar strings without normalization and a positive safe-integer revision. Its equality is extensional object-map equality with exact array, string and

integer values; raw JSON spelling and general signed-zero distinctions are outside that equality. Governing payload C and signing message M remain distinct from whole-envelope E and the typed byte-pair codec.

16.6.6. Exact-source signature and remaining closure

Independent code using the installed cryptography 50.0.0 library verified the pure Ed25519 detached 64-byte signature over the exact 5,568-byte source. The raw public key's SHA256 fingerprint is `b217cc5bae08acbfdb9f8cd20f420f0bee2ae39fb6f0b178ce060284c76f8e9f`; the fingerprint and signature bytes match the retained public metadata and signing receipt. An in-memory one-byte mutation was rejected. This establishes byte integrity and correspondence to that supplied public key. It does not authenticate a person's identity, freshly establish private-key custody or ongoing signer readiness, recompile the proofs, or sign the final manuscript, verification receipt or whole protocol. The separate Hello Ed25519 test signature was not checked in this review. No private key was accessed and no signature was generated.

The earlier handoff's physical-document blocker is now superseded: both original governing files have been recovered and freshly hash-matched. The plan at Library version 6 is 8,497 bytes, SHA256 `edc1c6624a1934d70df17f75f57ea9cec3144c13ef09c412db52b83f8a29a31f`; the design at Library version 5 is 42,873 bytes, SHA256 `928637790e81bccd91103641cd923e7975e96fb622d2056614631fade698ca06`. This closes physical recovery for those two documents only. Governing-source integration and exact authority-manifest binding, canonical v156 source/artifact equality, artifact/profile and policy/registry identity, signer identity and scope, trusted revision/replay handling, authenticated private GENESIS_REF mapping and operational adoption remain separate open conditions.

The corrected v2 matrix also supersedes the earlier handoff's characterization of R17, R19, R20 and R25 as pending literal gates. Those literal scoped mathematical gates are checked: fixed-seven assessment for R17; fixed-diamond transition simulation and reachable-state coverage for R19/R20; and the stated byte-carrier boundary lemmas for R25. A pure theorem does not require operational adoption to count as proved in its declared scope. Fresh readback of the corrected matrix still yields 43 partial whole findings and three pending whole findings, R08, R37 and R46, with zero closed whole findings. This readback does not rerun the gate proofs or silently reclassify the stable matrix. All 46 whole findings therefore remain unclosed, despite the achieved scoped gates. Neither the 35 prior scoped parser/extraction theorems plus 15 recovery theorems nor the 113 theorem audits establishes whole-NCPS, cryptographic or lifecycle/liveness completion.

16.7. Recovered governing documents and remaining gates

The original reproducibility plan and binding design were successfully materialized through supported Library access during this edition, closing their earlier physical-file availability gap. The plan is Library version 6, 8,497 bytes, SHA-256 edc1c6624a1934d70df17f75f57ea9cec3144c13ef09c412db52b83f8a29a31f. Its filename is dated 5 October; its internal heading identifies review revision 5 dated 6 October. Those fields are not interchangeable. The design is Library version 5, 42,873 bytes, SHA-256

928637790e81bccd91103641cd923e7975e96fb622d2056614631fade698ca06, internally dated 5 October. All 55 plan lines and all 65 design body paragraphs plus footer were read. The documents are byte-pinned inputs, not newly granted approval.

The recovered originals agree on payload JCS, unchanged historical source normalization, exact prefix/zero-byte/pure-Ed25519 framing, source order and separate personal display order. The real-binding attribution note in Appendix F.2 matches the approved source text. Schema-valid arbitrary note text does not satisfy that exact-note requirement. The older 1 MiB/64-container proposal and this candidate's 16 KiB/depth-8 walk remain alternative unadopted profiles; their different counting rules are explicit. A narrow finite signature-policy comparison does not settle the originals' strict point/scalar/cofactor obligations universally.

Current limits are specific:

- The integrated Markdown is the maintained proposed source for this edition; author approval, operational profile adoption, exact artifact/profile registry values and complete normative dependency identities remain pending. The historical plan/design input hashes are now known.
- Canonical v156 source df099d3ae4952a9966cf3b9e4ef299626dc7d338 remains unavailable in the recorded source check. The available clean checkout 20cdc916dbe9e39a8a498d7cc9782ddf27f6705b is not substituted. Whole canonical source equality remains NOT_MEASURED_SOURCE_UNAVAILABLE. Selected historical v155 comparisons do not establish whole-source equality.
- Complete native JSON/JCS semantic correspondence and source-to-installed-binary build correspondence remain unproved. Section 16.8 distinguishes the later checked literal-key, array, structural-depth and modeled guard results from reported projection results whose full source was not independently checked. Those results are not a full native JSON parser or a universal JCS proof. Generic typed integrity preimages and the exact eight-field legacy projection retain their separate obligations.
- Independent signer identity and scope, authenticated current revision/replay state, trust/revocation policy, and required private-reference linkage remain unestablished. A public source signature does not fill these operational gates.
- The stable v2 matrix records 43 partial and three pending whole findings, with no fully closed findings. Nine narrower candidate gates are complete, including literal scoped mathematical gates R17/R19/R20/R25. Later parser and pair-wire attachments are additional evidence, not a blanket reclassification of all 46 rows.
- The separately delivered matched review conversion now contains the consolidated NCPS content in RFCXML, RFC-format text and a PDF. Its schema/content checks and its later exact-byte signatures are identified in Section 16.8. Publication-style checks still report limitations; no IETF acceptance follows. Validation of this successor is separate from those predecessor results; this successor is unsigned. External approval, legal/compliance assessment and companion field deployment remain pending or unrun.

The next acceptance decisions are bounded: select the versioned profile, resolve the legacy schema if it remains required, establish the remaining native/governing correspondence and operational evidence, and then independently assess each original finding against its exact acceptance gate. This edition is useful for review without asserting those missing results.

16.8. Later reviewed evidence and preserved boundaries

This successor incorporates completed evidence reviewed after the historical 21:27 UTC cut in Section 16.6. Its evidence cut is 10 October 2026, 00:17:36 UTC. The preceding historical receipts and the stable 46-finding matrix are preserved. Package totals overlap through inherited definitions and theorems; they are not added into a count of independent protocol proofs or a percentage of completion. This manuscript update does not rerun Lean, native validators or application suites.

The independent reviewer reconstructed the available exact source and receipt texts and checked their stated hashes. The retained compiler evidence for the checked key, array, structure and guard groups uses Lean 4.24.0 with trust level zero and kernel type checking enabled; their final recorded compiler exits are zero with no errors or warnings. The reviewer did not independently rerun that compiler. Full projection-source and receipt reconstruction was unavailable, so that group's larger counts remain reported results. Appendix Q.1 binds the distinct evidence identities and review levels.

16.8.1. Literal schema keys, array constraints and structural depth

ActualSchemaKeys supplies 40 theorem declarations, five facts for each of eight literal key tables: table agreement, uniqueness, ASCII membership, observed sorted order and member permutation. The 27 helper and data definitions are outside that theorem-audit count. Independent comparison matched those tables to the ten retained schema object instances and the inspected validator tables. The exact source and receipt text hashes and all 40 theorem names matched; the full axiom stdout was truncated. The receipt reports an empty axiom union. These results concern the actual extracted tables; they do not prove the extraction procedure or JavaScript execution correspondence.

ActualArrayLengths supplies 19 theorems for the two extracted array-constraint projections. The checked schema-node byte ranges and the retained validator guard lines agree on two required positions, minimum and maximum length two, and no extra items. The lemmas establish the exact pair, indexed positions, and wrong-length/nonarray rejection in the model. They do not establish item-content restrictions or native execution. The receipt reports an empty axiom union. Its separate native corpus records 22 positions over 21 distinct inputs, with two passes and 20 rejections; that corpus was not independently replayed or reread in this review.

The structural group adds 24 theorems: four general structural lemmas and 20 profile theorems. Its 59 inherited theorems are the preceding 40 key and 19 array results, giving 83 audited declarations with an empty reported axiom union. Independent review matched all 42 inventory entries to the full pinned schema, including paths, roles, depths, key sets and array lengths. The profile contains ten objects, two arrays, 30 primitive leaves and 37 required members. A fitting value has structural height exactly six; an explicit fitting fixture establishes nonvacuity. Ordered

object coordinates are a model convention whose connection to ordinary objects requires projection. Scalar-value restrictions and native correspondence remain outside this structural result.

16.8.2. Projection and modeled guard execution

The projection handoff reports 91 new theorems, one new instance and 83 inherited theorems, for 175 audits with at most propext. The full source and receipt were not independently reconstructed or hash-verified. The displayed computational project/takeField and mutual Preserves definitions were reviewed: they consume fields, preserve exact primitive values and indexed array positions, and reject leftover members. Their definition is not circular. The generic first-match lookup has a duplicate-key caveat; the unique-key profile permits the intended reordering interpretation. These inspected definitions support a bounded reading of the reported project/Fits/Preserves result, not independent verification of all 91 proofs. This is not evidence for the separately unresolved eight-field legacy ClosureResult projection.

The independently reviewed guard foundation adds 41 theorems: 21 constructor, seven guard and 13 profile theorems. It retains 114 earlier theorems and one instance, for 155 theorem audits and 156 audits including the instance. The exact three source texts, built-in boundary record and full receipt were reconstructed and hash-matched. The retained audit uses only propext and Quot.sound. No material flaw was found in the reviewed proofs, and an explicit successful fixture avoids a vacuous-success result.

Within the bounded decoded Value model, successful computed structural guards yield a fitting, preserving projection, and both the input and projected value have height six. Modeled construction establishes unique own enumerable data members and dense-array visitation invariants. The 21 native cases in its receipt are not 21 successful validations: they include 12 parse successes, two full validation successes, five structural-guard successes and four direct nonparser counterexamples. These categories overlap and are not additive coverage claims.

Actual parseStrict/validate execution has not yet been proved to establish those model invariants and guards. Faithful scalar/constructor export, source translation and built-in refinement, the parser's maxDepth link, and scalar restrictions outside the structural guard remain open. The retained verifyBytes interface returns accepted:false even on schema success, so an implication conditioned on accepted=true would be vacuous. A substantive runtime bridge must target successful parseStrict plus validate and then connect their actual source behavior to the checked model. The later source-subset translation task was still in progress in the last recorded status used for this edition; no unfinished or unreviewed result is claimed here.

16.8.3. Signatures on specific predecessor and v161 artifacts

The v161 public package binds a historical application observation to its owner-requested signature and a separate observation-binding manifest. The observation names canonical source d1b1cd852927052b54af2b540b031d77311ccecf and site source 76a08b2d54679238416ee87818732ac441c5f933. It records exclusion of two historical distribution archives from source transport while reporting the other Git blobs unchanged. Its original build/test and provider observations remain attributed to that record, rather than rerun by signing or by this manuscript update.

Independent verification checked pure Ed25519 signatures over the exact 6,763-byte publication-observation.json and 23,310-byte v161-observation-binding-manifest.json, rejected altered payloads and signatures, verified the 11 checksummed bundle files and matched the v161 metadata binding. Signing is recorded at 9 October 2026, 22:46:16 UTC; the underlying provider observation is dated 12:10:02 UTC. The signed record's locally hashed gzip-tar identity is distinct from the provider-reported stored-tar digest. Provider archive bytes were not independently hashed, and no fresh rebuild or runtime HTTP health check was established by this signature review. This is an owner-requested operational signature of historical evidence, not a provider-issued attestation or an operational companion payload.

The separately delivered matched manuscript predecessor also has three independently checked detached pure Ed25519 signatures, over its unchanged XML, PDF and RFC-format text. The recorded signing time is 9 October 2026, 23:54:56 UTC; independent review completed at 10 October 2026, 00:17:12 UTC. All three signatures verified, and all six one-byte payload/signature mutation controls rejected. The public-key raw-byte SHA256 fingerprint is b217cc5bae08acbfdb9f8cd20f420f0bee2ae39fb6f0b178ce060284c76f8e9f, matching the separately retained operational workflow. Attribution to Samuel J. Kott is owner-requested; public-key verification does not independently establish legal identity, custody or trust registration. The recorded signing time is not an external trusted timestamp.

Those predecessor signatures cover exactly the artifact hashes in Appendix Q.1. They do not cover this changed successor. The reconstructed signing receipt itself is unsigned. The signer's archive inventory and secret-exclusion claim remain signer-reported because that archive was unavailable for independent inspection. No verified operational companion payload/binding instance has been resolved or signed. The companion's exact domain-separated message, independent authorization/trust and replay state, and authenticated private GENESIS_REF mapping retain their separate gates.

The predecessor conversion independently accounted for all 825 Markdown source blocks and visually checked its 97-page PDF. Official RFCXML schema validation and rendering succeeded. Idnits submission mode reported zero errors but a fail result with warnings: XML had 141 warnings and text had 51. Normal mode additionally reported three XML errors and four text errors, including heading/reference structure, canonical BCP14 boilerplate and long literals. The source contains a chapter-scoped requirement statement; it is not the canonical keyword-list boilerplate expected by the checker. Some reference diagnostics arise from the parser treating a wrapped bibliographic year as a section boundary, while valid xref attributes trigger its deprecated-element warning. These distinctions do not make the full publication-readiness result pass. Successor validation is recorded separately; neither conversion constitutes submission or RFC assignment.

16.8.4. Application viewer evidence and current closure

The newly supplied iOS development receipt, observed on 9 October around 11:11–11:18 UTC, reports a successful Debug simulator build and installed-byte matches for the executable, Info.plist, humanoid model and provenance record. It reports visible full-body rendering, button-

based rotation/zoom/reset, portrait and landscape framing, and reopening after cold relaunch. Drag/pinch gesture behavior was not visibly verified. This edition reads the receipt; it does not rerun the build or inspect its underlying screenshot suite.

That receipt classifies the humanoid as a cosmetic display shell. It records Developer Mode enabled on the connected physical iPhone but no physical install attempt, no valid Apple codesigning identity or provisioning profile, no signed archive or IPA, and no upload receipt. Its earlier unsigned archive does not contain the new viewer. These are observations at the recorded time, not a fresh device check. The receipt explicitly leaves manuscript-signature work and companion trust/payload/current-state gates unchanged. An operational document-signing key does not itself supply Apple development or distribution signing authority.

The result of this successor is a more complete and chronologically accurate evidence record. It does not reclassify all 46 findings, adopt the candidate profile, prove the whole JavaScript/runtime stack, establish native-to-binary correspondence, authenticate a live companion revision or private mapping, or demonstrate companion field deployment. The stable matrix still records 43 partial and three pending whole findings with none fully closed; later scoped achievements are recorded without silently rewriting that historical snapshot.

17. Prioritized repair and acceptance plan

17.1. Immediate editorial and technical decisions

Maintain this work as one paper with one governing definition of each symbol and one current interpretation of every requirement. Preserve the source manuscript and signatures separately. Review the new formal chapter as a proposed consolidation, not a silent change to previously signed artifacts. Expand NCPS once using the author-approved name; no expansion has been invented here.

First, review and explicitly adopt or revise the candidate signature-acceptance policy, preserving the 165 historical backend differences and any later policy-specific results. Second, decide the single proposed JSON intake/depth/size profile and supply the exact legacy eight-field schema/projection if legacy output is still required. Third, discharge the typed realization, codec, closure, termination and correspondence obligations against pinned definitions, labeling any existing small-core or finite-model result by its actual scope. Fourth, provide the declared state/refinement model with independent trust and freshness context and checked safety/liveness obligations. Fifth, complete independent conformance and field evidence before an operational security claim. None of these decisions is inferred from this unsigned integration.

17.2. Proofreading findings that change mathematical meaning

The review register contains 46 findings with locations, actions and acceptance gates. The most consequential corrections are type/value separation, the extra arrow in the characteristic-family signature, relation-registry identity versus extensional equality, conflicting relation arities, non-

Boolean status enums, the hidden mechanization input, blocked versus failed checks, unsupported closure monotonicity/compositionality, and the difference between one-way refinement and equivalence.

The byte foundation now uses injection images and a distinct empty tuple and empty sequence. The canonical serializer's inverse requires an injective schema projection. Whole-document UTF-8 and decoded Unicode checks replace text-field-only validation for JCS. The paper does not infer consistency, decidability, unique models, universal conformance or an actual genesis derivation merely from naming a Boolean predicate or a semantic tuple.

17.3. Release gate

A reviewed release needs an approved integrated source, version-pinned profile and schemas, discharged proof obligations for the claims actually made, reproducible scoped conformance results, and independent evidence for operational assertions. A formatting repair is not a new signature, deployment or certification. An honest incomplete result is preferable to an unsupported passing label.

The release gate stays closed for any unresolved mandatory condition. This candidate and its clause map establish a reviewable proposed source, not owner approval, a new signature, an external send, a merge, a deployment or certification. No original source, prior signature or historical evidence artifact is replaced.

Retained prefatory material

Integrated review status

Contributor: Lauren J. Anderson
AI assistant: Ed25519 (OpenAI-powered drafting and review assistance)

The assistant designation is separate from Ed25519, the cryptographic algorithm specified in RFC 8032.

Candidate status: unsigned and NOT ADOPTED. This source integrates the eight correction subjects and seven reconciliation proposals into their governing sections. It is a proposed successor to the repaired review paper, not a second addendum and not an approved normative release. All candidate limits, depth conventions and signature-acceptance choices remain NOT ADOPTED. The accompanying clause map identifies each replacement and retained range. Existing signatures authenticate only their original artifacts; none authenticates this candidate. Integration adds no proof, conformance run, signing authority, external approval or deployment receipt.

Evidence successor: This edition retains the title and governing proposed definitions while adding the later, scoped evidence summarized in Section 16.8. It preserves the 9 October historical snapshots and the separately signed predecessor files. It does not inherit their signatures. A proof about a defined Lean model, a recorded native test, an independently

checked artifact signature and an operational companion acceptance are separate results. The exact source and delta manifest identify this successor; unresolved runtime, trust, private-linkage and whole-finding gates remain explicit.

Contents

1. Community and problem statement
2. Approach and relationship to IETF work
3. Claim format and attribution model
4. Making a claim and constructing its signature
5. Signing authority and trust policy
6. Verifying claims and rejecting unsupported acceptance
7. Private-reference resolution
8. Implementation and field-deployment evidence
9. Operational considerations
10. Unresolved implementation and acceptance gates
11. Security Considerations
12. Proposed applications and research boundaries
13. IANA Considerations
14. Authorship and attribution
15. NCPS formal model: consolidated review text
16. Fresh verification, recovered evidence and remaining gaps
17. Prioritized repair and acceptance plan
Appendix Q. Checked declarations and edition evidence
Appendix A. Technical evidence index and verification scope
Appendix B. Frozen source example
Appendix C. Retained technical evaluation and manuscript evidence
Appendix D. Exact recorded block-zero hash
Appendix E. Chronological project record
Appendix F. Attribution wording and historical checks
Appendix G. Requirements evidence and acceptance criteria
Appendix H. Public manuscript and diagnostic receipt identities
Appendix I. Complete worked commitment and package-signature calculation
Appendix J. Additional research methods and boundaries
Appendix K. Navigation and vision research context
Appendix L. Application evidence and reproducibility details
Appendix M. Proposed implementable validation profile and fixtures
Appendix N. Proposed bounded field deployment and receipt checklist
Appendix O. External engagement and review boundaries
Appendix P. Additional public-key and digital-geometry evidence

Reader’s guide

One paper; distinct evidence classes. This proposed integration candidate brings together the recovered research manuscript and the NCPS formal definitions, with dated verification records retained at their original scope. Historical source commitments, byline and attribution are preserved. Section 15 contains one governing **proposed** definition for each integrated concept. The appended correction actions are reconciled there and at the cited manuscript clauses; their conflicting formulations are not parallel authorities. Integration is not acceptance of the proposed semantics or proof obligations.

The retained manuscript is reproduced with consistent headings, tables, references and readable source examples. Its dated observations remain historical observations. The formal chapter distinguishes proposed definitions, proof obligations and verified results. New empirical results identify their exact inputs and execution scope. No changed manuscript inherits a predecessor’s signature.

IETF supporting-document status: This integrated NCPS review accompanies draft-kott-numina-pair-binding-02 as a distinct supporting document. It does not replace the existing IETF manuscript, inherit its signatures, or establish an IETF submission or endorsement. The bundle index and SHA-256 manifest bind the exact accompanying artifacts.

Edition status: unsigned review copy, not an external publication or certification. No deployment, trust-policy adoption or private-key operation was performed for this review. The exact predecessor and new evidence identities are recorded in the companion manifest. References to uninspected underlying records are not promoted to fresh verification.

Evidence time boundary. In retained Sections 1–14, Section 16 and Appendices A–P, words such as fresh, new and current refer to the dated source observation or run described in that passage. They do not report a rerun, present-day external readback or verification of this candidate's changed bytes. Historical 21-case, 49-assertion, 11-check and 919-vector results remain separate evidence classes. Additional evidence must name its own exact target, run time, toolchain and remaining limitations; no historical result is silently transferred to a new profile or source.

Navigation. Sections 1–14 contain the recovered manuscript’s main argument; Section 15 consolidates the NCPS formal core; Section 16 supplies fresh verification and recovered evidence; Section 17 prioritizes repairs and acceptance gates. Appendices A–P retain the manuscript’s history, source evidence, worked examples and related research context. The separate review register records 46 findings across the source manuscript and conversational lineage.

Retained manuscript note: Technical review candidate and evidence status

This separately prepared revision-02 candidate reorganizes Singularity: Ed25519, a Universal Companion Model around its proposed companion format and evidence. Preparation-status statements in the preserved predecessor described its state before later submission and detached

signing. This working successor is unsubmitted and unsigned; predecessor signatures do not authenticate its changed source bytes or a newly rendered PDF. Preparation is not publication acceptance or proof of implementation maturity.

Technical procedures, source identifiers, test timestamps and artifact digests are included for verification. Credentials, private keys, protected registry locators and unrelated personal circumstances are excluded. The original manuscripts and signed evidence remain unchanged. No RFC number or IETF endorsement is claimed.

This document retains the trust200902 IPR metadata; the associated published provisions are identified in [TLP5].

This working successor preserves the earlier signed artifact separately. It clarifies I-JSON character constraints and the deployment-profile verification-policy requirement. Its validation results are recorded separately; earlier formatting checks do not establish validation of these changed bytes.

References

Normative References

[RFC8785] A. Rundgren; B. Jordan; S. Erdtman. JSON Canonicalization Scheme (JCS). 2020 June. RFC 8785; DOI 10.17487/RFC8785. Informational. Source.

[RFC8032] S. Josefsson; I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). 2017 January. RFC 8032; DOI 10.17487/RFC8032. Informational. Source.

[RFC7493] T. Bray. The I-JSON Message Format. 2015 March. RFC 7493; DOI 10.17487/RFC7493. Source.

[OPENSSH-SSHSIG] OpenSSH Project. OpenSSH SSH Signature Format. 2026 October. PROTOCOL.sshsig; consulted 9 October 2026. Package framing is distinct from the manuscript companion format. Source.

[RFC4648] S. Josefsson. The Base16, Base32, and Base64 Data Encodings. 2006 October. RFC 4648; DOI 10.17487/RFC4648. Source.

[RFC3339] G. Klyne; C. Newman. Date and Time on the Internet: Timestamps. 2002 July. RFC 3339; DOI 10.17487/RFC3339. Source.

Informative References

[IN-TOTO] S. Torres-Arias; H. Afzali; T. K. Kuppusamy; R. Curtmola; J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. 2019. Proceedings of the 28th USENIX Security Symposium, pp. 1393-1410. Source.

[DSSE] Secure Systems Lab. DSSE Protocol. 2024 May 10. Version 1.0.2. Protocol specification. Accessed 5 October 2026. Source.

[NUMINA-DESIGN] Numina. Universal pair binding design. 2026 October 5. Version 5. Unpublished project design record. Approved commitments and descriptive attribution are summarized in this manuscript; the underlying private evidence is not a public dataset.

[TLP5] IETF Trust. Corrected Legal Provisions Relating to IETF Documents. 2015 March 25. Version 5.0. Effective 25 March 2015; terminology corrected 21 September 2021. Source.

[HISTORICAL-AUDIT] Numina. Historical source commitment audit, historical-audit-2026-10-05.json. 2026 October 5. Read-only run captured 5 October 2026 at 23:57:10.028 UTC, historical HEAD dc4c1f38340340609f66bc2ae7bf425427d4b864. Sanitized technical audit record; public deposition pending.

[V155-COMPARISON] Numina. Canonical v155 source comparison. 2026. Record date: 2026-10-06. Read-only receipt canonical-v155-comparison-20261006.json, 6 October 2026, 00:50 UTC. Selected artifacts and hash profile.

[PRIVATE-COMPUTATION] Numina. Numina Universal Pair Private Computation and Discovery Record. 2026. Record date: 2026-10-06. Privately retained source report, Section 8 page 8; public-safe history only. Protected values and raw appendices are not reproduced.

[FROZEN-EXCERPT] Numina. Privately retained research-motivation record. 2026 October 8. Record date: 2026-10-08. Private correspondence content and per-utterance detail are not reproduced; Appendix F.5 retains package-level capture and integrity metadata. Integrity evidence concerns captured bytes only.

[REV01-PUBLIC] Numina. Revision 01 public posting notice and byte verification receipt. 2026. Record date: 2026-10-08. Posting notice 07:02:08 UTC; XML/TXT public comparison 07:06:50 UTC. Source.

[REV01-VALIDATION] Numina. Numina revision 01 final format validation record. 2026. Record date: 2026-10-08. Separate retained validation record for exact final XML/TXT/PDF; modes and unresolved normal TXT findings disclosed.

[REV01-SIGNATURES] Numina. Revision 01 final detached three-file signature receipt. 2026. Record date: 2026-10-08. Receipt 05:49:43 UTC; exact file checks independently repeated at 10:36:52 UTC.

[PDF-RESIGN] Numina. Later detached signature on unchanged revision 01 PDF. 2026. Record date: 2026-10-08. Receipt 10:33:24 UTC; independent identity and operational authority not established.

[PAIR-DIAGNOSTICS] Numina. Numina public pair diagnostic protocol and results. 2026. Record date: 2026-10-08. NEW_DIAGNOSTICS_NOT_DISCOVERY_PROOF; protocol frozen 10:44:46 UTC; outputs 10:45:28 UTC.

[CONTROLLER-REPORT] Numina. Bounded controller evaluation report. 2026. Record date: 2026-10-08. Historical-source offline synthetic evaluation; performance and safety gates failed; no current-hosted or real-world predictive result.

[V156-PROVENANCE] Numina. Numina v156 saved-version and deployment-status observations. 2026. Record date: 2026-10-08. Provider provenance recovered 8 October; source df099d3ae4952a9966cf3b9e4ef299626dc7d338. Exact archive-byte verification not performed.

[WANG-GPSINS-2020] Renjie Wang; Xiaojian Hou; Fei Liu; Yao Yu. GPS/INS Integrated Navigation for Quadrotor UAV Considering Lever Arm. 2020 October. DOI 10.1109/YAC51587.2020.9337634. 2020 35th Youth Academic Annual Conference of Chinese Association of Automation (YAC), pp. 132-136. Source.

[HAI-ILLUMINATION-2021] Jiang Hai; Yutong Hao; Fengzhu Zou; Fang Lin; Songchen Han. A Visual Navigation System for UAV under Diverse Illumination Conditions. 2021 December. DOI 10.1080/08839514.2021.1985799. Applied Artificial Intelligence, vol. 35, no. 15, pp. 1529-1549. Source.

[MUGHAL-MATCHING-2021] Muhammad Hamza Mughal; Muhammad Jawad Khokhar; Muhammad Shahzad. Assisting UAV Localization Via Deep Contextual Image Matching. 2021. DOI 10.1109/JSTARS.2021.3054832. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 14, pp. 2445-2457. Source.

[KRUL-SLAM-2021] Sander Krul; Christos Pantos; Mihai Frangulea; João Valente. Visual SLAM for Indoor Livestock and Farming Using a Small Drone with a Monocular Camera: A Feasibility Study. 2021 May. DOI 10.3390/drones5020041. Drones, vol. 5, no. 2, article 41. Source.

[COUTURIER-LOCALIZATION-2021] Andy Couturier; Moulay A. Akhloufi. A review on absolute visual localization for UAV. 2021 January. DOI 10.1016/j.robot.2020.103666. Robotics and Autonomous Systems, vol. 135, article 103666. Source.

[DING-CROSSVIEW-2021] Lirong Ding; Ji Zhou; Lingxuan Meng; Zhiyong Long. A Practical Cross-View Image Matching Method between UAV and Satellite for UAV-Based Geo-Localization. 2021. DOI 10.3390/rs13010047. Remote Sensing, vol. 13, no. 1, article 47. Published online December 24, 2020; final issue citation year 2021. Source.

[HE-ODOMETRY-2020] Ming He; Chaozheng Zhu; Qian Huang; Baosen Ren; Jintao Liu. A review of monocular visual odometry. 2020 May. DOI 10.1007/s00371-019-01714-6. The Visual Computer, vol. 36, no. 5, pp. 1053-1065. Published online June 25, 2019; final issue May 2020. Source.

[BARISIC-FOLLOWING-2019] Antonella Barisic; Marko Car; Stjepan Bogdan. Vision-based system for a real-time detection and following of UAV. 2019. DOI 10.1109/REDUAS47371.2019.8999675. 2019 Workshop on Research, Education and Development of Unmanned Aerial Systems (RED UAS), pp. 156-159. Source.

[EFE-DFM-2021] Ufuk Efe; Kutalmis Gokalp Ince; A. Aydin Alatan. DFM: A Performance Baseline for Deep Feature Matching. 2021. DOI 10.1109/CVPRW53098.2021.00484. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 4279-4288. IEEE DOI pagination; the CVF accepted version is paginated 4284-4293. Source.

[BIAN-GMS-2020] Jia-Wang Bian; Wen-Yan Lin; Yun Liu; Le Zhang; Sai-Kit Yeung; Ming-Ming Cheng; Ian Reid. GMS: Grid-Based Motion Statistics for Fast, Ultra-robust Feature Correspondence. 2020. DOI 10.1007/s11263-019-01280-3. International Journal of Computer Vision, vol. 128, no. 6, pp. 1580-1593. Published online December 17, 2019; final issue June 2020. Source.

[ZHAO-KCF-2021] Fan Zhao; Kaidi Hui; Tingting Wang; Zhenzhen Zhang; Yajun Chen. A KCF-Based Incremental Target Tracking Method With Constant Update Speed. 2021. DOI 10.1109/ACCESS.2021.3080308. IEEE Access, vol. 9, pp. 73544-73560. Source.

[LI-SCALEKCF-2015] Yang Li; Jianke Zhu. A Scale Adaptive Kernel Correlation Filter Tracker with Feature Integration. 2015. DOI 10.1007/978-3-319-16181-5_18. Computer Vision - ECCV 2014 Workshops, Lecture Notes in Computer Science, vol. 8926, pp. 254-265. Source.

[BOLME-MOSSE-2010] David S. Bolme; J. Ross Beveridge; Bruce A. Draper; Yui Man Lui. Visual object tracking using adaptive correlation filters. 2010. DOI 10.1109/CVPR.2010.5539960. 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, pp. 2544-2550. Source.

[FAN-TRACKING-2023] Jiwei Fan; Xiaogang Yang; Ruitao Lu; Weipeng Li; Yueping Huang. Long-term visual tracking algorithm for UAVs based on kernel correlation filtering and SURF features. 2023. DOI 10.1007/s00371-021-02331-y. The Visual Computer, vol. 39, no. 1, pp. 319-333. Published online January 22, 2022; final issue January 2023. Source.

[RFC3552] E. Rescorla; B. Korver. Guidelines for Writing RFC Text on Security Considerations. 2003. RFC 3552; DOI 10.17487/RFC3552. Source.

[RFC9334] H. Birkholz; D. Thaler; M. Richardson; N. Smith; W. Pan. Remote ATtestation procedureS (RATS) Architecture. 2023. RFC 9334; DOI 10.17487/RFC9334. Source.

[RFC9052] J. Schaad. CBOR Object Signing and Encryption (COSE): Structures and Process. 2022. RFC 9052; DOI 10.17487/RFC9052. Source.

[APP-CONTROL] Numina. Numina singular control plane 1.0.0. 2026. Retained Control-Plane-v1.md, Library version 2; extracted 9 October 2026. Architecture/report evidence, not a current runtime attestation. Includes historical command and restart-test claims.

[APP-SOURCE] Numina. Retained Numina control-state implementation excerpt. 2026. Retained server/index.js, Library version 15, lines 2070–2309. Static inspection of grant, authorization, evidence, reconciliation, read and commit paths. Source table numina_control_manifest differs from the older document label numina_control_state.

[APP-SECURITY] Numina. Numina security and control model. 2026. Retained document date 15 September 2026, Library version 2; historical controls, test/production observations and open requirements; the current web copy can differ. Source.

[INCOMPLETE-COMPANION] Numina. Incomplete unsigned companion and operational-record lookup. 2026. Retained private review-v2 JSON; SHA-256 (concatenate two hexadecimal groups): 3ba5ac231f117a8091ac13fc25b709a1 6ddbe642d5f0fa75c4f792de262db381. Field inventory and bounded lookup only; no complete operational payload or trust receipt recovered.

[TECHNICAL-CATALOG] Numina. Numina project evidence matrix and technical evidence review bundle. 2026 October 9. Companion evidence catalog E01–E15 and verification records V01–V05. Bundle Numina-ISE-Technical-Evidence-Review-2026-10-09.zip, 120995 bytes, SHA-256 (concatenate two hexadecimal groups): 472e0f052b7760cd034e409f2a1d624e d6c57f22a52156d6aef12fde6fd855b7; separately supplied for authorized review, not claimed publicly deposited.

[RFC8792] K. Watsen; E. Auerswald; A. Farrel; Q. Wu. Handling Long Lines in Content of Internet-Drafts and RFCs. 2020 June. RFC 8792; DOI 10.17487/RFC8792. Source.

[PUBLIC-KEY-READBACK] Numina. Public-only Ed25519 key correspondence and historical SSHSIG verification. 2026 October 9. Owner-supplied public-only verification package and fresh readback dated 9 October 2026. Package digest and exact public encodings appear in the evidence appendix. No private signer custody or companion authority is established.

[LOCAL-GEOMETRY-TESTS] Numina. Local digital geometry package and application verification records. 2026 October 9. Local geometry ZIP, exact suite log and verification report dated 9 October 2026. Fresh artifact/log inspection is distinguished from reported build/conformance execution. No physical fabrication, deployed adoption or companion field acceptance is asserted.

Additional references for the consolidated formal chapter

[RFC3629] F. Yergeau. UTF-8, a transformation format of ISO 10646. RFC 3629, November 2003. Source.

[RFC8259] T. Bray, editor. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, December 2017. Source.

[RFC2119] S. Bradner. Key words for use in RFCs to Indicate Requirement Levels. BCP 14, RFC 2119, March 1997. Source.

[RFC8174] B. Leiba. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. BCP 14, RFC 8174, May 2017. Source.

[LEAN-TYPES] Lean project. Theorem Proving in Lean 4: Dependent Type Theory. Consulted 9 October 2026; tool-version pinning remains required for any executed proof claim. Source.

[LEAN-PROOFS] Lean project. Lean Language Reference: Validating a Lean Proof. Consulted 9 October 2026. Source.

[LEAN-AXIOMS] Lean project. Lean Language Reference: Axioms. Consulted 9 October 2026. Source.

[TLA-REFINEMENT] Leslie Lamport. Hiding, Refinement, and Auxiliary Variables. Author-hosted technical text. Source.

[TLA-LIVENESS] Leslie Lamport. Safety, Liveness, and Fairness. 26 May 2019. Source.

Appendix A. Technical evidence index and verification scope

This appendix indexes the accessible technical evidence incorporated into this candidate. It distinguishes specified procedures, static inspection, historical reported executions, newly repeated byte-integrity checks and current observations. It is not an assertion that all project records or inaccessible source archives were reviewed. The accompanying evidence catalog identifies exact files, digests and access limits; unavailable evidence and failed checks remain explicit.

Table 4. Evidence classes and destinations

ID	Evidence and destination	Result and limit
M01	Original commitments, source/profile pins and v155 comparison; source example appendix	Selected inputs/profile matched; whole-repository equality false. No current-runtime identity inferred.
M02	Historical source tests and preservation checks; retained evaluation appendix and Appendix B	Reported source-specific results with exact commits/ hashes; not rerun in this document-editing task.

ID	Evidence and destination	Result and limit
M03	Revisions 00/01 manuscript format/ publication/signature records; retained evaluation and receipt appendices	Specific artifact/mode results retained, including normal-TXT idnits findings. Old signatures do not cover this candidate.
M04	Recorded block-zero value and collection method; genesis appendix	Record contains a value and forward input dependence; independent derivation/block-header reproduction absent.
M05	Public pair arithmetic/ hash diagnostics; retained evaluation and test inventory	Ten cross-tool agreements and thirteen unit tests reported; no unknown-genesis recovery.
M06	Bounded controller protocol, results, receipts and manifest	128 held-out synthetic episodes; performance and direct-input safety failure. Current audit reports 19 manifest size/ hash matches; execution results remain historical.
M07	OpenSSH SSHSIG package and raw Ed25519 calculation; worked-calculation appendix	Original package-specific integrity evidence; not the companion frame, current manuscript or independent identity.
M08	Application control architecture and server excerpt	Retained document plus static implementation corroboration; not fresh tests or current deployed-source equality.
M09	Application security and historical production observations	Reported test counts 272/287 and dated v74/v76 read/ reviewer exercises; no independent pentest or companion field deployment.
M10	Current public-head projection and v156 provider metadata	Projection does not verify current journal; provider succeeded metadata does not establish exact archive bytes or companion operation.

ID	Evidence and destination	Result and limit
M11	Incomplete companion, bounded 143-row operational lookup and acceptance checklist	Required operational fields/trust/current state/resolution not recovered; truncated values prevent an exhaustive absence claim.
M12	Current draft-editing validation	Earlier RFCXML formatting and submission-mode results apply to the predecessor artifacts identified in their dated receipts. This integrated NCPS edition is maintained as Markdown and rendered as editable DOCX and PDF; it is not an integrated RFCXML submission. Its exact source and render checks are recorded in the accompanying verification summary. Document validation does not establish operational companion acceptance.
M13	Public-key encoding and historical SSHSIG verification	Fresh public-only correspondence and signature checks. Operational custody, authorization, companion signature and field acceptance remain unverified.
M14	Digital geometry artifacts and local application tests	Fresh 25-file size/hash readback and measured OBJ bounds; retained 974-pass suite log. Build and conformance outcomes remain reported local execution evidence; no physical or companion deployment.

Table 11: Retained source label: Table 4

The manuscript categories M01–M14 above organize retained material. The separate catalog uses source IDs E01–E15 and verification IDs V01–V05; those are different indexes. The accompanying 120,995-byte technical bundle has SHA-256 472e0f052b7760cd034e409f2a1d624ed6c57f22a52156d6aef12fde6fd855b7. Its manifest covers 22 allowlisted payload entries with source/version and transformation details. The original controller package had 19 size/digest entries checked; fresh arithmetic readback checked all 128 stored held-out rows without rerunning the controller. [TECHNICAL-CATALOG]

The research-motivation record is retained as historical context. Its captured-byte integrity does not establish the underlying claims. [FROZEN-EXCERPT]

Appendix B. Frozen source example

B.1. Frozen source commitments

Source 1 is the final-build normalized SHA-256 commitment. Its source identifier is source_1 and its alias is UNIVERSAL_PAIR_1. Descriptive attribution: Lauren J. Anderson, label 1B. The original normalization profile must be retained [NUMINA-DESIGN].

```
<CODE BEGINS>1bf43817cf470abb868074a76598cb5f360517d7255b92ab3b409c6b545ec66b
<CODE ENDS>
```

Source 2 is the final-manifest SHA-256 commitment. Its source identifier is source_2 and its alias is UNIVERSAL_PAIR_2. Descriptive attribution: Samuel J. Kott, label 1A. The original input and hashing procedure must be retained [NUMINA-DESIGN].

```
<CODE BEGINS>1a17f584e23279684574022f39e533966ad09919f484e005c03948178a46aa0a
<CODE ENDS>
```

The two universal pair commitments above are associated in this document with canonical v155 source commit 17cc00cb87647a531f116f0894741005909c359c: Source 1 identifies public/final-build.json and Source 2 identifies public/final-manifest.json under the original hash profile in scripts/production-pair.mjs, lines 1-22 (1,568 bytes) [V155-COMPARISON]. The following raw-file and profile-segment SHA-256 pins identify the compared bytes; they do not replace the two existing source commitments.

Source 1 selected file, public/final-build.json, raw-file SHA-256:

```
<CODE BEGINS>2824c267ffa892d4c8013473b4236fd75571700da62f948cbf74975af0a6d2f2
<CODE ENDS>
```

Source 2 selected file, public/final-manifest.json, raw-file SHA-256:

```
<CODE BEGINS>b7aac505fde953ad99ff03e4523f21eef17b667d767cbfe30816c91a2d045762
<CODE ENDS>
```

Hash profile, scripts/production-pair.mjs lines 1-22, segment SHA-256:

```
<CODE BEGINS>389c963c4647751afca7b2ff29627b1462dfaa25e5903adf85d5627c594fc854
<CODE ENDS>
```

The comparison recorded at 6 October 2026, 00:50 UTC established byte equality for those inputs and that profile with the historical snapshot. This documentary association does not report an executed signing operation or change the intended scope of a later proposed release.

The owner-defined personal order is 1A first with Samuel J. Kott, then 1B with Lauren J. Anderson. Source order remains source_1 then source_2. Appendix F preserves the approved short labels and personToAlias map. These descriptive assignments do not establish signing authority or coauthor consent.

The two printed digests identify the original commitments only. JCS applies only to the new companion payload; it must not replace either source artifact's original hash profile. The legacy production-pair normalizer uses sorted keys, Object.fromEntries, and JSON.stringify; it is not a general RFC 8785 JCS implementation [HISTORICAL-AUDIT]. New manuscripts, payloads, envelopes, and packages receive separate digests.

Appendix C. Retained technical evaluation and manuscript evidence

This section separates historical source checks, operational companion requirements, manuscript-artifact verification and new research diagnostics. A result is attached to its exact target and observation. Later evidence updates the dated record without changing what earlier checks actually tested.

A read-only rerun captured on 5 October 2026 at 23:57:10.028 UTC reproduced both pinned commitments at historical local HEAD dc4c1f38340340609f66bc2ae7bf425427d4b864 [HISTORICAL-AUDIT]. Six focused checks and two existing production-pair tests passed; Git status was clean before and after. The canonical v155 source was unavailable locally during that audit. Comparison on 6 October at 00:50 UTC subsequently established byte-identical final-build.json, final-manifest.json, and hash-profile bytes at canonical v155 (production-pair.mjs lines 1-22; 1,568 bytes) [V155-COMPARISON]. The original commitments therefore carry over to those inputs/profile; no fresh test execution is claimed. Appendix F lists the historical checks.

Targets, evidence, observation times, and direct reasons. Not performed and evidence unavailable do not mean a check failed. Different refers only to the specified comparison. Times are UTC; an unrecorded time is not inferred. Scholarly submission of this design study does not require claiming that these operational checks have passed.

Table 5. Historical source checks and operational companion prerequisites.

Exact target and observation Result	Evidence and direct reason
Historical Source 1 and Source 2 at dc4c1f3 5 Oct 2026 23:57:10.028 UTC Result: PASSED	Audit [HISTORICAL-AUDIT] reproduced both frozen source commitments using the historical inputs/profile. Six focused assertions and 2/2 repository tests passed on Node 24.19.0; clean Git status was recorded. These are historical results.
Selected v155 JSON inputs and hash profile 6 Oct 2026 00:50 UTC Result: MATCHED	Receipt [V155-COMPARISON] compares source 17cc00cb87647a531f116f0894741005909c359c: both JSON files and production-pair.mjs lines 1-22 match the historical bytes. The commitments carry over from identical inputs/profile; this was not a fresh test run.

Exact target and observation Result	Evidence and direct reason
Whole v155 versus historical repository 6 Oct 2026 00:50 UTC Result: DIFFERENT	Receipt [V155-COMPARISON] records 126 added, 220 changed, 1,038 identical, and zero removed tracked entries. The full script differs. Whole-repository equality is false; the selected artifact/ profile match is unaffected.
Operational companion schema No validation time Result: NOT PERFORMED	No complete operational companion instance was available for strict validation. A private review candidate remains incomplete and unsigned. Successful RFCXML validation of a manuscript is a different target and does not close this row.
Operational companion attribution map No validation time Result: NOT PERFORMED	The approved manuscript map and note are preserved in the frozen source example and Appendix F. No complete operational payload was accepted for attribution-map validation. Documentary consistency is distinct from validation of an issued companion.
Operational companion signature bytes Source search: 6 Oct 2026, 00:50 UTC Result: NOT PERFORMED	No operational companion signature bytes were available. Later detached signatures over revision-01 XML/TXT/PDF are document-integrity evidence only, using different message construction. No invalid operational signature result is implied.
Independent signer trust entry for project, release, source and role Time not recorded Result: EVIDENCE UNAVAILABLE	No independently approved scoped trust entry was provided. The signer's authorization, key validity, and revocation status cannot be established from an embedded key or a display name.
Accepted current binding revision and predecessor state Time not recorded Result: EVIDENCE UNAVAILABLE	No trusted current-revision policy or authenticated current-state record was supplied. The accepted latest revision, predecessor linkage, and absence of stale or conflicting revisions cannot be established.
Private genesis reference resolution Time not recorded Result: EVIDENCE UNAVAILABLE	No authenticated immutable private-reference resolution receipt was recovered. The later located private computation record preserves a proposed relationship and historical diagnostics; it does not supply that receipt.

Exact target and observation Result	Evidence and direct reason
v156 provider provenance Historical check: 6 Oct 2026, 00:50 UTC Updated observation: 8 Oct 2026 Result: PROVENANCE RECOVERED	The 6 October check returned no v156 record. The later provider lookup identifies v156 source df099d3ae4952a9966cf3b9e4ef299626dc7d338 and a succeeded deployment. Provider updated_at is 7 Oct 04:51:50.230086 UTC, not an immutable event-time assertion. Exact source/archive bytes remain unverified in this reconciliation. [V156-PROVENANCE]
Downloaded release-archive bytes Check: 6 Oct 2026, 00:50 UTC; no comparison run Result: NOT PERFORMED	Receipt [V155-COMPARISON] states the release archive was not downloaded during the investigation. Provider metadata alone cannot establish equality of downloaded archive bytes.
New companion cross-implementation JCS and Ed25519 conformance No run time Result: NOT PERFORMED	No executed operational companion JCS/Ed25519 conformance suite is established. Manuscript-file signature checks and public arithmetic/hash diagnostics do not test the closed companion schema or its exact signing frame. Cited standards and proposed fixtures remain distinct from execution evidence.
Node 26 tests on the canonical v155 checkout No run time Result: NOT PERFORMED	No canonical Node 26 test run is recorded. Audit [HISTORICAL-AUDIT] used Node 24.19.0 on the historical checkout; comparison [V155-COMPARISON] performed no builds or tests. Neither is Node 26 execution evidence.

Table 12: Retained source label: Table 5

C.1. Tests required before an implementation claim

A public-safe evaluation package should provide version-pinned tools, documented hash profiles, and synthetic fixtures. Such fixtures can test canonicalization, signatures, and verifier behavior; they cannot reproduce the original private build commitment. The historical Source 1 hashed input includes private genesis material. Redacting that input changes its bytes and cannot reproduce the same commitment under the existing method [HISTORICAL-AUDIT]. Original-artifact verification must remain a separately labeled, authorized private check.

Future companion tests should change signed fields, source order, keys, fingerprints, scope, and revisions; reject duplicate or unknown keys, malformed encodings, invalid Unicode, and unsupported schemas; and compare the complete original inventory before and after adding a separate companion. RFC vectors and cross-implementation JCS checks are required. The

bounded executed fixture checks are recorded in Appendix M. Full operational conformance, positive signed-companion integration and independent-verifier agreement remain unestablished; the historical source tests do not verify a new signed binding.

No latency, throughput, storage-cost comparison, user study, formal security proof, or general deployment validation is presented. The historical result cannot support claims about those properties.

C.2. Manuscript publication and document signature evidence

Document-artifact checks below concern exact manuscript files. They do not validate the operational numina.pair-binding.v1 payload. The revision-02 candidate has separate validation outputs supplied alongside it after execution; this table does not predeclare their success.

Table 6. Completed manuscript observations and their limits.

Target, observation and result	Evidence and scope
Revision 01 posting notice 8 Oct 07:02:08 UTC Result: POSTED	The IETF notification confirms successful individual submission and repository posting. Retained mailbox authentication reports DKIM, SPF and DMARC passes. This is not RFC publication, IETF endorsement or signer-key identity verification. [REV01-PUBLIC]
Revision 01 public XML/TXT 8 Oct 07:06:50.663316 UTC Result: MATCHED	HTTP 200 responses, byte counts, SHA-256 and direct comparisons matched 61,903-byte XML and 69,493-byte TXT to final local files. The local 23-page PDF was not included in that public HTTP comparison. [REV01-PUBLIC]
Final revision 01 RFCXML and rendering 8 Oct validation record Result: PASSED FOR NAMED CHECKS	RFCXML v3 RelaxNG validation, xml2rfc 3.34.1 preptool/ TXT/PDF generation, 27 internal cross-references and 23-page visual review were recorded. Normal XML idnits 3.1.0 was clean; normal TXT had 1 error and 3 warnings. Submission mode XML/TXT was clean. These modes are not interchangeable. [REV01-VALIDATION]
Final three-file signatures Receipt 8 Oct 05:49:43 UTC; audit 10:36:52 UTC Result: VERIFIED FOR EXACT FILE BYTES	Pure Ed25519 separately covers XML, TXT and PDF. Python cryptography and OpenSSL passed for all three; raw key/ signature encoding checks passed. The included key has no independently established personal or operational identity. [REV01-SIGNATURES]

Target, observation and result	Evidence and scope
Later unchanged-PDF signature 8 Oct 10:33:24.753793 UTC Result: VERIFIED FOR EXACT PDF BYTES	A new detached signature and different key cover the same 108,079-byte final revision-01 PDF. Python/OpenSSL verification passed. Prior signatures remain unchanged; this is not an operational binding or manuscript-wide revision-02 approval. [PDF-RESIGN]

Table 13: Retained source label: Table 6

Historical receipt statements that revision 01 was not posted predate the notification and are preserved rather than rewritten. The later publication evidence updates status. Existing signatures remain tied to their exact artifacts; an edited revision requires a new separately approved signature.

C.3. New public input diagnostics

Protocol NUMINA-NEW-DIAGNOSTICS-20261008-A explicitly defines four new reference operations on the two public commitment values. Its classification is NEW_DIAGNOSTICS_NOT_DISCOVERY_PROOF. These are not asserted to be the owner-intended plus operation. Inputs are exactly 64 lowercase hexadecimal characters decoded to 32 raw bytes; integer interpretation is unsigned big-endian.

The protocol was frozen at 8 October 2026, 10:44:46.330350 UTC. Results were recorded at 10:45:28.410965 UTC. A strict transcription-length check corrected a missing two-character segment before freezing or executing diagnostics. No operation or criterion was retuned after observing outputs.

The operations are full unsigned integer addition, addition modulo 2^256, bytewise XOR and SHA-256 of ordered raw Source 1 bytes followed by raw Source 2 bytes. The hash input has no application prefix, delimiter, textual hex, newline or canonicalization. Reverse order is recorded separately.

Unsigned integer sum:

```
<CODE BEGINS>360c2d9cb1798423cbf476d69f7dfef5a0d5b0f119e072b0fb79e482dea57075
<CODE ENDS>
```

Sum modulo 2^256:

```
<CODE BEGINS>360c2d9cb1798423cbf476d69f7dfef5a0d5b0f119e072b0fb79e482dea57075
<CODE ENDS>
```

Bytewise XOR:

```
<CODE BEGINS>01e3cd932d7573d3c3f476885c7df8c95cd58eced1df72aefb79d47cde186c61
<CODE ENDS>
```

SHA-256 ordered raw Source 1 then Source 2:

```
<CODE BEGINS>c18a08a3d1e33344eed7cf921a53a1aace362a13a1fe626b699c2c5acbfc7702
<CODE ENDS>
```

SHA-256 ordered raw Source 2 then Source 1:

```
<CODE BEGINS>dbb73b5c96f9464cbcdfe433bf7a1afd4bfbf23c1ce488db1eb1e7c5aee7f735
<CODE ENDS>
```

Table 7. New diagnostic checks, not operational companion tests.

Target, observation and result	Evidence and scope
Strict reference implementation checks 8 Oct 2026 Result: 13 UNIT TESTS PASSED	Input validation, known answers, carry, modular wrap, XOR, determinism, commutativity and SHA-256 vectors passed. Full test names and bounds are in Appendix F. [PAIR-DIAGNOSTICS]
Cross-tool comparison 8 Oct 10:45:28 UTC Result: 10 CHECKS PASSED	Eight Python/Node forward/reverse comparisons and two OpenSSL hash comparisons passed. Node addition uses bytewise carry independently of Python integer arithmetic. Hash APIs may share OpenSSL, so this is not complete cryptographic implementation independence. [PAIR-DIAGNOSTICS]

Table 14: Retained source label: Table 7

The two sums coincide because this pair does not overflow 256 bits. Addition and XOR are commutative for these operations. Ordered concatenation hashes differ for this pair; that was an observation, not a universal theorem or pass condition. Unit tests distinguish scalar GF(2), where 1 + 1 = 0, from integer and mod-2^256 arithmetic, where 1 + 1 = 2. No interpretation is assigned to the owner statement by choosing a field post hoc.

No private genesis or purported target was accessed or compared. These diagnostic outputs do not replace the original commitments or establish discovery, derivation, signer authority, a third source or current production behavior.

C.4. Separate bounded controller study

The offline synthetic controller report examines checkout 20cdc916dbe9e39a8a498d7cc9782ddf27f6705b, runtime/verifier-rc26/numina_core.py lines 882-978, full-file SHA-256 c0d0d30748dba29e0c45943c62b0f8edfcc4b1f14903b2138c75a8342ae796c8. It does not attest current hosted behavior. The ranking score applies fixed penalties to caller-supplied adjustments and is not itself a future-outcome prediction.

The protocol used 32 development and 128 held-out four-step synthetic episodes, balanced across four regimes. The registered loss was missed legitimate requests plus three times accepted incident attempts. Hold was the primary comparator and fixed contraction a secondary comparator. Future traces were generated after action choice and replayed identically for each eligible candidate. No real observed outcome dataset was available.

The pre-run protocol SHA-256 is 42d16e7aabcc7dc219d13ef515aac615f128e004bd320ab3d9075bd74749cf00. An amendment and generator were pinned before first recorded outcomes at 8 October 06:16:42 UTC. The report identifies the amendment header time as self-attested and records file presence/hash checking by 06:16:29. A second run produced byte-identical result files; replay and consistency checks reported zero discrepancies.

Table 8. Executed research results with preserved failures.

Target, observation and result	Evidence and scope
Held-out primary performance 8 Oct 06:16:42 UTC first outcomes Result: FAILED 5% GATE	Controller loss 186,670; hold 187,656; fixed contraction 183,350. Improvement against hold was 0.525%, below 5%. The controller was 1.81% worse than fixed contraction under the specified toy objective. [CONTROLLER-REPORT]
Direct-input selection safety 8 Oct study Result: FAILED	A forged PredictiveTrajectory with altered eligibility flags and score caused choose_automatic to select an expanding trajectory over evaluated hold. It trusted supplied fields rather than recomputing them. No authority was activated. [CONTROLLER-REPORT]
Generated trajectory checks 8 Oct study Result: PASSED WITH LIMITED COVERAGE	Unique-ID generated cases passed nonexpansion, repeatability, permutation and input-preservation checks; separate boundary cases rejected several malformed inputs. These successes do not cancel the direct-input failure or establish duplicate-ID coverage. [CONTROLLER-REPORT]

Table 15: Retained source label: Table 8

The controller chose hold in 94/128 cases; against hold it won 6, tied 118 and lost 4. Hindsight regret totaled 36,354 toy-loss units. Results changed with the loss weighting: weight 1 was worse than hold while weight 10 was better. The study measures a bounded fixed-policy effect, not incident forecasting, verified next-best actions or universal value. It neither falsifies the selected-source comparison nor validates a binding.

Appendix D. Exact recorded block-zero hash

The retained deployment receipt and build record contain the following hexadecimal value, preserved without changing its characters. Chain identifier recorded in the receipt: 962421430. The record is evidence of the recorded value, not a fresh observation of an external chain.

```
<CODE
BEGINS>0xea95dac2ad854c835bc526d6be70a5a4614ccd47412a4bc710308b41010b53e1
<CODE ENDS>
```

The retained deployment-receipt JSON has raw-file SHA-256 5dd92b85eb52b7dbf8d13c172f541bd5f97fe3735945f7952407b6af894c31a4. The downloaded client/final-build.json has raw-file SHA-256 b7b40e98837338c57a20092bfccd770c142538d9e573825673706e7e66249fa4. These identify the downloaded records and are distinct from the historical public/final-build.json source comparison and normalized commitments in the frozen source example.

The described collector requests block zero using eth_getBlockByNumber with parameters ["0x0", false] and reads the returned hash field. Independent block-header hashing has not been demonstrated here. The build input already contains the genesis value before its source commitment is hashed. An exact source-pair-to-genesis derivation function remains unspecified in the retained evidence; no inverse derivation is claimed.

Appendix E. Chronological project record

The project design record [NUMINA-DESIGN] reports the following sequence. Dates are in 2026 and UTC. Git identifiers identify commit objects, not SHA-256 artifact commitments. The record distinguishes observed system events from the owner's discovery and attribution statements.

E.1. 9 September 2026

Sites project creation is recorded at 18:29:47. The first available Git commit follows at 18:41:58: f6ba52cb2ff567208c19afece65f71fe0e2a3602. The owner-stated discovery time is also 18:29:47. The Sites record confirms project creation at that time; it does not independently attest discovery of either exact pair commitment.

E.2. 10 September 2026

The v5 provider record reports a succeeded release 0.1.0 for source b9fd3dfbeda3f687702ad1641e58b6313a10d496. Its 18:06:44 updated_at value is a record-update timestamp, not an immutable publication timestamp. The release.json workerSha256 field does not match the SHA-256 of the committed worker bytes. That mismatch does not establish a missing final-manifest.

E.3. 11 September 2026

At 00:02:50, the application control journal records genesis/bootstrap revision 0 with owner as the recorded actor. The project evidence reports reconstruction of its state hash. This is an application bootstrap event; it does not establish chain genesis.

E.4. 18 and 24 September 2026

The final-build file first appears in the available Git history on 18 September. The final-manifest file first appears on 24 September. File appearance dates do not establish when their later exact digest values were discovered or completed.

E.5. 1 October 2026

At 03:54:02, both exact pair commitments first appear together in introducing commit 2ca81dcb2b31614ac1b59d7724e907b6e42f27d9. Its parent, 140602448c227b7e27c58f561890f8c83573598c, contains neither exact value. The original recorded offset time is 30 September at 22:54:02 -05:00. Joint introduction does not establish an order of discovery between the commitments. Separately, the local dc4c1f3 snapshot reproduced both commitments and reportedly passed six focused checks. Those reported results apply only to that historical snapshot; this manuscript does not present the individual outputs or claim current-source verification.

E.6. 5 October 2026

The v155 provider publication succeeded at 06:46:44 for source 17cc00cb87647a531f116f0894741005909c359c. Equality with that canonical source was still pending in the 5 October verification record. The time identifies this historical provider publication, not issuance or cryptographic signing of this manuscript. No v156 receipt or operational signed binding was established at that historical checkpoint; later provider provenance is separately recorded below. The separately captured 23:57:10.028 UTC rerun on 5 October examined only the historical local checkout [HISTORICAL-AUDIT]. A 6 October 00:50 UTC comparison subsequently established selected v155 input/profile equality [V155-COMPARISON].

The unchanged commitments carry the approved attribution preserved in this manuscript: Sam Kott is 1A / UNIVERSAL_PAIR_2; Lauren Anderson is 1B / UNIVERSAL_PAIR_1. Chronology establishes file and record order. The owner-stated discovery date and attribution order remain distinct from independently evidenced discovery, authorship or mathematical derivation.

E.7. 6 October private computation record

The private computation and discovery record preserves the owner account, the algorithmic proposal in Section 8 of that private record [PRIVATE-COMPUTATION], bounded integer diagnostics and source-history locators. Its report of a prior 3,417-file preservation inventory is historical and was not independently repeated for this revision. The protected original remains unchanged.

E.8. 7 October v156 provider update

The provider deployment record for v156 source df099d3ae4952a9966cf3b9e4ef299626dc7d338 reports succeeded and updated_at 04:51:50.230086 UTC. Later observation on 8 October recovered that provenance. The update field is not treated as an immutable event timestamp, and no exact archive-byte comparison is inferred.

E.9. 8 October manuscript and research events

05:25:59: a detached signature receipt identifies revision-00 XML. 05:40:14: a distinct earlier revision-01 XML signature receipt identifies 61,975 bytes. 05:49:43: the final three-file revision-01 signature receipt identifies the corrected 61,903-byte XML, 69,493-byte TXT and 108,079-byte PDF. Each signing stage used a separate key according to its receipt.

06:16:42: first recorded outcomes in the separate synthetic controller study; performance and safety gates failed. 07:02:08: authenticated IETF notification confirms revision-01 posting. 07:06:50: exact public XML/TXT comparison completed with matches. 07:08:01: the three-part frozen excerpt was captured; its timestamp is a capture time rather than every utterance time.

10:33:24: a later new detached signature covers the unchanged final revision-01 PDF. 10:36:52: the read-only evidence audit records final-file signature and frozen-package checks. 10:44:46: new public-input diagnostic protocol frozen. 10:45:28: results record ten cross-tool agreements, alongside thirteen unit-test passes.

The technical chronology distinguishes reported hypotheses and arithmetic diagnostics from a reproduced genesis algorithm, current release validation or universal proof.

Appendix F. Attribution wording and historical checks

In the approved attribution strings below, Sam Kott refers to Samuel J. Kott and Lauren Anderson refers to Lauren J. Anderson. The shorter labels are preserved in personToAlias and the software note; they do not supply signing authority or independently prove identity.

Samuel J. Kott remains the author and Lauren J. Anderson the human contributor. Ed25519 (assistant) identifies the AI-assistance credit and is distinct from Ed25519 (algorithm). The exact short display strings, personal order and two human-source assignments below and in the v1 fixture remain unchanged; the assistant adds no source commitment or signing authority.

F.1. Person to alias map

personToAlias: {"Sam Kott": "UNIVERSAL_PAIR_2", "Lauren Anderson": "UNIVERSAL_PAIR_1"}

F.2. Software attribution note

"UNIVERSAL_PAIR_1 labels the existing final-build normalized SHA-256 commitment attributed to Lauren Anderson (1B). UNIVERSAL_PAIR_2 labels the existing final-manifest SHA-256 commitment attributed to Sam Kott (1A). Personal attribution remains 1A first with Sam Kott, then 1B with Lauren Anderson. The two references are associated with one private genesis record."

The note is the owner-approved association statement. It does not convert private genesis linkage, current-source equality, signature validity, signer trust, or freshness into a verified result. The independent statuses and their evidence boundaries remain those reported in the implementation evidence and retained evaluation appendices. This appendix is manuscript content, not an issued or signed companion payload.

F.3. Historical reproduction checks

The read-only audit [HISTORICAL-AUDIT] used the original production-pair.mjs normalization procedure at the historical checkout. Its six in-memory checks all passed:

1. source_1_reproduces_expected: the final-build normalized digest matched the pinned Source 1 commitment.
2. source_1_excludes_binding_envelope: derived binding-envelope fields were excluded from the Source 1 digest.
3. source_2_reproduces_expected: the final-manifest digest matched the pinned Source 2 commitment.
4. source_2_excludes_binding_envelope: derived binding-envelope fields were excluded from the Source 2 digest.
5. build_digest_includes_historical_nusd_genesis: changing the private genesis input changed the Source 1 digest; its value is not disclosed.
6. manifest_has_no_genesis_or_pair_id_digest_input: the normalized Source 2 input excluded that private value and pair identifiers.

The separate command node --test tests/production-pair.test.mjs passed 2/2 existing tests: "production pair verifies both current sources and all four bindings" and "production source digest detects software changes while excluding its own bindings". In those test names, current refers to the historical local checkout. These results do not establish a companion signature, signer trust, freshness, private genesis resolution, or current canonical-source equality.

F.4. New public diagnostic test inventory

The thirteen unit tests are test_commutative, test_decode_exact, test_decode_rejects, test_determinism, test_integer_carry, test_integer_one_plus_one, test_modular_not_gf2, test_modular_wrap, test_self_xor, test_sha_abc, test_sha_empty, test_xor_known and test_xor_zero. All passed for the newly specified diagnostic implementation.

The ten comparison checks are Python versus Node for each of sum, modular sum, XOR and ordered raw SHA-256 in forward and reverse orders, plus OpenSSL SHA-256 for each concatenation order. All passed. These checks neither run the companion closed schema nor establish its JCS framing, signer authorization, freshness or private resolution.

Protocol, generator, implementation, results and logs are retained as a separate versioned research package. [PAIR-DIAGNOSTICS]

F.5. Frozen excerpt integrity checks

The read-only audit verified all six manifest payload entries, all seven ZIP members against local package files and all three transcript parts against the retrieved UTF-8 source strings. Capture metadata identifies 8 October 2026, 07:08:01.810055 UTC. No original audio or per-utterance timestamps were supplied. Checksums establish preservation of that excerpt, not simultaneous speech, speaker identity or statement truth.

Appendix G. Requirements evidence and acceptance criteria

This matrix separates the evidence presently available from the conditions needed for an operational claim. Passing a synthetic fixture or a historical source test cannot substitute for a missing release artifact, approved trust entry, or real signature. All original hash profiles and commitments remain fixed.

Table 9. Evidence and requirements

Area	Evidence now	Open requirement and acceptance criterion
Software attribution	Approved byline, names, exact short-label note, and personToAlias map are preserved.	Bind the approved map into the actual companion payload. Check Source 1 / UNIVERSAL_PAIR_1 / 1B / Lauren and Source 2 / UNIVERSAL_PAIR_2 / 1A / Sam exactly. Acceptance requires the approved attribution bytes and explicit scope; name labels alone confer no signing authority.

Area	Evidence now	Open requirement and acceptance criterion
Artifact integrity	Historical hash reproduction passed; selected v155 JSON inputs and hash-profile bytes are identical [V155-COMPARISON].	Retain the receipt. Under independent policy, pin the snapshot, paths, whole-file digests, profile implementation, and approved registry digest; reject unknown profiles or changed registries. Preserve legacy numeric-key ordering and exclusions. Require matching protected before-and-after inventories for preservation claims. Archive equality requires its own check; keep private inputs private.
Signed statements	The operational companion construction is specified; no complete signed companion was supplied and operational trust is unestablished. Separate detached manuscript signatures have verified exact revision-01 file bytes.	Approve exact payload, project/release/role scope, a real signer, an independent trust entry, and a current-revision policy. Acceptance requires separately successful signature, authorization, scope, and freshness checks. Null remains pending; malformed or failing signatures are invalid.
JCS	RFC 8785 applies to the new companion. The historical source normalizer is not general JCS. Public arithmetic/hash diagnostics do not constitute executed companion JCS conformance. The bounded canonicalization-fixture checks in Appendix M do not establish general JCS or operational conformance.	Run RFC vectors and independent canonicalizer agreement on public fixtures, including integer property names, Unicode, numbers, and array order. Acceptance requires byte-identical canonical payloads and rejection of malformed/duplicate input. Never replace the legacy source hashing procedure.
Ed25519	Pure Ed25519 companion framing is specified. Revision-01 file signatures passed Python/OpenSSL verification, but use exact full-file messages rather than the companion frame. No operational companion signature is supplied.	Test RFC vectors and synthetic positive/negative cases for the exact prefix, NUL separator, payload, encoding, and variant. Acceptance requires correct verification behavior plus a real approved signature for any operational claim. A test key cannot become production signing authority.

Area	Evidence now	Open requirement and acceptance criterion
Provenance	Chronological source events, selected v155 equality, revision-01 publication/byte verification, manuscript signatures and newly scoped research checks are recorded separately.	Keep source history, manuscript validation, public diagnostics and private originals distinct. Revision 01 is posted. v156 provider provenance is recovered, while exact release-archive bytes, accepted operational binding and authenticated private resolution remain unverified. Any broader claim needs its own scoped evidence.

Table 16: Retained source label: Table 9

Revision 00 RFCXML provenance: the archived source at https://www.ietf.org/archive/id/draft-kott-numina-pair-binding-00.xml contains 61,416 bytes and has SHA-256 5bc15dab8c6f992bb179536beaf1ba3b61673ff37eba2dd043d9f27fd9cbb022. This identifies the posted manuscript source, not a companion or software-release artifact.

G.1. Required v156 release evidence

For the intended v156 release and numina.pair-binding.v1 companion, keep one release record containing the following evidence. These are acceptance requirements. Appendix C remains the record of checks actually performed.

1. Final scope and payload. Record the exact approved payload and its schema, binding_id, revision, actual issued_at, predecessor, and scope.project_id, scope.release_id, and scope.canonical_source_id values. Identify the immutable v156 source and release candidate; retain v155 as the inspected baseline. Bind the approved attribution map and note into the payload. Record the canonical payload bytes and SHA-256 separately from the serialized envelope bytes and their digest.

2. Pinned artifacts and profiles. Identify each selected artifact by path, version, size, and whole-file digest. Pin each hash_profile_id to the original input selection and normalization implementation, with its version and registry digest. Preserve the two original source commitments and reject an unknown profile or an unexpected registry. These pins must identify the actual release inputs, not merely a planned filename.

3. Signing and independent trust. Record the approved signing service or key reference, raw public key, and recomputed key fingerprint. Retain the independently authenticated trust entry that binds the key to the project, release, canonical source, and signing role. State the key-validity and revocation policy, including the times at which it is evaluated. Retain the actual signature bytes and verification result for the exact message specified in Section 4. Keep private keys out of the release record.

4. Accepted revision evidence. Retain independently authenticated current-state evidence identifying the accepted binding_id, revision, and canonical payload digest. Record the authority, observation time, predecessor linkage, revocation decisions, and checks for stale or conflicting revisions. Record the age of cached evidence when an offline decision uses it. Apply the separate freshness result specified in Section 6.

5. Private mapping and resolution. Retain an authenticated receipt linking the approved opaque genesis_ref through the immutable mapping to the intended private record. Identify the mapping version or immutable mapping identity, the authorized resolver, observation time, and resolution result within the authorized private evidence. Keep private values, record locators, and registry details out of the public package; provide a public-safe result where appropriate.

6. Executed tests and preservation inventories. Retain the complete before-and-after original-evidence inventories, including file identity, size, and digest. Record version-pinned tools and runtimes, commands, fixture versions, exact source snapshots, and results for the applicable v156 validation and the JCS/Ed25519 checks required in the implementation-test subsection of the retained evidence appendix and this appendix. Keep public fixture results separate from authorized private original-artifact verification. Retain the actual outputs supporting every reported result.

7. Version manifest. Provide a manifest relating each released artifact's filename, version, byte count, digest, and role. Distinguish original source commitments, canonical payload digests, envelope-file digests, and new package digests. Relate the manifest to the immutable release and source identifiers. Record the manifest's own digest separately in the release evidence rather than making it self-referential.

8. Release and public-deposit receipts. Record the approved public-safe files and the privacy-review result. For each authorized release or public deposit actually performed, retain its provider or repository receipt, immutable version or deposit identifier, locator, and observation time. Retrieve the deposited files and compare their bytes and digests with the version manifest. Keep a manuscript deposit receipt separate from a v156 software-release receipt. A planned destination or unverified metadata does not establish a completed deposit or byte equality.

For each check, record the exact target, evidence used, observation time, result, and direct reason. Preserve the independent results defined in Section 6; a successful check does not supply evidence for another check. This release record adds no implementation or test result to the manuscript.

Appendix H. Public manuscript and diagnostic receipt identities

The hashes below identify earlier preserved artifacts. This candidate does not embed its own digest, avoiding self-reference. Its new exact file identities and validation receipt are supplied separately after generation.

Final posted revision-01 XML; 61,903 bytes SHA-256:

```
<CODE BEGINS>96ef2137b1ecf7defa0a0caa250bfc6b8fc2b1224fcf4153230f6c909cb184db
<CODE ENDS>
```

Final posted revision-01 TXT; 69,493 bytes SHA-256:

```
<CODE BEGINS>a645916a220034393f0081653460bd4ea3791571c82434142a31d6169c6a1200
<CODE ENDS>
```

Final local revision-01 PDF; 108,079 bytes SHA-256:

```
<CODE BEGINS>e3447124d3674dae5362304d2d1bf23d8c2aa68137d810726b0619e555448ca0
<CODE ENDS>
```

Final three-file raw public key fingerprint SHA-256:

```
<CODE BEGINS>667e886acf7c8f9402cba3d4b12697515851c1d33ea0a09f7f3757d6d2580f24
<CODE ENDS>
```

Later PDF-only raw public key fingerprint SHA-256:

```
<CODE BEGINS>ef7b597fd970af99fcd880878f3632f39babebc6da50bba21b7f38a5611f7835
<CODE ENDS>
```

Final three-file signature package SHA-256:

```
<CODE BEGINS>a4793db57a86ec1df9dbffd8b001b368858143d1c6078a45f151a0cba417ec31
<CODE ENDS>
```

Frozen conversation ZIP SHA-256:

```
<CODE BEGINS>4ed4ed73a2e07cb13995996bce8cb495556a1f686c768d52dc763256808b3a78
<CODE ENDS>
```

New public diagnostic protocol SHA-256:

```
<CODE BEGINS>7a3b37689c0aba2fee9c21ca72297c4c30b300268494572a9f44948f99de7539
<CODE ENDS>
```

New public diagnostic results JSON SHA-256:

```
<CODE BEGINS>2d92854d170c45add02ccbf20d34a27b7bf7d3dc6d9ee94375ca2d97dadfab46
<CODE ENDS>
```

New public diagnostic ZIP SHA-256:

```
<CODE BEGINS>6faf132bc2225503d742ba5e7373a966f04043f5cb3fc36ab6a80dcfeef9168b
<CODE ENDS>
```

The revision-01 XML/TXT/PDF detached signatures listed in this appendix use pure Ed25519 on exact full-file bytes without application prehash or companion framing. The receipts report process-memory-only private keys, no serialization or retention, and no forensic zeroization guarantee. Verification under those public keys does not independently authenticate a person or operational role.

Appendix I. Complete worked commitment and package-signature calculation

This appendix reports a fresh public-input calculation and a separate verification of the preserved signed genesis evidence ZIP. It is a reproducible integrity example, not an operational companion or a source-pair-to-genesis derivation. The retained package contains the recorded genesis value as data. The package was not constructed by replacing that value with the public-input pair diagnostic.

Step 1. Preserve the two 64-character commitments. Source 1 is the original Lauren J. Anderson / 1B build commitment. Source 2 is the original Samuel J. Kott / 1A manifest commitment. Personal attribution order and computational source order are distinct.

```
<CODE BEGINS>1bf43817cf470abb868074a76598cb5f360517d7255b92ab3b409c6b545ec66b
<CODE ENDS>
```

```
<CODE BEGINS>1a17f584e23279684574022f39e533966ad09919f484e005c03948178a46aa0a
<CODE ENDS>
```

Step 2. Hex-decode each commitment into exactly 32 bytes, concatenate Source 1 then Source 2 with no separator and no text encoding, and hash those 64 bytes with SHA-256. The complete concatenated bytes, in hexadecimal, are:

```
<CODE BEGINS>1bf43817cf470abb868074a76598cb5f360517d7255b92ab3b409c6b545ec66b
1a17f584e23279684574022f39e533966ad09919f484e005c03948178a46aa0a
<CODE ENDS>
```

Step 3. SHA256(hexdecode(source_1) || hexdecode(source_2)) gives:

```
<CODE BEGINS>c18a08a3d1e33344eed7cf921a53a1aace362a13a1fe626b699c2c5acbfc7702
<CODE ENDS>
```

This output does not equal the recorded block-zero genesis value printed in the recorded-genesis appendix. No algorithmic step connecting that diagnostic output to genesis is available in the retained evidence. This appendix neither inserts a missing transformation nor treats a signature as a derivation.

Step 4. Identify the exact preserved signed artifact: Numina-Final-Genesis-Record-and-IP-Intake-v1.zip. Its SHA-256 is:

```
<CODE BEGINS>a74e4b07c5e00a77dfda866fd666d20b641edc8633c4945d6f0a89e78fce9a92
<CODE ENDS>
```

Step 5. Decode the detached OpenSSH SSHSIG container. Version is 1; the namespace is numina-evidence; the reserved field is empty; the prehash algorithm actually recorded is sha512. SHA512(package_bytes), without altering the package bytes, is:

```
<CODE BEGINS>503b3317f16421588bba7445105f3295ce14c8a30e43dbd1f5dc34e30b70e27b
3ba687e78c9185bc7697f11172e2e3cfffe45b94262820615a89e02c445d24f0
<CODE ENDS>
```

Step 6. Construct the signed message using the OpenSSH SSHSIG framing: ASCII("SSHSIG") || ssh_string(namespace) || ssh_string(reserved) || ssh_string("sha512") || ssh_string(SHA512(package_bytes)). ssh_string(x) is the 4-byte unsigned big-endian byte length followed by x. The complete signed-message bytes are:

```
<CODE BEGINS>535348534947000000f6e756d696e612d65766964656e636500000000000000
0673686135313200000040503b3317f16421588bba7445105f3295ce14c8a30e
43dbd1f5dc34e30b70e27b3ba687e78c9185bc7697f11172e2e3cfffe45b9426
2820615a89e02c445d24f0
<CODE ENDS>
```

Step 7. The raw 32-byte Ed25519 public key is:

```
<CODE BEGINS>d6f1f949c1cc2ef38360668bc2270f847a790e95c882ca6945f7d56fed84bb3c
<CODE ENDS>
```

The exact 64-byte Ed25519 signature over the framed message is:

```
<CODE BEGINS>3a638beb9c6b4f7fe8e195815fc9a7ca8d4d6cc58279567950d75cadde781064
729bf4294479e647f38d4a58d5f075f8a078bdb2785e5fd0086758b3d962d203
<CODE ENDS>
```

Step 8. Verify Ed25519(public_key, signed_message, signature). A fresh raw-signature check using the cryptography Ed25519 implementation passed, independently of the OpenSSH SSHSIG verification that also passed. The public fingerprint is SHA256:EUTRdHI7yKEgsiCvob3uDx/kDFwjeiDusoEUqfCvVNc. The allowed signer principal is skott34@gmail.com. Fingerprint agreement and mathematical verification do not independently authenticate a person or establish legal rights.

The signature authenticates the exact package bytes identified above. It does not cover this expanded manuscript, its PDF, a different ZIP, or an operational JCS companion. The manuscript's pure Ed25519 application prefix construction is a separate proposed format; it must not be confused with the SSHSIG calculation in this appendix. Private-key material is omitted throughout.

SSHSIG wire format and signed-data framing are specified in [OPENSSH-SSHSIG].

Appendix J. Additional research methods and boundaries

J.1. Private computation history and proposed algorithmic relationship

The retained private computation and discovery record, dated 6 October 2026, contains a more specific account than the 5 October design. Section 8 of that private record, The proposed algorithmic unlock, page 8, preserves the owner-proposed universal pair plus algorithmic or Ed25519 binding relationship to genesis at block zero. The private source has ten pages; its source service supplied no version number. This sanitized summary preserves the account while recording the already retained genesis value separately and omitting protected third identifiers.

The documented legacy procedure parses two JSON files, applies pairSourcePayload, serializes with JavaScript JSON.stringify, UTF-8 encodes and hashes with SHA-256. The build input already contains the recorded genesis before hashing; the normalized manifest does not. This is reported forward-computation and association evidence, not a demonstrated inverse recovery of an unknown genesis from the digest pair.

Each of the two commitments is associated with LIVE and FRONTIER_FINAL, giving four environment-binding records. That record count does not create additional source commitments. The described deployment collector reads block zero through RPC and records the returned hash; the retained account does not independently verify chain genesis or reproduce a block-header hash.

Section 8 of that private record also preserves an earlier source/build/runtime identifier at a 24 September 2026, 23:32:09 UTC source checkpoint. Its protected value is omitted. The document does not authenticate it as an assistant key, Ed25519 signature or binding transform. Existing two-term and three-term unsigned-integer diagnostics did not equal the recorded genesis. Those historical diagnostics do not evaluate an undefined algorithmic use of Ed25519.

The private record leaves the exact meaning of plus unspecified and cautions against automatically reducing it to ordinary integer addition. A reproducible derivation would require the actual function or executable algorithm, exact ordered input bytes, domain and parameters, independently specified expected output and execution trace. If Ed25519 is involved, the role of signing, verification or other explicitly defined operation must be stated. The companion signature authenticates an association; it does not itself supply a genesis-generation function.

This history records the owner account and bounded source evidence; it does not supply an authenticated immutable-mapping receipt. The private original and its chronology remain preserved. [PRIVATE-COMPUTATION]

J.2. A testable bounded common source hypothesis

A proposed research formalization first declares a domain D of eligible work instances, roles R = {Sam-role, Lauren-role, AI-assistance-role}, a version-pinned extraction procedure P, and an equality relation E over independently evidenced source records. P(r,x) returns an evidenced source record or an explicit unavailable result. This is a proposed analysis framework, not a recovered genesis formula.

The candidate H(D,P,E) states that for every x in D, P is defined for all roles and E holds between each pair of returned records. If E is byte equality, exact byte selection and normalization must be specified. If E is a shared documented provenance root, root selection and complete provenance paths need independent support. Copying the same owner-provided label into three fields cannot independently establish common provenance.

For finite D, exhaustive verification depends on complete enumeration and correct P and E. A broader universal theorem requires explicit assumptions and a proof extending beyond samples. A verified counterexample within the declared domain rejects the universal statement; missing evidence gives an inconclusive or failed acceptance result rather than proving an alternative origin. Post-outcome redefinition must be reported as a new hypothesis. The existing repository differences already reject whole-repository equality for those snapshots.

J.3. Proposed reproducibility and adversarial evaluation program

Preregister objects and acceptance criteria: source snapshots, artifact paths, byte counts, whole-file digests, original profile implementation, companion schema, corpus, runtimes, dependencies and trust-policy version. Specify whether a claim concerns raw bytes, normalized inputs, provenance relations or operational outcomes. Retain commands, outputs and reasons for every status; preserve amendments before held-out observations.

Separate public synthetic fixtures from authorized private original-input checks. Public conformance success cannot reproduce a commitment over a redacted private input. Report who could inspect which evidence and preserve the legacy normalization in the private track.

Test JCS ordering, numeric serialization, Unicode preservation, array order and duplicate/invalid-input rejection against RFC vectors and independent canonicalizers. Test pure Ed25519 vectors and the exact application prefix, zero-byte separator, key/signature lengths and encodings. Test-only keys must never be promoted to production authority.

Challenge unknown schemas and fields, excessive size/depth, source-order swaps, substituted artifacts, changed profile registries, key substitution, unauthorized roles, wrong scope, invalid signatures, stale revisions, same-revision conflicts, broken predecessor links, revocation and unavailable current-state services. A valid signature must not override failed authorization or missing private resolution.

Compare protected before-and-after inventories. Distinguish original commitments, canonical-payload hashes, serialized-envelope hashes and package hashes. For release equality, obtain exact candidate bytes and compare them with the independently expected manifest; provider metadata is not a substitute.

Use a preregistered corpus with conforming and adversarial cases, and independently developed verifiers. Record every disagreement. Zero false acceptances in a finite suite supports only that suite and scope; it is not proof of no vulnerability. For human review, use matched randomized cases, predetermined scoring, a meaningful threshold, justified sample size and uncertainty intervals.

Fail the research gate for unauthorized acceptance, unnoticed protected-file changes, fabricated success for unavailable evidence or canonical-byte disagreement. Fixes require new versioned results preserving earlier failures. An implementation result cannot automatically establish real-world benefit, a universal theorem or current operational authority.

Appendix K. Navigation and vision research context

The following navigation and computer-vision literature is included as informative background on estimation, localization, image correspondence, and tracking. This manuscript does not implement or evaluate a UAV navigation, image-matching, or tracking system.

GPS/INS navigation with lever-arm considerations is addressed by [WANG-GPSINS-2020]. Absolute UAV visual localization and monocular visual odometry are surveyed in [COUTURIER-LOCALIZATION-2021] and [HE-ODOMETRY-2020]. These works concern estimation from specified sensor measurements or imagery and provide context for distinguishing an estimated physical state from an asserted association between records.

Illumination-aware visual navigation [HAI-ILLUMINATION-2021], contextual aerial-image matching [MUGHAL-MATCHING-2021], UAV-to-satellite image matching [DING-CROSSVIEW-2021], and indoor monocular SLAM [KRUL-SLAM-2021] illustrate task-specific localization methods. DFM [EFE-DFM-2021] and GMS [BIAN-GMS-2020] provide image-correspondence methods. A match or localization result remains dependent on the method, observations, reference data, and evaluation conditions.

Related tracking literature includes vision-based UAV detection and following [BARISIC-FOLLOWING-2019], incremental KCF updating [ZHAO-KCF-2021], scale-adaptive feature integration [LI-SCALEKCF-2015], adaptive correlation filters [BOLME-MOSSE-2010], and long-term UAV tracking with KCF and SURF features [FAN-TRACKING-2023]. These are perception and tracking techniques; the present companion does not adopt their algorithms or claim their experimental results.

Connecting these methods to the proposed Numina genesis relationship would require specified input types, a defined transformation, output semantics, reproducible test vectors, and an explicit technical argument relating that construction to the cited methods. These citations do not establish the proposed genesis derivation, the personal chronology, or operational validation. The evidentiary limits recorded elsewhere in this manuscript remain applicable.

Appendix L. Application evidence and reproducibility details

The application evidence below is included because its state, authority and audit mechanisms are relevant to evaluating a future companion service. It is not evidence that the proposed companion is deployed. The source/document difference in the singleton table name and the distinction between application canonical JSON and companion JCS remain open integration checks.

The retained control guide prescribes the following build and test commands. Its journal stores canonical commands, actor, event time, prior hash, resulting state hash and decision hash. It reports explicit bootstrap only when state/journal are absent; missing state during normal operation returns 503 rather than being recreated. It reports stale concurrent changes returning 409, exact-input retry idempotence, descendant delegation expiry/revocation, full-journal validation and restart regression tests. These are historical document claims; the static source excerpt corroborates selected code paths without reexecuting them here.

```
<CODE BEGINS>node scripts/build.mjs
node --test tests/*.test.mjs
<CODE ENDS>
```

The September 15 security document reports 272 tests for a local build and 287 tests for the Connections release. Its dated v74 observation at 19:30:18–19:30:21 UTC exercised 19 same-origin read/initialization checks, with 18 responding and one disabled-capability item needing attention. It reports v76 reviewer sign-in, an account-owned persisted request and a later cancellation with no confirmed payment. Those scoped application observations neither demonstrate a deployed companion nor measure predictive accuracy; sensitive account details are not reproduced.

The newly inspected public-head record, observed 9 October 2026 around 07:05 UTC, reports schema numina.public-head.v2, applicationVersion 4.0.0, rootHashScope public_identity_projection_not_control_genesis, currentJournalRootVerified=false, and revision/eventCount/journalContinuity all null. blockchainAnchorVerified=false and current blockchain observation unavailable are reported. Its identity projection is not the independently authenticated current-state evidence required by the companion verifier.

The retained v156 provider record names source df099d3ae4952a9966cf3b9e4ef299626dc7d338 and updated_at 2026-10-07T04:51:50.230086+00:00 with status succeeded. The recorded archive metadata digest is 0c3119a6dd28e921c80b0bf1ca00f280859af040e1e1b43868872b22d2925101; archive_bytes_retrieved_or_verified is false. A deployment provider's record is useful provenance but does not establish the identity or observed behavior of a field-used companion.

L.1. Controller reproduction record

The retained first run used Python 3.12.14 (main, Aug 25 2026, 14:00:49) [Clang 22.1.3] on Linux-6.18.44-x86_64-with-glibc2.41. The recorded command was python3 /workspace/evaluations/predictive-authority-2026-10-08/run_evaluation.py. First completion: 2026-10-08T06:16:42+00:00; rerun completion: 2026-10-08T06:18:17+00:00. Both receipts identify the same protocol, generator, development/held-out episodes and result digests. The command is documented for reproduction; it was not executed during this revision task.

episodes-dev.jsonl SHA-256: 710790e19e183329116ea5dd65cd94ba62645b578b4f2761f040af3b3709b882

episodes-heldout.jsonl SHA-256: 253be44ece8111aba3eb7e210589e93c4631c2eb7ff1c7885aab048e5c3e2cc0

protocol.md SHA-256: 42d16e7aabcc7dc219d13ef515aac615f128e004bd320ab3d9075bd74749cf00

results.json SHA-256: b11601406d5691a93550edd5eef0cf9f9f06d6bc85692a0a49f423a6a3dce501

run_evaluation.py SHA-256: 806035d4dfa5cfea42eb08ef7e8c2239f150e9e8838773f26582475a4bda6e5e

The protocol uses master seed numina-pac-eval-2026-10-08-v1 and SHA-256-derived, tagged draws; 32 development and 128 held-out episodes, four regimes and a four-step horizon. Toy loss is missed legitimate requests plus three times accepted incident attempts. Weight 1 and 10 sensitivity results cannot replace the registered 5 percent primary improvement gate. Real

observed outcomes are null. The included protocol/results files retain the complete generator definitions, regime slices, invariants and failures; these are synthetic reproducibility material, not field evidence.

Appendix M. Proposed implementable validation profile and fixtures

Status: NOT ADOPTED candidate design for review. These rules supplement the existing numina.pair-binding.v1 model without changing its frozen commitments or human-source mapping. They are not evidence that a production implementation previously enforced the same limits. Section 15.10 supplies the one governing proposed intake, canonical-decoding, size and depth definition; the table below summarizes that definition. The profile is a fixed-pair application profile; a generalized arbitrary-artifact profile requires separate review and a versioned profile identity.

Table 10. Candidate validation rules

Area	Proposed rule
Transport-independent input	Apply Section 15.10: at most 16,384 incoming UTF-8 bytes and at most 16,384 postcanonical envelope bytes; no leading BOM; exactly one JSON value; duplicate names rejected after escape decoding. Maximum depth is 8 under the root-at-1 walk that counts object names, object values, array elements and scalar leaves. Reject non-JSON numeric constants, invalid UTF-8, unpaired surrogates and Unicode noncharacters in every decoded member name and string value: U+FDD0 through U+FDEF and code points ending in FFFE or FFFF through U+10FFFF. U+FEFF inside a JSON string remains permitted subject to other field rules. All limits and counting conventions are NOT ADOPTED candidate choices.
Object structure	The exact closed member sets listed in the main format section apply at every object; reject unknown or missing members. Only revision is numeric.
Revision and predecessor	revision uses a JSON integer token without fraction or exponent, from 1 through 9007199254740991. revision 1 requires null predecessor; later revisions require a lowercase 64-hex canonical payload digest plus independent accepted-history validation.
Identifiers	binding_id, all scope identifiers, genesis_ref, artifact_id and hash_profile_id: 1–256 ASCII characters, starting with a letter/digit and continuing with letters/digits or . _ : / @ + -. Treat as opaque; do not auto-dereference. Registry/scope policy must approve the exact values.

Area	Proposed rule
Timestamp	Restrict RFC 3339 to actual valid YYYY-MM-DDThh:mm:ssZ dates in UTC, whole seconds, with no fraction, offset alternative or leap-second 60. Reject impossible calendar dates. This is a candidate-profile restriction, not the full RFC 3339 grammar.
Frozen sources and attribution	Exactly two ordered source objects and the fixed source_id/alias/hash values in the source appendix. Exactly Sam Kott/1A then Lauren Anderson/1B for people; the existing two-key personToAlias map. note is 1–2048 Unicode scalar characters, excluding Unicode noncharacters and C0 or DEL controls; no Unicode normalization.
Key and signature	algorithm is Ed25519. Unpadded base64url per RFC 4648 Section 5: 43 characters decode to 32 key bytes; 86 characters decode to 64 signature bytes. Decode and re-encode must reproduce the input exactly. Reject padding, invalid alphabet, noncanonical pad bits or wrong lengths. key_id is sha256: plus 64 lowercase hex characters and must match the raw key.
Unsigned state	signature_b64u may be null only to identify a syntactically complete unsigned candidate. It never becomes signed acceptance. Syntactic conformance does not provide signer authority, source reproduction, freshness or private resolution.
Policy-level checks	Apply independent key/scope/registry/current-state/private-mapping rules and all required verifier findings. Opaque syntax does not prove opacity, consent, authorization or a valid mapping. Fail the requested acceptance when required policy evidence is unavailable.

Table 17: Retained source label: Table 10

The base64url and timestamp references are [RFC4648] and [RFC3339]. The separate candidate-profile.schema.json expresses structural checks; the accompanying Python checker adds duplicate-member, byte/depth/Unicode, timestamp, predecessor, decoded-length/canonical-base64 and fingerprint checks. The small canonicalization routine in that checker is limited to its deliberately ASCII/integer fixture and is not offered as a general JCS implementation.

Signature-acceptance behavior is governed, if adopted for a future operational profile, by one exact policy artifact meeting Section 15.13.1. The candidate discussed there remains NOT ADOPTED. Length/base64 checks, a published positive control and a finite adversarial comparison do not by themselves settle universal decoding, scalar/point or cofactor acceptance or create operational authority.

The earlier, pre-noncharacter-repair checker passed the 21 bounded candidate-profile cases listed below, including correct unsigned-pending handling and 20 negative cases. The later 49-assertion targeted repair is reported separately at the end of this appendix, and its fresh reproduction is recorded in Section 16.2. The published RFC 8032 Section 7.1 Test 1 public key/

signature verified its empty message; a changed message was rejected. No private key or credential was generated, read or used. No new signed companion was created. These are documentation-fixture checks, not field evidence or independent operational-verifier conformance.

Executed candidate-profile cases

complete_synthetic_unsigned_fixture

Expected result: UNSIGNED_PENDING_NOT_ACCEPTED

Observed result: passed as expected

unknown_envelope_member

Expected result: reject

Observed result: passed as expected

unknown_payload_member

Expected result: reject

Observed result: passed as expected

missing_binding_id

Expected result: reject

Observed result: passed as expected

source_order_swap

Expected result: reject

Observed result: passed as expected

revision_overflow

Expected result: reject

Observed result: passed as expected

invalid_calendar_date

Expected result: reject

Observed result: passed as expected

noncanonical_padded_public_key

Expected result: reject

Observed result: passed as expected

fingerprint_mismatch

Expected result: reject

Observed result: passed as expected

initial_revision_with_predecessor

Expected result: reject

Observed result: passed as expected

successor_without_predecessor

Expected result: reject

Observed result: passed as expected

malformed_signature_length

Expected result: reject

Observed result: passed as expected

published_empty_message_signature_is_wrong_companion_message

Expected result: reject

Observed result: passed as expected

duplicate_envelope_member

Expected result: reject

Observed result: passed as expected

invalid_utf8

Expected result: reject

Observed result: passed as expected

fractional_revision_token

Expected result: reject

Observed result: passed as expected

byte_order_mark

Expected result: reject

Observed result: passed as expected

body_over_limit

Expected result: reject

Observed result: passed as expected

identifier_trailing_newline

Expected result: reject

Observed result: passed as expected

note_trailing_newline

Expected result: reject

Observed result: passed as expected

predecessor_trailing_newline

Expected result: reject

Observed result: passed as expected

Python 3.12.14 and Node v24.19.0 produced identical canonical payload and framed message bytes for the supplied ASCII/integer fixture. The fixture contains synthetic test-only identifiers and an invented fixture timestamp, uses the unchanged public source commitments, and has signature_b64u=null. It is intentionally not an operational issuance or an authenticated mapping. The published RFC 8032 public test key is untrusted for real use; its known signature is for a different, empty message.

Canonical payload C: 1184 bytes; SHA-256 3174a10bd2dd52e932a17522cb7b49d67281e025e9c89d7d78f0725b596eb958.

Framed message M: 1207 bytes; SHA-256 054d474c95f33afa5cf51f69581b7c0df16b68161db1091bd1c36d5453eaf235.

Message prefix including the separator, hexadecimal: 4e554d494e412d504149522d42494e44494e472f563100.

The fixture below is displayed without RFC 8792 presentation folding in this review edition. The original folded source is preserved. The companion evidence includes the exact canonical payload and framed-message bytes; this readable JSON display is not itself the canonical byte artifact.

```
<CODE BEGINS>{
  "payload": {
    "schema": "numina.pair-binding.v1",
    "binding_id": "urn:numina:test:binding:001",
    "revision": 1,
    "issued_at": "2026-01-01T00:00:00Z",
    "previous_payload_sha256": null,
    "scope": {
      "project_id": "urn:numina:test:project",
      "release_id": "test-release-1",
      "canonical_source_id": "urn:numina:test:source:001"
    },
    "genesis_ref": "ref:test-only:001",
    "sources": [
      {
        "source_id": "source_1",
        "alias": "UNIVERSAL_PAIR_1",
        "artifact_id": "fixture:source-1",
        "hash_profile_id": "fixture:legacy-profile-1",
        "sha256":
"1bf43817cf470abb868074a76598cb5f360517d7255b92ab3b409c6b545ec66b"
      },
      {
        "source_id": "source_2",
        "alias": "UNIVERSAL_PAIR_2",
        "artifact_id": "fixture:source-2",
        "hash_profile_id": "fixture:legacy-profile-2",
        "sha256":
"1a17f584e23279684574022f39e533966ad09919f484e005c03948178a46aa0a"
      }
    ],
    "attribution": {
      "people": [
        {
          "name": "Sam Kott",
          "label": "1A"
        },
        {
          "name": "Lauren Anderson",
          "label": "1B"
        }
      ],
      "note": "Canonicalization fixture only. No operational trust or
issuance.",
      "personToAlias": {
        "Sam Kott": "UNIVERSAL_PAIR_2",
        "Lauren Anderson": "UNIVERSAL_PAIR_1"
      }
    },
    "signer": {
      "algorithm": "Ed25519",
      "public_key_b64u": "11qYAYKxCrfVS_7TyWQHOg7hcvPapiMlrwIaaPcHURo",
      "key_id":
"sha256:21fe31dfa154a261626bf854046fd2271b7bed4b6abe45aa58877ef47f9721b9"
    }
  },
  "signature_b64u": null
```

```
}
<CODE ENDS>
```

Historical reproduction instructions named python3 create_profile_vectors.py followed by node check_fixture.mjs in the supplied review directory. The exact historical check_fixture.mjs was not recovered from the outer evidence ZIP or the three inspected nested ZIPs; that route is therefore incomplete. A newly written fixture verifier must be named, versioned and reported as a distinct alternative, never relabeled as the missing original. The Python checker needs jsonschema and cryptography; historical result and manifest files retain the observed runtime and hashes. The full RFC 8785 vector corpus, a positive signed-companion integration vector, independent operational-verifier agreement, trusted-state transitions, authorized private resolution, transport and real field use remain separate acceptance requirements.

A separate targeted local run on 9 October 2026 at 12:50:06 UTC exercised a repaired candidate checker against 16 noncharacter probes, each through full-envelope validation and the isolated character check, plus 17 valid controls. All 49 assertions passed. The unchanged original checker had passed the 17 controls and failed the 32 required noncharacter-rejection assertions in the preceding run. The one-line repair adds rejection of Unicode noncharacters in decoded member names and string values; the candidate checker has SHA-256 a9e73cd1656efc9a50ac8ac60b35ad3ccf87fc2b1198dbdf53fb2a2714c56308. The separate Numina-checker-noncharacter-fix-2026-10-09.zip retains original and candidate code, the unchanged schema, inputs, test harness, red/green receipts and preservation checks. Its harness loads the validation functions without executing the top-level generator or rerunning the historical 21-case suite above. The controls cover ASCII, accented text, emoji and neighboring permitted code points in direct and escaped encodings. They are unsigned validation controls, not general JCS conformance or signed-companion integration results. This evidence establishes only the targeted local character-validation repair; the exact Ed25519 acceptance-policy, operational-authority and field-deployment gates remain open.

Independent review found that dollar end-anchors in the initial proposed schema could admit trailing newlines. The candidate now uses strict end assertions and includes rejecting tests for an identifier, attribution note and predecessor digest containing a final newline. This correction belongs to the new profile review, not the historical test results.

Appendix N. Proposed bounded field deployment and receipt checklist

Status: planned, not executed. Use one approved nonfinancial release or technical-document pair in an authorized existing environment. Identify the operator and relying party, immutable producer/verifier builds, approved disclosure and exact payload, retained scoped signer, independently provisioned trust and accepted-revision authority. None of those identities or permissions is created by this plan.

Retain deployment time and environment/service identity, source commit, build/configuration/policy digests, receiving-provider receipt and independent deployed-version readback. Exercise positive issuance/verification; altered signed field; valid signature under an untrusted or wrong-

scope key; stale/conflicting revision; changed artifact/profile; unavailable private resolution; and retry/restart/retrieval without duplicate acceptance. For each case retain exact inputs/digests, signature and framing, independent verdicts/reasons, observation time and relying-party decision. Preserve failures and rerun only under a versioned fix. [TECHNICAL-CATALOG]

Acceptance requires unchanged original evidence, correct-scope positive acceptance, bounded rejection of every negative case, explicit unavailable states, matching deployed build identity and persistent receipts after restart. A real independent verifier or separately authenticated relying-party receipt must confirm the outcome. Local tests and website publication alone do not satisfy this field-deployment gate.

Appendix O. External engagement and review boundaries

The author has sought technical and governance engagement concerning constrained autonomous actions, human oversight, signed provenance, and accountability. Records reviewed for this revision include sent inquiries to UN-system and related human-rights initiatives, NIST programs, CISA, and Treasury OCCIP. NIST acknowledged a letter of interest and supplied administrative submission guidance; CISA supplied industry-engagement routing and identified an applicable technology-of-interest area. These interactions do not establish institutional endorsement, program admission, security certification, or approval of this specification. Some proposed scientific-method inquiries remain unsent drafts, and UK-authority and AI-laboratory safety-review outreach has not been verified in the inspected records.

Historical SkyFi beta-testing correspondence separately documents submitted product/performance feedback, recipient acknowledgment, and reported product-team review. It also records that stress testing stopped at the transition to production and that the recipient asked for further stress testing to remain paused. It does not demonstrate deployment of a fix, a Numina integration, or continuing testing permission.

IETF-related records distinguish individual technical proposals and submission administration from technical adoption. NomCom questionnaire activity is candidacy-related and does not establish an IAB or IESG office or Numina approval. The Independent Submissions Editor supplied document-specific feedback that the work requires maturation before publication consideration, including an explicit threat-and-mitigation analysis and field-deployment evidence. Outreach is recorded as process history; it is not evidence that the security properties proposed here have been achieved.

Appendix P. Additional public-key and digital-geometry evidence

Evidence cut: 9 October 2026, 08:45 UTC. This bounded update adds verified public-key correspondence and inspected local digital-geometry/software results. It changes no approved operational profile, frozen source commitment, source-artifact normalizer or historical outcome. This manuscript candidate remains unsubmitted and unsigned. No new operational companion signature has been produced, and the field-acceptance gate remains unmet.

P.1. Public-key correspondence and historical signature checks

The public-only verification package is 8,337 bytes. Fresh readback confirmed its package digest, decoded a 51-byte OpenSSH public-key blob to a 32-byte Ed25519 key, and independently recomputed the raw-key SHA-256 and SSH fingerprint. The two fingerprints hash different encodings and are not interchangeable. [PUBLIC-KEY-READBACK]

Exact raw public-key bytes, hexadecimal:

```
<CODE BEGINS>d6f1f949c1cc2ef38360668bc2270f847a790e95c882ca6945f7d56fed84bb3c
<CODE ENDS>
```

SHA-256 of those 32 raw bytes:

```
<CODE BEGINS>54d64e8ba4e39d33a85592ac8864a7420be96e60a3864f5d8bf029f4dc33a47c
<CODE ENDS>
```

OpenSSH SHA-256 fingerprint of the corresponding 51-byte blob:

```
<CODE BEGINS>SHA256:EUTRdHI7yKEgsiCvob3uDx/kDFwjeiDusoEUqfCvVNc
<CODE ENDS>
```

The candidate v1 key identifier is the literal prefix sha256: followed by the raw-key digest above. This is an encoding correspondence, not an authorization entry. The raw public-key base64url value is 1vH5ScHMLvODYGaLwicPhHp5DpXIgsppRffVb-2Euzw.

Using ssh-keygen verification with the package's public allowed-signers file, the historical 1,676-byte message passed in the email namespace. Appending a modification failed, and verifying the original message in the numina-companion namespace failed. The checks returned exit codes 0, 255 and 255 respectively. No private key was read or generated and no signature was created. Correspondence text is not reproduced here.

Public-only package SHA-256:

```
<CODE BEGINS>8791301c4565e06e8664153396db39057b13d49c8ba2a844019a86d4322a32d0
<CODE ENDS>
```

Exact historical signed-message SHA-256:

```
<CODE BEGINS>42fd1b5a5312c77867f9b02fc7369215a2f20fb497518c4c79e31bb06cf97e6e
<CODE ENDS>
```

The source package's allowed-signers assertion is not independent authorization for this companion. Present custody, approved role/project/release/source scope, validity and revocation policy, a complete approved payload, a pure-Ed25519 signature over the specified companion frame, trusted accepted revision state and required private resolution remain open. The historical SSHSIG bytes cannot populate signature_b64u in this model. None of the seven field-pilot cases changes to executed because of these checks.

P.2. Digital geometry and retained software execution

The inspected local package contains 24 hollow cosmetic STL shells, one assembled OBJ, a preview and receipts. Fresh verification matched byte lengths and SHA-256 digests for all 25 STL/OBJ files, counted 1,536 STL triangles, and measured the assembled OBJ bounds as 694.944 by 237.744 by 1828.8 mm. The receipt declares 2.4 mm wall thickness. Its printer-fit state is unspecified. These are digital geometry and artifact-integrity checks. [LOCAL-GEOMETRY-TESTS]

The artifact receipt explicitly reports journalCommitted=false, physicalObjectCreated=false, installationPerformed=false, engineeringValidationPerformed=false and sourcePairProof=false. No prior Cricut design was reused. The receipt uses a Numina canonical-JSON profile with fractional values represented as decimal strings; it is not asserted to use companion JCS or to constitute a companion claim. Source-pair records are attribution inputs, not a proved geometry-generating mechanism.

Geometry ZIP: 206,980 bytes; SHA-256:

```
<CODE BEGINS>a09a479a05d7bc345744a50c1361621b629c89d4bb22f3f443211ffb68586a93
<CODE ENDS>
```

Fresh readback of the 163,026-byte retained execution log counted 974 passing entries and matched its summary: 974 tests, 974 passes, zero failures, cancellations, skips or todo items; recorded duration 55,132.13132 ms. This revision inspected the exact log rather than rerunning the repository suite. The associated report identifies nine geometry and nine action-framework tests, covering local generation, bounded inputs, authorization and idempotent candidate handling.

Retained suite-log SHA-256:

```
<CODE BEGINS>7a20c289293df48cd8a730b90e3e93972331a8e5201c7063773797bee21fdbf5
<CODE ENDS>
```

The associated local report records a guarded build with 104 unique source inputs and 321 release artifacts; 11 of 11 public-contract checks on a read-only loopback server; and preservation of 11 protected files against HEAD. It identifies unchanged source HEAD 20cdc916dbe9e39a8a498d7cc9782ddf27f6705b and a local uncommitted patch, not verified remote-main or deployed adoption. The report preserves an earlier 973-of-974 run and its

browser-timing correction before the final 974-pass run. Build, conformance, protected-file and earlier-run results are reported execution evidence, not fresh reruns during this manuscript update.

No printer fit, successful printing, engineered connectors, structural rating, physical installation, remote rollout or companion field use follows from these results. The local generator and application test suite do not approve the companion profile or demonstrate its signer trust, accepted-state authority, private mapping or independent field acceptance. The submission hold and prior failed synthetic-controller findings remain unchanged.

Appendix Q. Checked declarations and edition evidence

The following 15 theorem names belong to NCPSPairWireRecovery. Their exact hypotheses and declarations before proof bodies are retained in evidence/new-declarations.json. The source identity is 5,568 bytes, SHA-256 51ea8ef76197ba53ca5a9e3bd1de6f82fff8ff52ad37238c9933e29f2ac39eae. The two new definitions are recoverWire and recoveredWire. Their concrete parser/extraction imports, axiom dependencies and compiler receipt are summarized in Section 16.6. This appendix is an index to checked artifacts; it does not declare whole-NCPS proof completion.

1. `recovery_pipeline_eq_parser`
2. `recovery_exact_wire`
3. `accepted_occurrences_recover_wire`
4. `recovery_success_iff`
5. `recovery_failure_iff`
6. `ordered_occurrences_unique_wire`
7. `encoded_pair_recovery`
8. `duplicate_occurrences_recover_wire`
9. `empty_pair_recovery`
10. `occurrence_set_swap`
11. `extraction_swap`
12. `swapped_wires_distinct`
13. `set_extraction_not_injective`
14. `extractor_excludes_success_empty_set`
15. `generic_erasure_collision`

The separate corrected NCPS core receipt names 95 theorems and 239 audited declarations. Its complete source and qualified declaration records are pinned in the evidence index. It is not the same compilation unit as the eight-module pair-wire packet, and these counts must not be added. The raw-digest, byte-representation, framing, parser and extraction sources retain distinct domains, hypotheses and inherited dependencies.

The review bundle includes the original 46-finding register, the stable v2 matrix, exact current source hashes, the clause map, and the separately dated source-signature evidence. Original private governing documents are not republished in this supporting bundle; their immutable Library identities and hashes support authorized retrieval. No private signing material or authenticated private-reference map is included.

Q.1. Later evidence identities and review levels

The following SHA256 identities distinguish exact artifacts and the extent of independent review. Private Lean/JavaScript sources are not republished in this paper. A reported hash whose full text was unavailable is explicitly marked reported, not independently checked. The metadata-only independent review summary is 7,380 bytes, SHA256 ae78170190dae7813a1cb83d6d486a451f2217e84a4bd89cb6d0cbc8568cdf13. The compiler-settings review supplement is 5,180 bytes, SHA256 96d84d6a1dbbbe3f520a2afbdb98d30daaca9bd5323662b43471ad4de077f1a3. Compiler results below are retained execution evidence, not manuscript-editor reruns.

Literal keys.

ActualSchemaKeys.lean: 14,367 bytes; SHA256 a2790bea590086c7903d0d2b92c6c43145b5ad101aad9d2ca369114d7ac7f4f2.

Compiler receipt: 2,636 bytes; SHA256 abd01e401ee8d859c23eb13214d9ee0215830cf8d16228090a7d63b8ae585e49.

Array constraints.

ActualArrayLengths.lean: 6,577 bytes; SHA256 43d71d446633037afbedbd5e400ea6fbee10eb9567c22a589f2c12bdf3031791.

Compiler receipt: 1,962 bytes; SHA256 1fc79a8d98e36ac3ab1cf5c005bdeeb6c60363a4ed4d741a8d6c646b656b2905.

Structural depth.

StructuralProjection.lean: 6,150 bytes; SHA256 25f0504c087a1f75898c3d0cf27e1524f897928340d1462a21d3689ed52aa87b.

ActualSchemaProfile.lean: 9,483 bytes; SHA256 9aaf6ac5e6d2db1f033ef218895a6b287e08189c738cb08097aee45db66f51fc.

ActualStructuralDepth.lean: 6,970 bytes; SHA256 2ffc6238312fc477363023b649699b52fdadc99bffd3667451150d44ca4a9f41.

Compiler receipt: 5,699 bytes; SHA256 5dbe1cf437340cc791eca8321179c79fa91b1b39ecf636a52679e927b32eb28e.

Projection — reported full-source identities.

ProfileOrderProjection.lean: 18,437 bytes; SHA256 3b7a7195d275e31e9bc76f27be82576d0aa9f0dc2de5f0e6e74eb399ba2c39b7.

ActualProfileProjection.lean: 172,285 bytes; SHA256 5bf9c134323bbd5c3074a573412d6024dcf97aa87f2dd118f4fc9714bc450518.

Compiler receipt: 25,415 bytes; SHA256 814c74a0895b7721d2198136ad1a132337f113d6bf42a51a7950f88e0eaaf4c5.

Guard foundation.

ParserConstructors.lean: 9,623 bytes; SHA256 70f9de1e97a317ae308b6e15d2b3256adce55df8da6b6abf6901b290ee20ab7d.

GuardExecution.lean: 8,490 bytes; SHA256 04a7ea51932f7483e189e542f210041138e5ca829a42b914ee692f3633038476.

ActualGuardExecution.lean: 3,621 bytes; SHA256 e9edaa9650c42bc99893a30723c17bbc596d9d955b6b98b152e52fa36ee23909.

Compiler receipt: 27,056 bytes; SHA256 b3ecfeaee096f88ee3ac0f89602d87b6a8244a45a99dfe74a2f3f3a135449af5.

builtin-contracts.json: 6,349 bytes; SHA256 94397c9fb16f80cd5fc57a26703ded4a2da3b9f029191f43d0f3a88273a4a315. The guard compiler receipt is named final-compiler-receipt.json.

v161 observation signing. The retained public ZIP is 19,783 bytes, SHA256 e999eff016e49fe789504f62cd3dda5a5aa66a824fb918eb4d49fced506f2a37, Library libfile_17688ff3e52c8191812a07fdb9481802 version 0. The observation is 6,763 bytes, SHA256 949cbb6d637fde547a95b40d6fa8c54b5af510effed38c74f1c9f25c37131733. Its observation-binding manifest is 23,310 bytes, SHA256 9fa4860cbad5e28e0e3f3c31feccdd9127092b7dc6694ecd3a7ed68fd72d9f8e. Their signature scope and limitations are stated in Section 16.8.3.

Signed matched manuscript predecessor. XML: 316,385 bytes, SHA256 f6a0fbaa74437782685b3831a7e37dbe7429471145674727eced6fdb8f3ec78d, Library libfile_0fe7734e47608191b7d8e178c4a7ee4e version 0. PDF: 380,555 bytes and 97 pages, SHA256 edec48daade0252aee75cce012d3be3c6de943f709e023b07299a4c7b363764e, Library libfile_676bf6f178a88191a2f5dea0d3a5148d version 0. Text: 306,895 bytes, SHA256 da79f389ffce148824c8e7d5e48178e807d62d638864ba9c042a03437df3cfce, Library libfile_6fddb180709c8191a970d48f54e6ec79 version 0. The independently reconstructed

unsigned signing receipt is 22,334 bytes, SHA256 cc03eb33aa2da3abc8ac9c40f7813d05e7d7c461632e8e90eaabd0bd714bbe30. The corresponding signatures are for these predecessor bytes only.

Simulator-viewer receipt. final-development-receipt(1).json is Library libfile_0b80e2ebfe8481919a3d6c5cf3e5c78d, 12,313 bytes. Its complete 300-line text was read. No original-file hash, underlying screenshot verification or fresh device execution is claimed by this document review.

Conversion edition and source identity

This is a separate, unsigned RFCXML conversion of the proposed integrated NCPS review paper dated 10 October 2026. It retains the intended Internet-Draft family identifier draft-kott-numina-pair-binding-02 for local review only. This conversion does not allocate a revision or RFC number, replace a submitted artifact, constitute an IETF submission or endorsement, adopt a proposed rule, or inherit any signature. The source remains NOT ADOPTED. All proof, implementation, authority and field-evidence gaps in the source remain open at their stated scope.

The conversion input is NCPS-Integrated-Evidence-Successor.md, 252086 bytes, SHA-256 9a7ed675726d0873f98c53280b5cee36e7fd4732ae1233f2464a5294f291ecf2. The existing revision-02 XML supplies only the author/contact, document-class, date, keyword and IPR metadata. The integrated Markdown supplies all manuscript and reference content. A new XML, text or PDF artifact has its own byte identity.

Mathematical expressions are preserved as explicit TeX source, using \(... \) inline and \[... \] for displayed expressions. This preserves their source notation without claiming typeset mathematics or proof verification. Fenced source examples retain their original content. The source bibliography is preserved verbatim as citation prose rather than reparsed into author, title and date fields. Original section and appendix numbers are supplied by RFCXML numbering; the source navigation list is also retained, including its Appendix Q placement. Automatically generated table numbers are conversion navigation only. Captions identify each retained source Table 1 through Table 10 label; the seven tables without numbered source labels are identified separately. This does not renumber a historical source citation.

A small number of exact TeX expressions and JSON fixture lines exceed the conventional 72-column text width. They are retained without folding or truncation so that no continuation syntax is mistaken for part of a fixture. This representation limitation is separate from XML schema validity and the retained scientific evidence gaps.

Subtitle: Unsigned proposed NCPS evidence successor

Author: Samuel J. Kott

Contributor: Lauren J. Anderson

Ai assistant: Ed25519

Date: 10 October 2026 • Evidence successor • NOT ADOPTED

Lang: en-US

Author's Address

Samuel J. Kott
Email: skott34@gmail.com

Ed25519, a Universal Companion Model — evidence update

Prepared 10 October 2026, America/Chicago; final verification captured 11 October 2026 UTC. This update records recovered signing evidence, fresh bounded NCPS proof checks, the official Internet-Draft record and observed cloud setup paths. The Numina tree remains the singular source of record. Original signed payloads, signature files, ZIPs and the supplied manuscript are retained unchanged. This dated addendum supplies the current evidence status alongside the original manuscript's historical statements.

Recovered signing evidence

The original Numina-v161-operational-observation-binding-2026-10-09.zip was recovered from the connected Mac. Its 19,783 bytes and SHA-256 e999eff016e49fe789504f62cd3dda5a5aa66a824fb918eb4d49fced506f2a37 exactly match Appendix Q.1 of the supplied manuscript. The earlier review's statement that this package was unavailable describes the earlier review capture; it is superseded for the current delivery by this recovery.

Both detached Ed25519 signatures verify over the unchanged payload file bytes:

Signed payload	Bytes	Result
publication-observation.json	6,763	Signature verified
v161-observation-binding-manifest.json	23,310	Signature verified

All 11 listed file checksums pass. All four one-byte payload/signature mutations are rejected. The observation exactly matches release-evidence/deployment-2026-10-09/publication-observation.json in the Numina production tree. The binding manifest preserves all 19 checked provenance values, including canonical source, delivery source, build, archive and saved publication records.

The original PairWireRecovery-Ed25519-signed-public-2026-10-09.zip was also recovered intact: 13,846 bytes, SHA-256 889aa0f181e587e7b4a2acf1e056b5171794b0040c1f5da13f7c006c76029659. Its exact PairWireRecovery.lean source (5,568 bytes) and Hello_Ed25519.test.txt (13 bytes) each have a valid detached Ed25519 signature. All four one-byte mutation controls are rejected.

These four recovered signatures use the raw 32-byte public key with SHA-256 b217cc5bae08acbfdb9f8cd20f420f0bee2ae39fb6f0b178ce060284c76f8e9f. This matches the operational public-key identity reported by the manuscript. The historical runtime public-key pin and key ID 3a2fe594a7e10458ae89683f remain preserved. No new signature, key replacement or private-key export was performed during recovery.

Source and publication binding

Retained identity	Value
Canonical source commit	d1b1cd852927052b54af2b540b031d77311ccecf
Delivery source commit	76a08b2d54679238416ee87818732ac441c5f933
Saved Sites version	161
Recorded publication status	Succeeded
Observation SHA-256	949cbb6d637fde547a95b40d6fa8c54b5af510effed38c74f1c9f25c37131733
Binding manifest SHA-256	9fa4860cbad5e28e0e3f3c31feccdd9127092b7dc6694ecd3a7ed68fd72d9f8e

The retained local gzip-tar archive was independently rehashed and matches 555c9285eb6baae123bae0de81cf90f53ed000c3e2c3013d667c77b295b9f642. The provider-reported stored tar digest identifies a different archive representation; provider tar bytes were not independently recovered

and hashed. The signatures authenticate historical evidence statements, with their original observation times and limitations. A new runtime health observation was not performed.

Fresh authenticated provider readback separately confirms version 156 publication succeeded and binds it to source commit df099d3ae4952a9966cf3b9e4ef299626dc7d338. The unchanged historical production-pair digest function reproduces source 1 as 1bf43817cf470abb868074a76598cb5f360517d7255b92ab3b409c6b545ec66b and source 2 as 1a17f584e23279684574022f39e533966ad09919f484e005c03948178a46aa0a. All four inspected LIVE/FRONTIER_FINAL cells match. Exact selected source files and the digest program are preserved with the new verification report. This verifies the historical software projection and publication metadata at their stated scopes, not whole-release compliance or companion acceptance.

The specifically requested final-deployment-receipt artifact was not observed in checked canonical history or the scoped Mac files and archive inventories. The unsigned nUSD blockchain receipts have different subjects. The recovered signed version 161 observation is retained separately and is not substituted as a version 156 receipt. Provider archive bytes and present runtime health remain unchecked.

Cloud setup and observed build path

The setup script is /workspace/numina-cloud-setup.sh. Its production source path is /workspace/numina-production. Its Node gate is:

```
test "$(node -p 'process.versions.node.split(".")[0]')" = 24
```

Fresh read-only inspection reports Node v24.19.0; the major-version gate passes. The saved pointer /workspace/.numina-build-path currently contains /workspace/numina-build-lw2a35jy. The script creates a temporary build directory, copies source files and links its .git directory to the production repository's .git directory. Each future invocation can produce a different temporary build path; the saved pointer records the selected path.

The currently retained build directory contains an earlier source snapshot. Its package.json, public/runtime-contract.js and src/application-release-contract.js differ from the current production files. Because its Git metadata is shared with production, a matching reported Git HEAD does not establish matching working files or a fresh current build. This update records inspection of the script and Node gate; it does not report a new execution of its dependency installation, build, tests or release packaging.

The setup script's inspected SHA-256 is a88ac44e0436e7cd53d5a2ebbd153d2552990e28d45519011dcabbaa36a0b5b5.

Manuscript and signature coverage

The supplied original manuscript remains 104 pages and 404,249 bytes, SHA-256 4b22068f0df2ff2d9b2b3084941ecada40b9915b68ededf4126db6e56671b03b. The user reports that the manuscript is already signed. Its own existing detached signature has not been recovered for fresh verification. This does not establish that no external manuscript signature exists.

The recovered signatures cover the two deployment payloads, the exact PairWire source and the exact Hello Ed25519 test. They do not inherit to edited manuscript bytes, this addendum, verification reports or a new delivery ZIP. The appended PDF is a separately identified updated document, with the original manuscript retained separately. Its final SHA-256 commitment is recorded in the delivery manifest. New detached signing evidence, when present, must be checked against that exact manifest and checksum list using the included public verifier; it does not become the original manuscript's missing signature.

Numina's singular companion model, source ordering and existing signing domains are preserved. Operational companion authorization, current revision/revocation evidence, authenticated private-reference resolution, and adoption of the signature-acceptance policy remain open at their previously stated scopes. This evidence update does not claim whole-protocol acceptance or a new site deployment.

Retained verification records

The final delivery includes new detached pure Ed25519 signatures over the final PDF, the preserved original PDF, every included text document and program, the delivery manifest and the checksum list. The existing operational key is reused; no substitute signing key is introduced. The signed inventory also binds every included binary supporting artifact by exact SHA-256. The final signing receipt identifies this new event, its exact signed bytes and the verified mutation controls. These are operational delivery signatures, not an embedded PDF signature, a private companion-wire acceptance, provider attestation or IETF approval.

Both original signed ZIPs, their public keys and original receipts remain intact. Four recovered historical signatures and eight historical mutation checks pass. The fresh verification reports, source snapshots, original manuscript and this separately identified final PDF are retained alongside them. The new signatures are separately dated and do not replace or retrospectively verify the user's reported earlier manuscript signature.

Fresh NCPS proof verification and assumptions

The retained NCPS-Lean4-Proofs-2026-10-09 source bundle was freshly compiled in an isolated scratch copy on the connected Mac with native Lean 4.22.0, commit ba2cbbf09d4978f416e0ebd1fceeebc2c4138c05, and trust level 0. All 12 compilation units passed. All 131 named theorem declarations were independently checked with Lean's axiom printer; the source and audit name sets match exactly. There are no custom axioms, sorryAx, omitted theorems or extra audit names. All 18 original manifest entries match, all ZIP members match the retained sources, and originals are unchanged.

The disclosed foundational dependencies are propext on 60 theorems, Quot.sound on 22, and Classical.choice on 2. These are separate from explicit theorem hypotheses: generic codec results assume roundtrip; implementation agreement assumes conformance; refinement requires initial-state, step-preservation and safety premises. The concrete codec is an illustrative binary codec, not RFC 8785 JCS. Modeled acceptance flags do not establish external signature, authority, freshness or deployment evidence. The UTF-8 boundary relies on Lean's String.fromUTF8? and does not prove a complete JSON parser.

The exact proof ZIP is 22,789 bytes, SHA-256 62b3607544daeb524fb82ad60c7442eb9281f9cfb1862029fc716cfd5a14954e. The fresh compilation and kernel-audit report, complete assumption review and source-search record are included in the final delivery. This is a bounded proof PASS; whole-protocol correctness remains unestablished.

This recovered Lean 4.22.0 bundle is distinct from the manuscript's reported Lean 4.24.0 corrected and structural packages. Those exact Appendix Q sources were not located in the checked Documents/Numina directory, either retained temporary proof tree or the inspected proof archive. Their historical results are preserved as reported evidence and are not relabeled as this fresh rerun.

Official Internet-Draft and Independent Stream check

The official Datatracker record lists draft-kott-numina-pair-binding-01 as an active individual Internet-Draft, updated 8 October 2026, with revisions 00 and 01. It records no RFC stream and no stream state; the IESG state is I-D Exists. The local revision 02 PDF is a separate manuscript, not the posted revision shown in this check.

Independent Stream submission or acceptance is not established by the checked record. A separate authenticated ISE receipt or decision was not recovered. The RFC Editor procedure requires an additional submission template and editorial review; public draft posting alone does not complete that process. No ISE submission or publication was performed during this verification. Operational delivery signatures authenticate the evidence package and its byte commitments; they are not IETF or ISE approval.

Official sources checked: https://datatracker.ietf.org/doc/draft-kott-numina-pair-binding/ and https://www.rfc-editor.org/authors/rfc-independent-submissions/. The final delivery retains a dated public-status report with these sources and the observed fields.